Pusan National University

Computer Science and Engineering

Technical Report 2025-09

RAG를 활용한 컨테이너 기반 마이크로서비스 운영 환경 관리 지원 시스템

201914116 김휘수

202055645 신세환

202255663 설종환

지도교수: 염근혁

목 차

1. 서론 4

1.1. 연구 배경 4

1.2. 기존 문제점 5

1.3. 연구 목표 5

2. 연구 배경 6

2.1. 요구사항 분석 6

2.1.1. 기능적 요구 사항 6

2.1.2. 비기능적 요구 사항 9

2.2. 선행기술 조사 10

2.2.1. Docker 10

2.2.2. Kubernetes 12

2.2.3. Filebeat 14

2.2.4. Logstash 16

2.2.5. Metricbeat 17

2.2.6. Elasticsearch 18

2.2.7. Flask 20

2.2.8. Langchain 21

2.2.9. LangSmith 23

2.2.10. Spring Boot 25

3. 연구 내용 26

3.1. 설계 상세화 26

3.1.1. 유스케이스 다이어그램 26

3.1.2. 유스케이스 명세 27

3.1.3. 클래스 다이어그램 46

3.1.4. 시퀀스 다이어그램 52
3.2. 시스템 구성 73
3.2.1. 시스템 구성도 73
3.2.2. 개발 환경 74
3.3. RAG를 활용한 컨테이너 기반 마이크로서비스 운영 환경 관리 지원 시스템 75
3.3.1. 조직 계정 관리 75
3.3.2. MSA 프로젝트 관리 76
3.3.3. 조직 내부 데이터 관리 77
3.3.4. RAG 기반 MSA 애플리케이션 배포 지원 81
3.3.5. 서비스 운영 모니터링 88
4. 연구 결과 분석 및 평가 91
4.1. 사례 연구 내용 91
4.1.1. 선물하기 서비스의 마이크로서비스 구성 92
4.1.2. 선물하기 서비스 운영 조직 내부 데이터 구성 92
4.1.3. 제안된 시스템을 활용한 배포 및 운영 시나리오 95
4.2. 정량평가 103
4.2.1. 비교군 설정 103
4.2.2. 실험 결과 및 분석 104
4.3. 정성평가 107
5. 결론 및 향후 연구 방향 109
5.1. 결론 109
5.2. 향후 연구 방향 109
6. 참고 문헌 110

1. 서론

1.1. 연구 배경

마이크로서비스 아키텍처(MSA, Microservice Architecture)[1]는 시스템을 소규모의 독립적인 기능 요소로 구분하여 서비스를 배포할 수 있게 만드는 구조이다. MSA는 주로 컨테이너 오케스트레이션 기술(예: Kubernetes, Docker Swarm)을 통해 구현되며, 컨테이너 오케스트레이션 기술을 활용하여 마이크로서비스 배포를 수행하기 위해서는 배포 명세(Kubernetes Template, Dockerfile 등)를 활용하여 컨테이너가 동작하는 환경을 구축할 필요가 있다.

컨테이너 기반 마이크로서비스 아키텍처는 마이크로서비스 간의 약결합[2]을 통해 기능의 흐름을 구축하게 된다. 이러한 약결합을 수행하기 위해서는 네트워크 인터페이스 명세, 컨테이너 이미지 명세 등의 마이크로서비스 배포 명세를 정확하게 작성하고 배포해야 한다. 그러나 컨테이너 오케스트레이션 기술은 컨테이너 오케스트레이션 기술에 대한 이해도가 부족한 사용자가 마이크로서비스 배포 명세에 대한 구조를 명확히 파악하여 작성하기는 어렵다. 또한 마이크로서비스가 배포된 이후, 운영 중인 마이크로서비스에서 발생하는 오류를 파악하고 해소하기 위한 방안을 마련하기 어렵다. 이러한 기술적 비전문가들의 마이크로서비스 측면의 배포 및 운영을 보조하기 위하여 LLM과 같은 도구를 활용[3]하여 배포 및 운영을 지원할 수 있다. . 그러나 이러한 LLM과 같은 개발 보조 도구를 활용하더라도 서비스 운영에 필수적인 요소인 조직 내부 데이터는 서비스 운영 주체의 요구사항에 따라 관리되는 비공개 데이터이므로 접근하기 어렵다. 이러한 데이터는 프로젝트 상황에 따라 지속적으로 갱신되는데, 실제 배포 명세를 작성하는 조직 내 배포 담당자가 대규모 마이크로서비스에 대해 배포 명세 작성에 필요한 모든 도메인 지식을 보유하기 어려우므로 요구사항에 부합하는 배포 명세를 구성하기 어렵다는 문제가 있다. 추가적으로, 배포 명세를 최초에 작성할 때는 컨테이너 기반 마이크로서비스 배포 이전에 알 수 없는 동적 운영 정보(CPU/메모리 사용량 등)를 식별하여 배포 명세를 개선할 수 있는 체계 또한 부족하다.

따라서, 본 과제에서는 RAG(Retrieval-Augmented Generation)를 활용한 컨테이너 기반 마이크로서비스의 운영 지원 시스템을 제안한다. RAG는 신뢰할 수 있는 외부 데이터베이스를 참조 및 활용하여 LLM이 생성하는 답변의 정확도 및 활용성을 향상시키는 기법이다[4]. 본 과제에서는 사용자의 자연어 기반의 질의가 주어질 때 지속적으로 갱신되는 조직 내부 데이터를 활용하여 RAG 기반의 사용자의 요구에 부합하는 배포 명세 생성과 동적 운영 정보를 기반으로 명세 수정안을 제공하는 컨테이너 기반 마이크로서비스 운영 지원 기술을 목표로 한다.

1.2. 기존 문제점

- 조직 내부 데이터를 반영한 배포 명세 생성의 어려움

컨테이너 기반 마이크로서비스 초기 배포 시에는 조직 내부 데이터가 정확히 반영되어야 한다. 그러나 이러한 내부 데이터는 외부 도구에서 접근하기 어려울 뿐 아니라 프로젝트 상황에 따라 지속적으로 변경되므로, 배포 담당자가 모든 정보를 완전히 숙지하기 어렵다. 그 결과, 조직의 요구 사항에 완전히 부합하는 배포 명세를 작성하는 과정은 높은 복잡성을 띠며, 이는 배포 효율성을 저해하는 주요 원인으로 작용한다.

- LLM을 활용한 기존 접근 방법의 한계로 인한 문제

마이크로서비스 배포·운영에는 지속적으로 변하는 조직 내부 데이터가 신속히 반영되어야 한다. 하지만 기존의 LLM을 활용한 접근 방식인 파인튜닝은 한계를 지닌다. 파인튜닝은 특정 도메인에 특화된 모델을 이용할 수 있다는 장점이 있지만, 새 데이터마다 재학습이 필요해 시간·비용 부담과 반영 지연이 발생한다[5]. 또한, 생성된 답변의 출처를 제시하지 못하기 때문에 정확성이 중요한 배포 명세 작성 지원에 적합하지 않다.

- 오류 로그 및 리소스 사용량을 반영한 배포 명세 개선 기능 부재

기존 시스템들(예: Prometheus, Grafana 등)은 대부분 리소스 상태 진단 기능은 제공하지만, 운영 중 수집되는 마이크로서비스의 에러 로그와 같은 운영 정보나 CPU/메모리 사용량 등의 리소스 메트릭을 반영하여 배포 명세의 구체적인 개선 방법을 제공하는 기능은 제공하지 않는다.

1.3. 연구 목표

- 배포 명세 작성을 위한 조직 내부 데이터의 정의와 분류 기준 제시
- RAG 기반 배포 명세 생성·수정 기능을 제공하는 MSA 운영 지원 시스템 구축
- 컨테이너 에러 로그 및 리소스 메트릭 수집을 통한 배포 명세 동적 개선 방안 제시 기술 구현

2. 연구 배경

2.1. 요구사항 분석

2.1.1. 기능적 요구 사항

표 1은 컨테이너 기반 마이크로서비스 운영 환경 관리 지원 시스템의 기능적 요구사항이다.

표 1 기능적 요구사항

기능		설명
조직 계정 관리	조직 계정 생성	시스템은 사용자가 입력한 ID, 비밀번호, SSH 인증 키, IP 주소, AI 서비스 API키 정보를 받아 조직 계정을 생성하고 관련 데이터를 안전하게 저장해야 한다.
	조직 계정 삭제	시스템은 특정 조직 계정에 대한 삭제 요청 시, 해당 계정 및 연관된 모든 데이터(프로젝트, AI 서비스 키 등)을 영구적으로 삭제해야 한다.
	조직 계정 정보 수정	시스템은 사용자가 조직 계정의 비밀번호, SSH 접속 정보, API 키를 수정할 수 있는 인터페이스를 제공하고, 변경 사항을 시스템에 반영해야 한다.
	시스템 로그인	시스템은 조직 계정 정보를 기반으로 사용자를 인증하고, 인증된 사용자에게만 시스템의 주요 기능에 대한 접근을 허용해야 한다.
MSA 프로젝트 관리	MSA 프로젝트 생성	시스템은 SSH 키, pem 파일, IP 주소, 접속 계정명 등 MSA 애플리케이션 접속 정보를 입력받아 신규 프로젝트를 등록하고 관리할 수 있어야 한다.
	MSA 프로젝트 삭제	시스템은 특정 프로젝트에 대한 삭제 요청 시, 해당 프로젝트와 관련된 모든 정보(배포 이력, 조직 내부 데이터 등)를 완전히 삭제할 수 있어야 한다.

	MSA 애플리케이션 원격 접속	시스템은 저장된 프로젝트 정보(SSH 키, pem 파일, IP, 접속 계정명)를 활용하여 프로젝트에 등록되어있는 MSA 애플리케이션 환경에 원격으로 접속하여 애플리케이션 제어 기능을 제공해야 한다.
조직 내부 데이터 관리	**조직 내부 데이터 업로드**	시스템은 사용자가 업로드한 조직 내부 데이터 내의 문서를 추출하고, RAG가 유사도 기반 검색을 할 수 있도록 텍스트를 벡터 임베딩하여 Elasticsearch에 저장해야 한다.
	조직 내부 데이터 삭제	시스템은 사용자의 요청에 따라 파일 이름을 기준으로 Elasticsearch에 저장된 특정 조직 내부 데이터를 삭제할 수 있어야 한다.
RAG 기반 MSA 애플리케이션 배포 지원	**배포 명세 생성**	시스템은 사용자의 자연어 질의를 입력받아, Elasticsearch에 저장된 조직 내부 데이터를 검색하여 배포 명세 생성에 필요한 정보(환경변수, 서비스 포트, 서비스 간 의존성 등)를 반영한 배포 명세를 생성할 수 있어야 한다.
	배포 명세 오류 수정	시스템은 사용자가 제출한 자연어 질의와 배포 명세를 입력받아, Elasticsearch에 저장된 조직 내부 데이터를 검색하여 필수 필드 누락, 운영 정책 위반 등의 오류를 진단하고 수정된 배포 명세를 제안할 수 있어야 한다.
	배포 지원 이력 조회	시스템은 RAG 기반 질의응답 기록을 모두 저장하고, 사용자에게 과거 이력 목록 및 상세 내용을 조회할 수 있는 기능을 제공해야 한다.
	배포 지원 이력 삭제	시스템은 사용자가 선택한 특정 RAG 채팅 기록을 삭제할 수 있는 기능을 제공해야 한다.
서비스 운영 모니터링	**로그 수집기 배포**	시스템은 시스템의 Elasticsearch에 로그를 전송하는 로그 수집기 배포 파일(Filebeat, Metricbeat, Logstash)을 사용자에게 제공할 수 있어야 한다.
	기존 배포 명세 업로드	시스템은 시스템 모니터링 정보로 활용하기 위해, 사용자가 기존 MSA 애플리케이션에 적용된 배포 명세를 업로드 할 수 있는 기능을 제공해야 한다

	기존 배포 명세 삭제	시스템은 사용자가 업로드한 기존 배포 명세를 삭제할 수 있는 기능을 제공해야 한다.
	에러 로그 모니터링	시스템은 Elasticsearch로부터 주기적으로 에러 로그를 수집하고, 조직 내부 데이터에 명시된 해결 방안을 기반으로 개선된 배포 파일을 생성하여 사용자에게 제공해야 한다.
	리소스 메트릭 모니터링	시스템은 Elasticsearch로부터 주기적으로 리소스 메트릭(CPU, 메모리 사용량)을 수집하고, 조직 내부 데이터의 리소스 정책에 맞춰 최적화된 배포 명세 파일을 생성하여 사용자에게 제공해야 한다.
배포 명세 개선안 관리	**배포 명세 개선안 조회**	시스템은 수집한 로그 및 메트릭 정보를 기반으로 생성한 배포 명세 개선안을 사용자가 조회할 수 있도록 해야 한다.
	배포 명세 개선안 삭제	시스템은 사용자의 요청에 따라 특정 배포 명세 개선안을 삭제할 수 있어야 한다.

2.1.2. 비기능적 요구 사항

표 2는 컨테이너 기반 마이크로서비스 운영 환경 관리 지원 시스템의 비기능적 요구사항이다.

표 2 비기능적 요구사항

요건	설명
성능	- 시스템은 로그 이상 탐지 주기를 최대 3 분 이내로 유지해야 함. - 시스템이 생성한 배포 명세를 이용한 배포의 변경 실패율(Change Failure Rate)은 DORA Elite 등급 기준인 15% 미만(이는 배포 성공률 85% 이상에 해당)[6]을 충족해야 함.
신뢰성	- 시스템은 사용자의 입력 오류(유효하지 않은 API 키 또는 접근 자격)에 대해서도 오류 없이 예외를 처리하고 사용자에게 유의미한 오류 메시지를 제공해야 함.
보안성	- 시스템은 비공개 데이터, 로그 등 민감 데이터의 전송 구간 데이터 보안을 위해 TLS 1.2 이상을 사용하는 HTTPS 를 통해 통신을 암호화해야 함.
안전성	- 시스템을 구성하는 각 핵심 모듈(계정 관리, 프로젝트 관리, RAG 배포 지원, 동적 모니터링)은 장애가 발생하더라도 다른 모듈의 서비스 가용성에 영향을 주지 않아야 함.
가용성	- 시스템은 정기 점검이나 예기치 못한 장애 발생 시에도 서비스 중단 시간을 최소화할 수 있도록 자동 복구 또는 10분 내 수동 조치가 가능해야 함.

2.2. 선행기술 조사

2.2.1. Docker

Docker는 애플리케이션 실행에 필요한 환경을 하나의 컨테이너 이미지로 패키징하여 어디서든 동일한 방식으로 실행되도록 보장하는 오픈소스 기반 컨테이너 플랫폼이다[7]. 운영체제 레벨에서 격리된 환경을 제공하여 개발 환경과 운영 환경 간의 차이를 제거하고 배포 및 관리를 단순화한다.

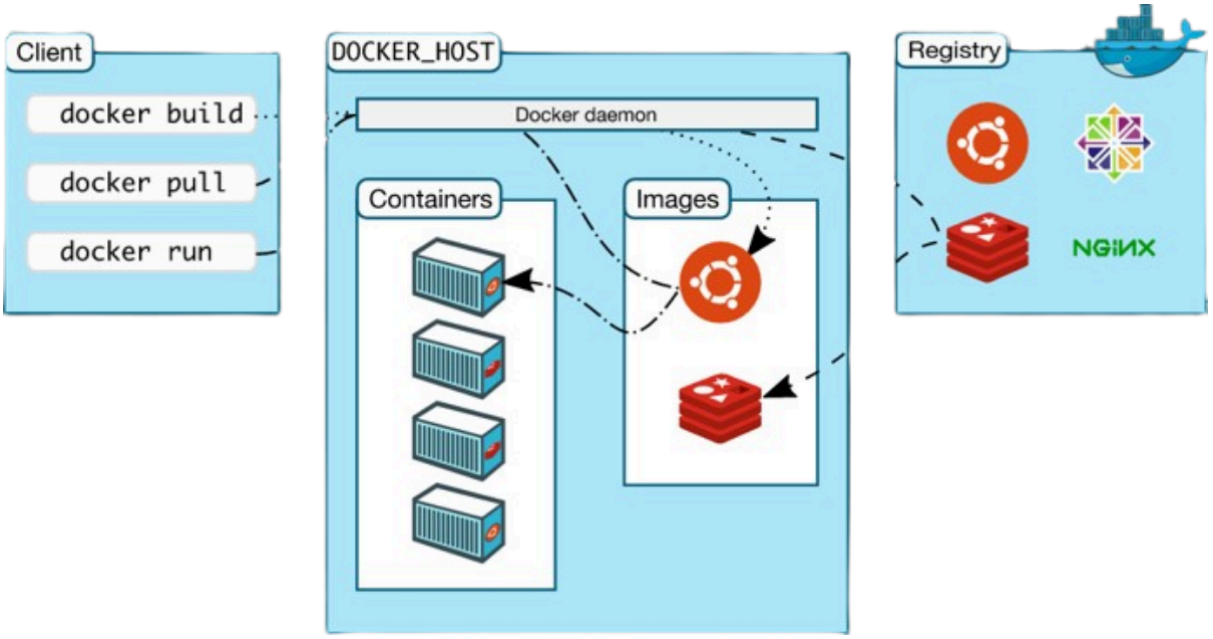


그림 1 Docker 아키텍처

그림 1은 Docker 시스템 아키텍처의 주요 구성 요소를 보여준다. 각 구성 요소의 기능은 다음과 같다.

i) Docker Daemon

Docker API 요청을 수신하고 이미지, 컨테이너, 네트워크 및 볼륨과 같은 Docker 개체를 관리한다.

ii) Docker Client

사용자가 Docker와 상호 작용하기 위한 미들웨어로 이미지 생성, 컨테이너 생성 및 실행과 같은 명령어들을 실행할 수 있다.

iii) Docker Desktop

Docker 데스크탑 지원용 프로그램이다. Docker Daemon, Docker Client와 쿠버네티스(Kubernetes) 와 같은 다양한 응용프로그램들이 함께 탑재되어 있다.

iv) Docker Registry

Docker 이미지를 저장하는 레지스트리로 Docker Desktop 설치 시 개인 레지스트리가 함께 설치되며, 공용 레지스트리인 Docker hub에서 이미지 검색을 통하여 이미지를 다운 받을 수 있다.

v) Docker Object

Docker를 사용하는 데 있어서 필요한 이미지, 컨테이너, 네트워크, 볼륨, 플러그인들이 여기에 속한다.

vi) Docker Image

Docker 컨테이너를 만드는데 사용되는 템플릿이다. 다른 이미지에 사용자 설정을 추가하여 사용할 수 있는 구성 세부 정보 설치 기능을 수행할 수 있다.

vii) Container

이미지의 실행 가능한 인스턴스로 Docker API 혹은 CLI를 통하여 생성, 중지, 시작, 이동, 삭제가 가능하다.

2.2.2. Kubernetes

Kubernetes는 컨테이너화된 애플리케이션을 자동으로 배포, 관리, 확장 및 복구하는 오픈소스 컨테이너 오케스트레이션 플랫폼이다[8]. 복잡한 마이크로서비스 환경을 안정적으로 운영할 수 있도록 설계되었다.

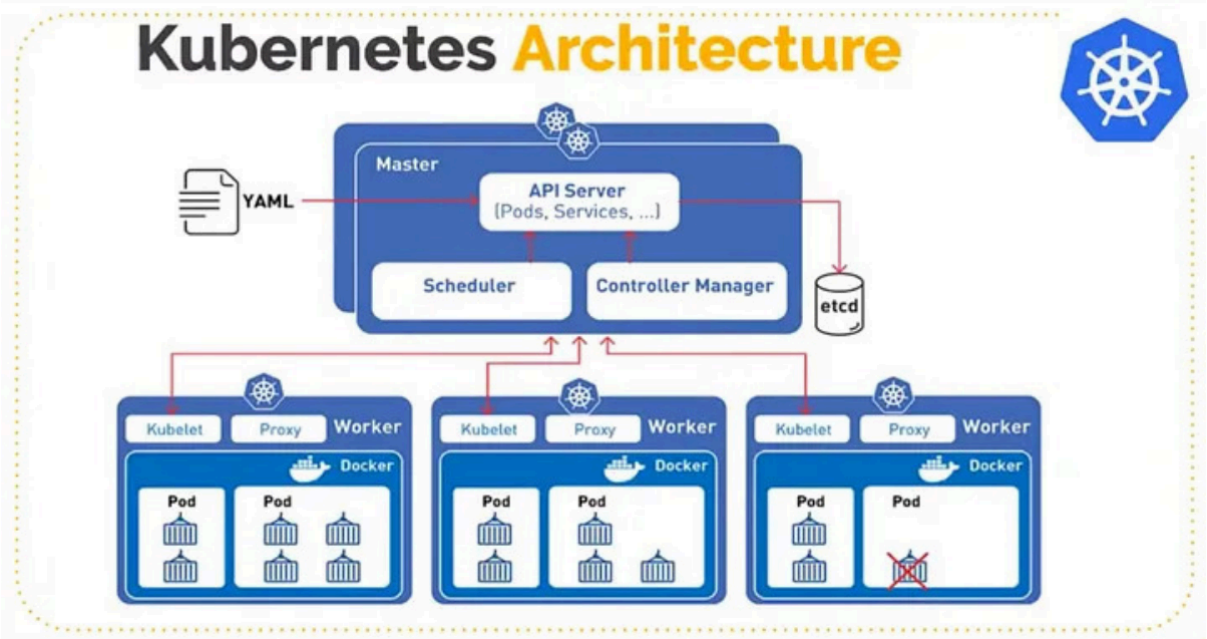


그림 2 Kubernetes 아키텍처

그림 2는 Kubernetes 아키텍처를 나타낸 그림이다. 각 구성 요소의 기능은 다음과 같다.

i) Master Node

클러스터의 중앙 제어 장치 역할을 한다. 전체 클러스터의 상태를 추적하고, 각 워커 노드에 작업을 할당하는 기능을 담당한다.

1) API Server: 클러스터 외부와 내부에서 발생하는 모든 요청을 수신하고 처리하는 진입점이다.

2) Scheduler: 새로 생성된 파드를 어떤 워커 노드에 배치할지 결정한다.

3) Controller Manager: 파드 수 유지, 노드 상태 확인 등 다양한 제어 루프를 수행한다.

4) etcd: 클러스터의 상태 정보를 저장하는 키 - 값 저장소이다.

ii) Worker Node

컨테이너가 배포되어 실행되는 노드다.

1) kublet: 마스터 노드로부터 명령을 수신해 파드를 생성하고 상태를 모니터링한다.

2) kube-proxy: 파드 또는 외부 네트워크 간의 통신을 중계한다.

3) Container Runtime: Docker 등 실제 컨테이너를 실행하는 런타임이다.

iii) Pod

가장 작은 배포 단위로, 하나 이상의 컨테이너가 실행되는 단위를 말한다. 동일한 네트워크 네임스페이스와 볼륨을 공유한다.

iv) Deployment

Pod의 상태(예: 레플리카 수, 컨테이너 이미지 버전 등)를 사용자가 정의하면, Kubernetes가 이를 자동으로 관리하는 리소스이다. 실제 애플리케이션이 지속적으로 안정적으로 운영되도록 한다.

v) Service

동적으로 생성되고 소멸하는 파드들을 외부에서 고정된 주소로 접근할 수 있도록 해주는 추상화 객체이다. 클러스터 외부에서 내부 Pod에 접근할 수 있도록 라우팅 역할을 수행한다.

vi) ConfigMap

애플리케이션의 설정 데이터를 키-값 쌍으로 저장하는 리소스이다. 컨테이너 이미지와 설정을 분리해서 애플리케이션의 이식성을 높인다. Pod에서는 환경 변수나 볼륨 마운트를 통해 ConfigMap의 데이터를 사용할 수 있다.

vii) Secret

비밀번호, API 키, 인증서 등 민감한 데이터를 저장하는 데 사용되는 리소스이다. 데이터는 Base64로 인코딩되어 저장되며, ConfigMap과 유사하게 Pod에 환경 변수나 볼륨으로 전달될 수 있다. 이를 통해 민감한 정보를 안전하게 관리할 수 있다.

2.2.3. Filebeat

Filebeat는 애플리케이션 로그, 사용자 정의 로그 등 다양한 로그 파일을 실시간으로 수집하여 중앙 시스템으로 전송하는 경량 로그 수집 도구이다. 각 서버에 에이전트 형태로 설치되어 지정된 로그 파일을 Logstash 또는 Elasticsearch로 전달한다. 효율적인 로그 수집 환경을 구성하는 데 사용된다.

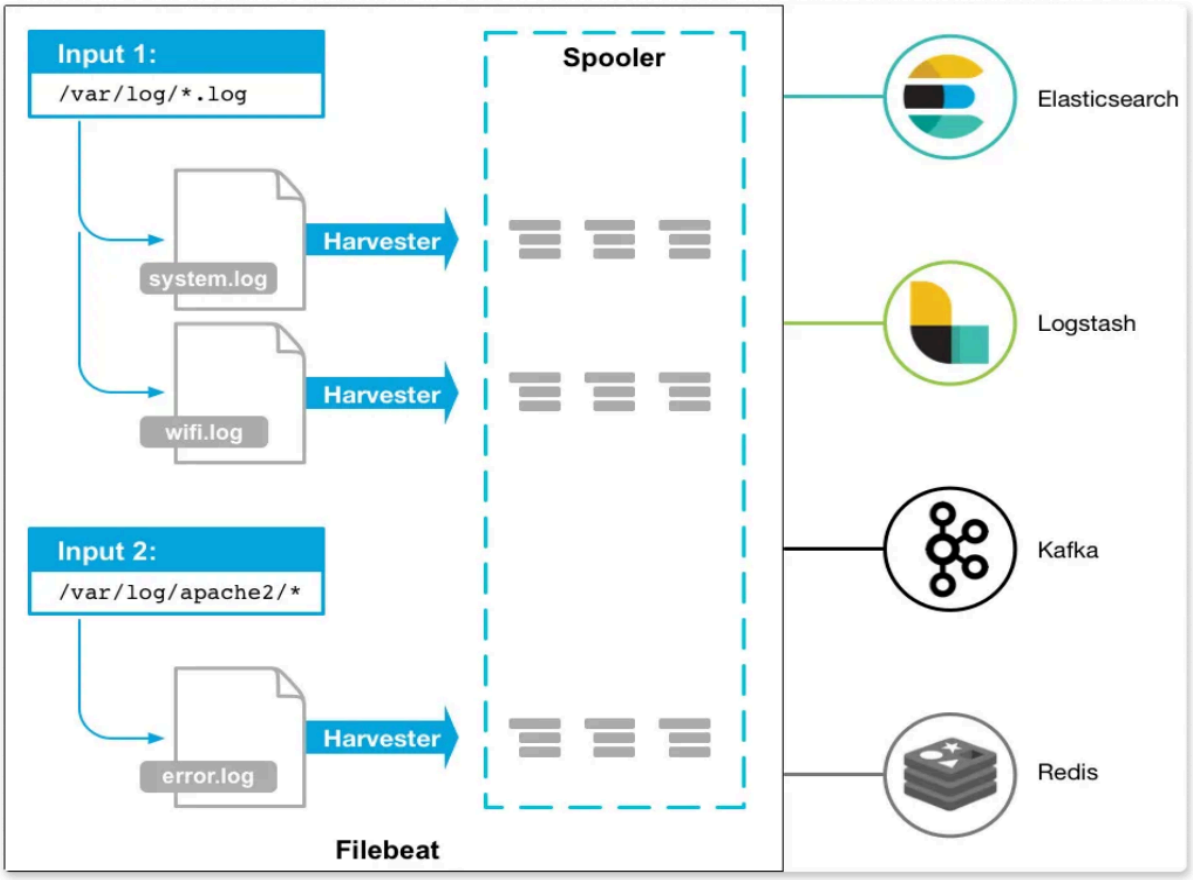


그림 3 Filebeat 흐름도

그림 3은 Filebeat의 흐름도를 나타낸 그림이다. 주요 구성 요소는 다음과 같다.

i) Prospector / Input

로그 수집 대상 파일을 정의한다. 지정된 경로에 따라 각 로그 파일마다 Harvester를 생성한다.

ii) Harvester

실제로 로그 파일을 읽는 구성 요소이다. 파일의 변경 사항을 지속적으로 추적하는 방식으로 로그 데이터를 수집한다.

iii) Spooler

수집된 로그 데이터를 메모리 버퍼에 임시로 저장한다. 버퍼 단위로 데이터를 묶어서 다음 단계로 전달한다.

iv) Publisher

Spooler로부터 전달받은 데이터를 Logstash, Elasticsearch 등의 외부 시스템에 전송한다. 이 과정에서 포맷변환이나 인코딩 처리가 가능하다.

2.2.4. Logstash

Logstash는 실시간 데이터 수집 및 처리 파이프라인 구성 도구이다. 다양한 형식의 로그 데이터를 입력받아 가공한 후, 출력 대상(Elasticsearch, 파일, 데이터베이스)으로 전송한다.

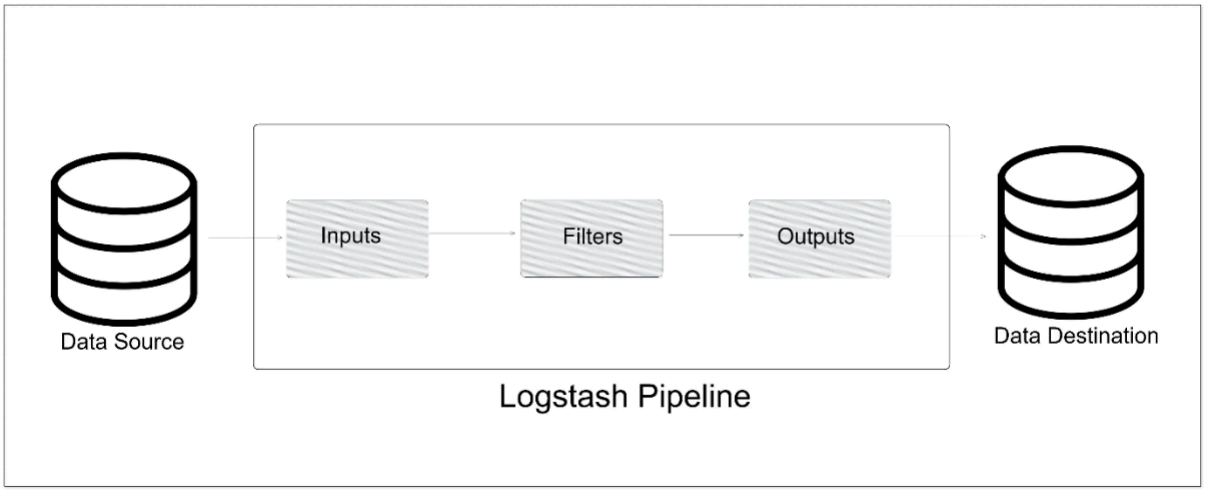


그림 4 Logstash 파이프라인

그림 4는 Logstash 파이프라인 구성도를 나타낸 그림이다. 주요 구성 요소는 다음과 같다.

i) Input Plugin

로그 데이터를 수신하는 단계이다. Filebeat, TCP, UDP, Kafka 등 다양한 방식으로 데이터를 수신할 수 있다.

ii) Filter Plugin

수신된 데이터를 변환하거나 특정 필드를 추출하는 중간 처리 단계이다. 다양한 플러그인(Grok, mutate, date, geoip, kv 등)을 활용하여 로그 포맷을 정제하고 구조화할 수 있다.

iii) Output Plugin

처리된 데이터를 출력 대상(Elasticsearch, 파일, 데이터베이스)으로 전송한다.

2.2.5. Metricbeat

Metricbeat는 서비스에서 시스템 및 서비스 수준의 메트릭(Metric)을 주기적으로 수집하는 경량 데이터 수집기다. CPU 사용률, 메모리, 디스크 I/O, 네트워크 통계와 같은 시스템 지표뿐만 아니라, 다양한 서비스의 특정 메트릭을 수집하여 Elasticsearch 또는 Logstash로 전송한다. 이를 통해 시스템과 서비스의 상태를 실시간으로 모니터링하고 성능 분석을 수행할 수 있다. Metricbeat는 모듈(Module)과 메트릭셋(Metricset)을 기반으로 동작하며, 주요 구성 요소는 다음과 같다.

i) Module

수집 대상이 되는 특정 서비스나 시스템을 의미한다. 각 모듈은 해당 기술에 맞는 메트릭을 수집하도록 사전 구성되어 있다. 사용자는 간단한 설정만으로 원하는 서비스의 모니터링을 활성화할 수 있다.

ii) Metricset

각 모듈 내에서 수집할 메트릭의 구체적인 집합을 정의한다. 예를 들어, system 모듈 안에는 cpu, memory, network, diskio와 같은 여러 Metricset이 포함될 수 있다. 이를 통해 사용자는 필요한 데이터만 선택적으로 수집하여 불필요한 리소스 낭비를 줄이고 효율적인 데이터 관리를 할 수 있다.

iii) Output

수집된 메트릭 데이터를 전송할 대상을 지정한다. 일반적으로 Elasticsearch로 직접 전송하여 Kibana를 통해 시각화하거나, 추가적인 데이터 가공이 필요할 경우 Logstash로 전송하는 방식으로 구성된다.

2.2.6. Elasticsearch

Elasticsearch는 JSON 기반 분산 검색 및 분석 엔진으로, 다양한 구조의 데이터를 실시간으로 검색할 수 있도록 설계된 시스템이다. Apache Lucene을 기반으로 하고, RESTful API를 통해 데이터의 저장, 조회, 집계를 처리한다. Elasticsearch의 주요 구성 요소는 다음과 같다.

i) Node

Elasticsearch의 실행 단위로, 데이터 저장 및 검색이 가능하다. 여로 노드가 하나의 클러스터를 형성한다.

ii) Cluster

여러 노드의 집합이다. 클러스터 이름으로 식별할 수 있고, 데이터와 Index를 통합적으로 관리한다.

iii) Index

데이터를 저장하는 논리적 단위이다. 하나의 인덱스는 다수의 Shard로 구성될 수 있다.

iv) Document

실제로 저장되는 데이터 단위이며, JSON 형식으로 표현된다. 각각의 Document는 필드와 값을 포함하고, 텍스트, 수치, 벡터 값을 가질 수 있다.

v) Shard & Replica

Shard는 Index를 나누어 저장하기 위한 물리적 단위이다. Replica는 Shard의 복제본으로, 고가용성과 데이터 안정성을 보장하기 위해 사용한다.

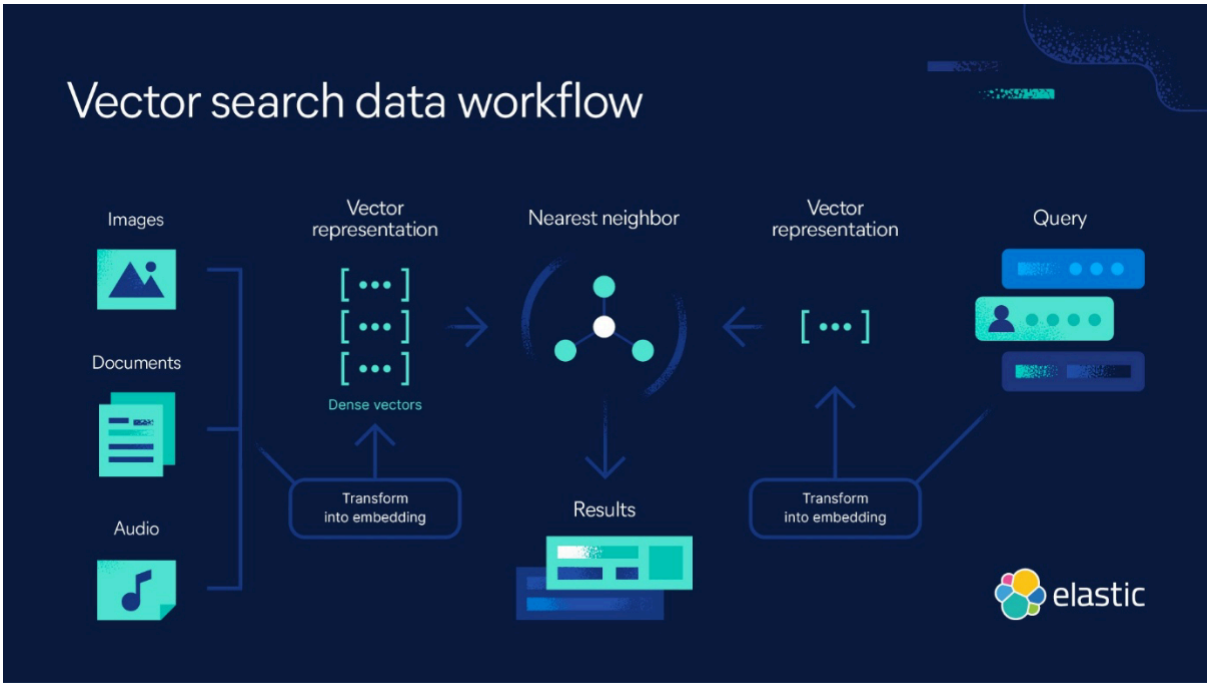


그림 5 Elasticsearch 벡터 검색 흐름도

그림 5는 Elasticsearch의 벡터 검색 지원 흐름도이다. Elasticsearch는 벡터 데이터 기반의 유사도 검색 기능이 추가되면서 벡터 데이터베이스로도 활용되고 있다. Elasticsearch가 지원하는 주요 벡터 검색 기능은 다음과 같다.

i) KNN(K-Nearest Neighbors) 검색

Elasticsearch의 knn_search API를 사용하여, 특정 벡터와 가장 유사한 문서(nearest neighbor)를 찾아낼 수 있다[9].

ii) Hybrid Search

텍스트 기반 BM25 검색과 벡터 기반 검색을 함께 수행해서 검색 품질을 높일 수 있다.

2.2.7. Flask

Flask는 파이썬 웹 애플리케이션 프레임워크로, 간단하고 유연한 구조를 제공하여 개발자들이 빠르게 웹 애플리케이션을 구축할 수 있게 해주는 도구이다.

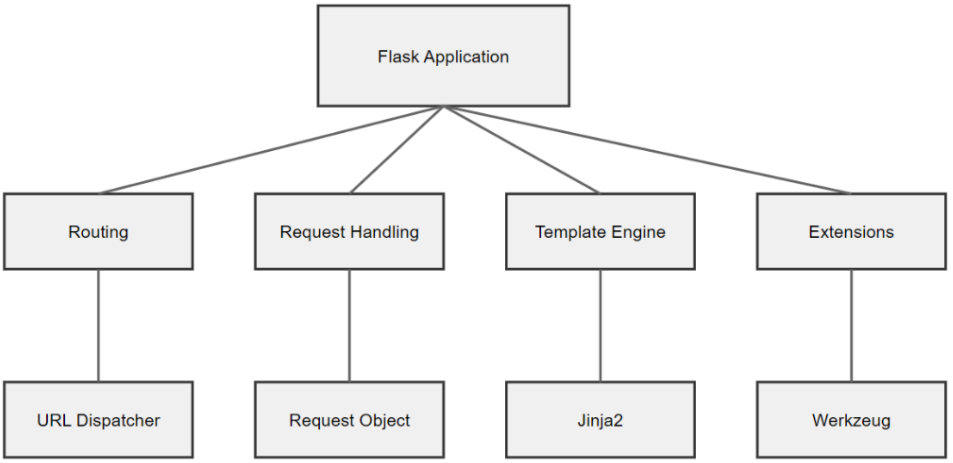


그림 6 Flask 애플리케이션 아키텍처

그림 6은 Flask 애플리케이션 아키텍처를 나타낸다. 각 구성 요소의 기능은 다음과 같다.

i) Flask Application

Flask의 핵심 객체로, 모든 Flask 애플리케이션의 시작점이 된다. 라우팅, 설정 관리, 요청 처리 등을 담당한다.

ii) 라우팅(Routing)

URL과 Python 함수를 연결하여 웹 요청을 처리하는 메커니즘 이다. @app.route() 데코레이터를 사용하여 구현한다.

iii) 요청 객체(Request)

클라이언트로부터 받은 HTTP 요청 데이터를 담고 있는 객체다. 폼 데이터, 쿠키, 헤더 등에 접근할 수 있다.

iv) 응답 객체(Response)

서버에서 클라이언트로 보내는 HTTP 응답을 나타내는 객체이다. 상태 코드, Header, Body 등을 설정할 수 있다.

2.2.8. Langchain

Langchain은 대규모 언어 모델(LLM)을 활용한 애플리케이션 개발에 특화된 오픈소스 프레임워크이다[10]. 특히 Langchain은 LLM과 외부 도구(문서, DB, API 등)를 결합해 사용자 질의에 대해 문서 검색, 도구 호출, 대화 흐름 유지 등을 지원하는 기능을 제공한다.

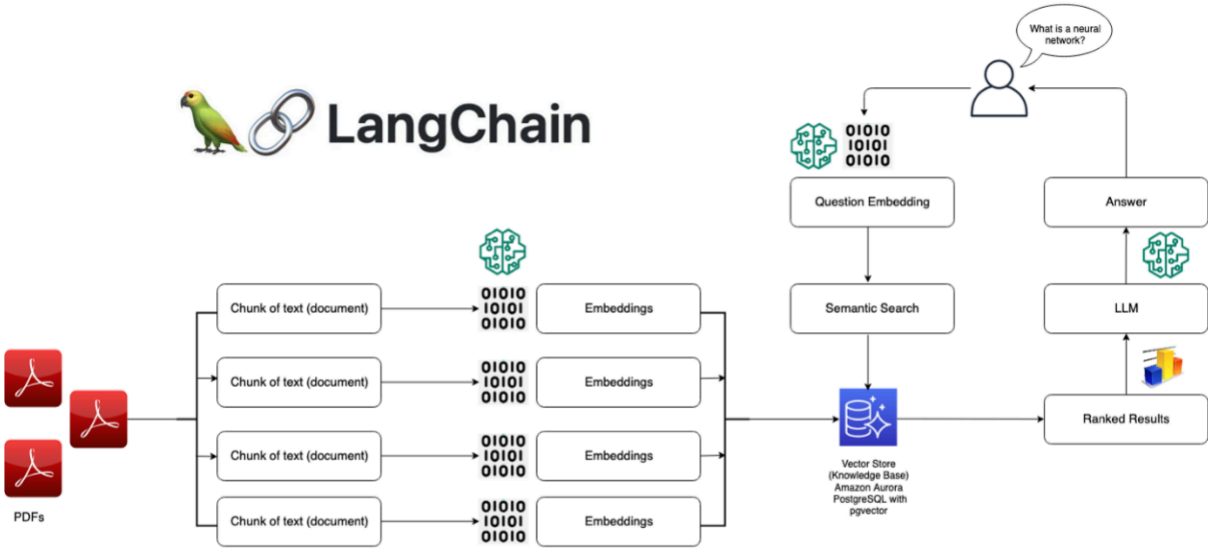


그림 7 Langchain 아키텍처

그림 7은 LangChain의 아키텍쳐 구조이다. 각 구성 요소의 기능은 다음과 같다.

i) Prompt Template

LLM에게 줄 입력 프롬프트를 템플릿화하여 일관되게 관리할 수 있는 기능이다. 변수 삽입 방식을 지원하여 사용자 입력이나 검색된 문서를 쉽게 프롬프트에 반영할 수 있다.

ii) Document Loader

PDF, HTML, Github 등 다양한 소스에서 문서를 불러와 처리할 수 있다. 단순 텍스트뿐만 아니라 구조화된 데이터도 로드할 수 있어, RAG 파이프라인에서 입력 데이터를 유연하게 확보 가능하다.

iii) Embeddings

문서를 벡터화하여 벡터 DB에서 검색할 수 있게 한다.OpenAI, HuggingFace 등 다양한 임베딩 모델을 지원하며, 변환된 벡터는 벡터 저장소(Elasticsearch, Pinecone, FAISS 등)에 저장된다.

iv) Retriever

사용자의 질의(Query)를 벡터화하여, 벡터 DB에서 의미상으로 유사한 문서를 찾아주는 모듈이다. 단순 매칭이 아니라 문맥적으로 가까운 결과를 반환하므로 RAG의 성능을 좌우하는 핵심 컴포넌트이다.

v) Chain

LLM 호출, 검색LLM 호출, 검색, 프롬프트 조합, 응답 생성을 순차적으로 연결하여 하나의 파이프라인으로 구성하는 기능이다.

2.2.9. LangSmith

LangSmith는 대규모 언어 모델(LLM) 기반 애플리케이션의 개발, 디버깅, 평가, 운영을 지원하는 관리 플랫폼이다. LangChain과 긴밀하게 연동되며, LLM 호출 과정에서 발생하는 입력·출력·중간 과정을 추적(Trace)하고, 데이터셋 관리 및 자동 평가 기능을 제공하여 모델의 품질을 체계적으로 개선할 수 있도록 한다.

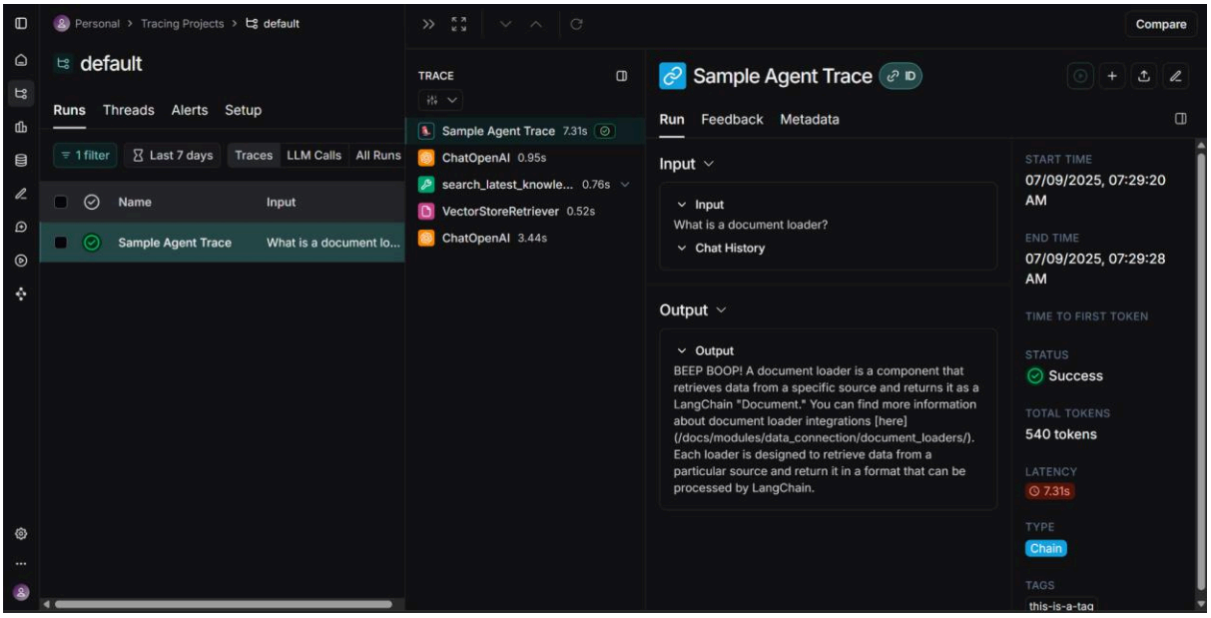


그림 8 LangSmith 대시보드 화면

그림 8은 LangSmith의 대시보드 화면 구성이다. 각 구성 요소의 기능은 다음과 같다.

i) Trace

체인(Chain), 에이전트(Agent), LLM 호출 과정을 단계별로 기록하여 투명하게 실행 흐름을 확인할 수 있다. 입력 프롬프트, 중간 출력, 최종 응답까지 모두 추적 가능하여 디버깅 및 성능 분석에 유용하다.

ii) Dataset

질문-응답 쌍이나 테스트 데이터를 체계적으로 관리할 수 있는 기능이다. 벤치마킹, 리그레션 테스트 등에 활용하여 모델의 일관된 품질을 검증할 수 있다

iii) Evaluation

LLM 응답을 자동/수동으로 평가할 수 있는 기능이다. 정확성, 관련성, 간결성 등 다양한 지표를 기준으로 응답 품질을 측정할 수 있다.

iv) Monitor

배포된 애플리케이션의 LLM 호출 현황을 모니터링하고, 실패율·지연시간·비용 등을 추적 가능하다. 서비스 운영 중 발생하는 에러나 품질 저하 문제를 빠르게 감지하고 대응할 수 있다.

v) Collaboration

여러 팀원이 동일한 프로젝트에서 프롬프트, 체인, 데이터셋을 공유하고 협업 가능하다

2.2.10. Spring Boot

Spring Boot는 Java 기반 애플리케이션 개발을 간소화하고 효율성을 극대화하기 위해 설계된 오픈 소스 프레임워크이다.

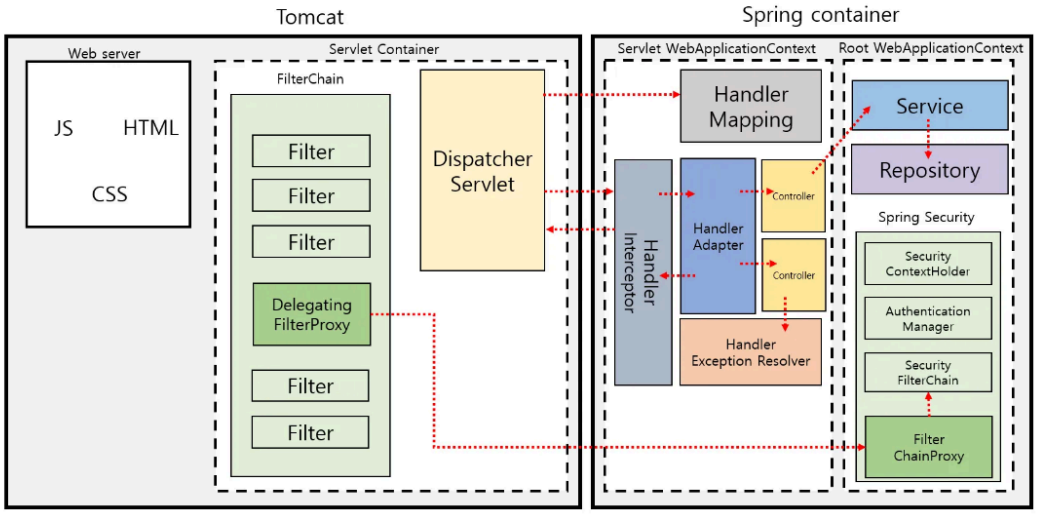


그림 9 Spring Boot 요청 처리 아키텍처

그림 9는 Spring Boot 웹 애플리케이션의 요청 처리 아키텍처를 나타낸다. 각 구성 요소의 기능은 다음과 같다.

i) Tomcat & FilterChain

내장된 웹 서버가 최초로 사용자의 HTTP 요청을 받는다. 해당 요청은 서블릿 컨테이너의 필터 체인을 통과하고 스프링 컨테이너에 등록된 Spring Security의 필터 체인으로 위임하여 인증/인가 처리를 수행한다.

ii) Spring Boot Security

Spring Boot Security는 인증, 인가, 암호화 등 다양한 보안 기능을 쉽게 구현할 수 있게 도와주는 프레임워크이다 기본적인 보안 설정을 제공하며, 사용자는 필요에 따라 설정을 커스터마이징할 수 있다.

iii) Dispatcher Servlet

요청 정보를 바탕으로 요청을 처리할 컨트롤러와 메서드를 찾아낸다. 인터페이스

역할을 수행해서 다양한 컨트롤러를 일관된 방식으로 호출 가능하게 구성한다.

3. 연구 내용

3.1. 설계 상세화

3.1.1. 유스케이스 다이어그램

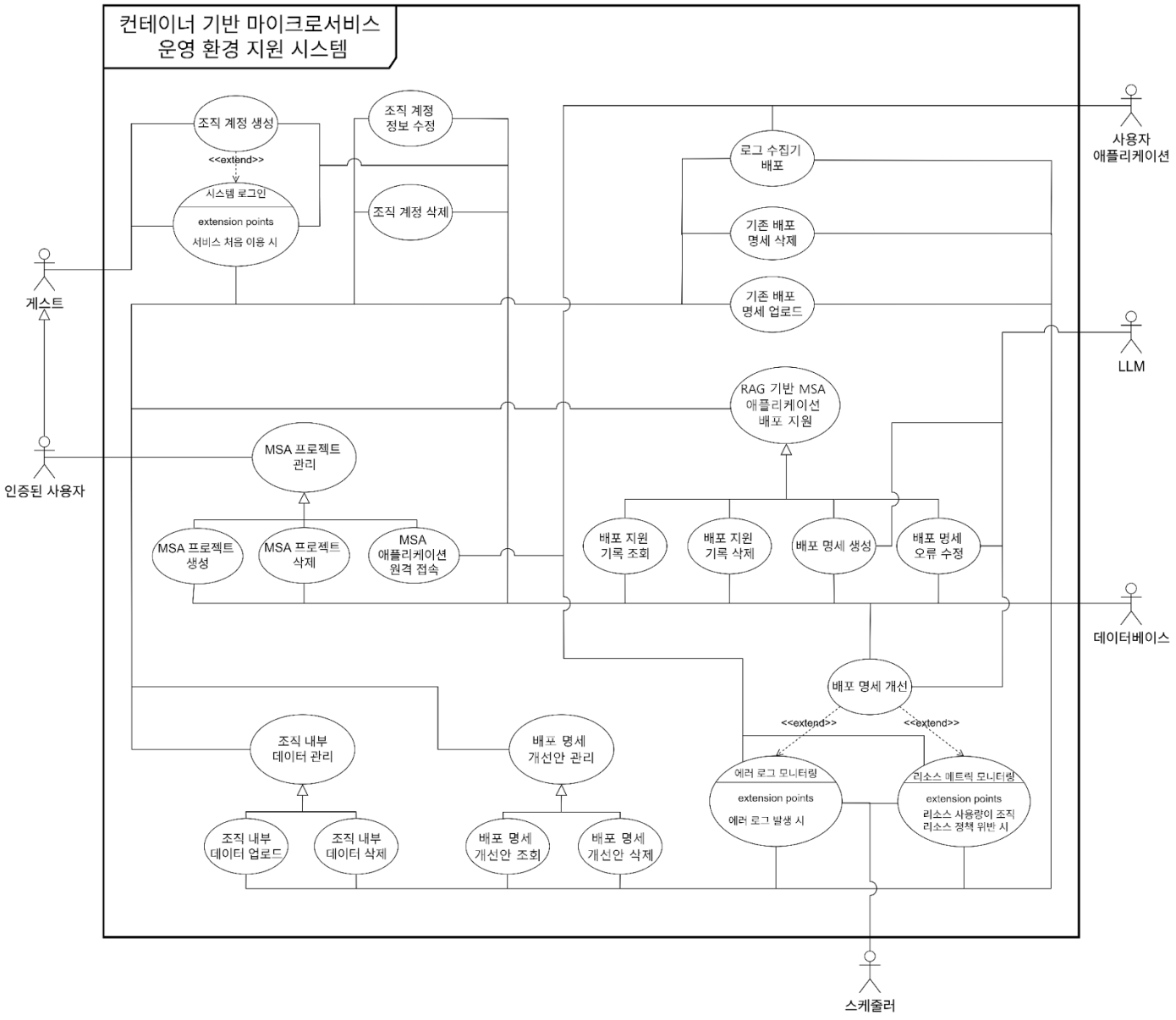


그림 10 유스케이스 다이어그램

3.1.2. 유스케이스 명세

i) 조직 계정 생성

유스케이스명	조직 계정 생성
개요	사용자는 회원가입을 통해 ID, 비밀번호, AI 서비스 API 키 등의 정보를 입력해서 계정을 생성한다.
관련 액터	사용자, 데이터베이스
선행 조건	없음
이벤트 흐름	기본흐름: 1. 사용자는 ID, 비밀번호, AI 서비스 API Key 등 회원가입에 필요한 모든 정보를 입력 후 제출한다. 2. 시스템은 ID 중복 여부를 확인한다. 3. 시스템은 AI 서비스 API키로 샘플 요청이 가능한지 확인한다. 4. 시스템은 사용자 계정을 데이터베이스에 저장한다. 대안흐름: 2-1. ID가 데이터베이스에 이미 존재하는 경우 5. 시스템은 사용자에게 입력한 ID가 이미 사용중임을 알리는 에러 메시지를 출력한다. 4-1. AI 서비스 API키로 샘플 요청이 불가능한 경우 1. 시스템은 사용자에게 API 키가 유효하지 않음을 알리는 에러 메시지를 출력한다.
후행 조건	사용자는 조직 계정을 생성한다.

ii) 조직 계정 삭제

유스케이스명	조직 계정 삭제
개요	사용자는 시스템에 등록된 조직 계정을 삭제할 수 있으며, 시스템은 해당 계정을 데이터베이스에서 제거하고 해당 계정과 관련된 데이터(API 키, 프로젝트 목록 등) 또한 모두 제거한다.
관련 액터	사용자, 데이터베이스
선행 조건	사용자는 시스템에 로그인해야 한다.
이벤트 흐름	기본흐름: 1. 사용자가 비밀번호를 입력하고 계정 탈퇴를 요청한다. 2. 시스템은 사용자의 비밀번호를 검증한다. 3. 시스템은 사용자 계정과 관련된 모든 데이터를 데이터베이스에서 제거한다. 4. 시스템은 사용자에게 삭제 완료 메시지를 출력한다. 대안흐름: 2-1. 비밀번호가 올바르지 않은 경우 1. 시스템은 비밀번호 불일치 메시지를 출력하고 삭제를 중단한다. 예외흐름: 3-1. 데이터베이스 삭제 처리 중 오류 발생 시 1. 시스템은 삭제 작업을 롤백하고 삭제 실패 메시지를 출력한다.
후행 조건	해당 조직 계정은 시스템에서 더 이상 유효하지 않으며 로그인할 수 없으며 계정과 연결된 API 키, 프로젝트 목록 등 관련 데이터는 모두 삭제된다.

iii) 조직 계정 정보 수정

유스케이스명	조직 계정 정보 수정
개요	사용자는 등록한 비밀번호, API 키 등을 수정한다.
관련 액터	사용자, 데이터베이스
선행 조건	사용자가 로그인되어 있어야 한다.
이벤트 흐름	기본흐름: 1. 사용자가 조직 계정 정보(API 키 값)를 수정한 값을 시스템에 전달한다. 2. 시스템은 입력된 API키로 샘플 요청이 가능한지 확인한다. 3. 수정된 정보가 데이터베이스에 반영된다. 대안흐름: 3-1. AI 서비스 API키로 샘플 요청이 불가능한 경우 1. 시스템은 사용자에게 API 키가 유효하지 않음을 알리는 에러 메시지를 출력한다.
후행 조건	데이터베이스는 수정된 사용자 정보를 저장한다.

iv) 시스템 로그인

유스케이스명	시스템 로그인
개요	사용자는 계정 정보를 통해 시스템에 로그인하고, 시스템은 인증된 사용자만 주요 기능에 접근할 수 있다.
관련 액터	사용자, 데이터베이스
선행 조건	없음
이벤트 흐름	기본흐름: 1. 사용자가 ID와 비밀번호를 입력해 사용자 인증을 요청한다. 2. 시스템은 데이터베이스에서 해당 ID의 사용자 정보를 조회한다. 3. 시스템은 해당 사용자의 비밀번호와 입력된 비밀번호를 비교하여 일치하는지 판단한다. 대안흐름: 3-1. 입력된 비밀번호가 데이터베이스 내부 정보와 일치하지 않는 경우 1. 시스템은 사용자에게 인증 정보가 일치하지 않다는 오류 메시지를 출력한다.
후행 조건	사용자는 로그인하여 시스템의 기능을 사용할 수 있다.

v) MSA 프로젝트 생성

유스케이스명	MSA 프로젝트 생성
개요	사용자는 MSA 애플리케이션을 관리할 프로젝트를 생성할 수 있으며, 이 때 프로젝트 이름, SSH 접속 정보(IP 주소, 서버 사용자 명, 포트, pem 파일)을 입력해야 한다.
관련 액터	사용자, 데이터베이스
선행 조건	사용자는 시스템에 로그인해야 한다.
이벤트 흐름	기본흐름: 1. 사용자가 프로젝트 이름, SSH 접속 정보(IP주소, 서버 사용자 명, 포트, pem 파일)을 입력한다. 2. 시스템은 동일한 이름의 프로젝트가 이미 존재하는지 확인한다. 3. 시스템은 프로젝트 정보를 데이터베이스에 저장한다. 4. 시스템은 사용자에게 프로젝트를 프로젝트 목록 화면에 추가하여 사용자에게 보여준다. 대안흐름: 2-1. 동일한 프로젝트 이름이 이미 존재하는 경우 1. 시스템은 프로젝트 이름 중복 오류 메시지를 출력한다.
후행 조건	새 프로젝트가 데이터베이스에 저장된다.

vi) MSA 프로젝트 삭제

유스케이스명	MSA 프로젝트 삭제
개요	사용자는 시스템에 등록한 프로젝트를 삭제할 수 있으며 시스템은 해당 프로젝트와 관련된 모든 데이터(조직 내부 데이터, 모니터링 작업 이력, 배포 지원 이력 등)이 데이터베이스에서 제거된다.
관련 액터	사용자, 데이터베이스
선행 조건	사용자는 시스템에 로그인해야 해야 한다.
이벤트 흐름	기본흐름: 1. 사용자가 삭제할 프로젝트를 선택한다. 2. 시스템은 해당 프로젝트가 존재하는지 확인한다. 3. 시스템은 프로젝트와 관련된 모든 데이터(모니터링 작업 이력, 배포 지원 이력 등) 를 데이터베이스에서 제거한다. 4. 시스템은 삭제된 프로젝트가 화면에서 사라진 것을 사용자에게 보여준다. 대안흐름: 2-1. 선택한 프로젝트가 데이터베이스에 존재하지 않는 경우 1. 시스템은 프로젝트를 찾을 수 없다는 오류 메시지를 출력한다.
후행 조건	프로젝트와 관련된 모든 데이터가 데이터베이스에서 제거되고, 화면에서도 더 이상 표시되지 않는다.

vii) MSA 애플리케이션 원격 접속

유스케이스명	MSA 애플리케이션 원격 접속
개요	사용자는 시스템에 등록된 프로젝트 정보(SSH 키, pem 파일, IP 주소, 포트, 접속 계정명)를 이용하여 프로젝트에 등록된 MSA 애플리케이션 환경에 원격으로 접속할 수 있으며, 시스템은 해당 정보를 검증한 뒤 원격 연결 세션을 생성하여 사용자에게 제공한다.
관련 액터	사용자, 데이터베이스
선행 조건	사용자는 시스템에 로그인해야 한다.
이벤트 흐름	기본흐름: 1. 사용자는 특정 프로젝트에 대해 원격 접속을 요청한다. 2. 시스템은 데이터베이스에서 해당 프로젝트의 SSH 키, pem 파일, IP 주소, 접속 계정명을 조회하고 유효성을 검증한다. 3. 시스템은 검증된 정보를 기반으로 SSH 클라이언트를 통해 원격 서버와 연결을 시도한다. 4. 시스템은 연결이 성공한 경우, 사용자에게 원격 접속 세션을 제공한다. 대안흐름: 2-1. SSH 키 또는 pem 파일이 손상되었거나 누락된 경우 1. 시스템은 잘못된 인증 정보 메시지를 출력한다. 2-2. IP 주소가 유효하지 않거나 네트워크에 도달할 수 없는 경우 1. 시스템은 호스트에 연결할 수 없다는 메시지를 출력한다. 3-1. SSH 인증 과정에서 실패한 경우 1. 시스템은 접속을 중단하고 사용자에게 인증 실패 사유 메시지를 출력한다.
후행 조건	사용자는 시스템이 제공하는 웹 터미널 세션을 통해 프로젝트 환경에 접근할 수 있다.

viii) 조직 내부 데이터 업로드

유스케이스명	조직 내부 데이터 업로드
개요	사용자는 시스템에 조직 내부 데이터(MSA 애플리케이션 정의, 마이크로서비스 기본 정보, 커스텀 이미지, 운영 정책)를 .zip 파일 형태로 업로드 해야 하고, 시스템은 압축을 해제하여 지원 포맷의 데이터를 추출한 뒤 대해 벡터 임베딩을 수행하고 벡터 저장소에 저장한다.
관련 액터	사용자, 데이터베이스
선행 조건	사용자는 시스템에 로그인해야 한다.
이벤트 흐름	기본흐름: 1. 사용자는 조직 내부 데이터(zip 파일)를 업로드한다. 2. 시스템은 zip 파일을 사용자별 임시 디렉토리에 압축 해제한다. 3. 시스템은 해제된 파일 중, 지원하지 않는 포맷(.exe, .sh, .bat 등)을 제외하고 처리 가능한 파일만 선별한다. 4. 시스템은 선별된 파일에서 텍스트를 추출하고 벡터 임베딩을 수행하여 데이터베이스에 저장한다. 5. 시스템은 사용자에게 처리 결과 메시지를 출력한다. 대안흐름: 3-1. 압축 해제된 파일 중 저장 가능한 포맷의 파일이 존재하지 않는 경우 1. 시스템은 사용자에게 저장 가능한 데이터가 없다는 메시지를 출력한다. 예외흐름: 3-2. 데이터베이스에 저장 중 오류가 발생한 경우 1. 시스템은 저장 작업을 롤백하고 사용자에게 저장 실패 메시지를 출력한다. 3-3. 지원 가능한 포맷의 파일의 텍스트를 추출하는 중 오류가 발생한 경우 1. 시스템은 해당 파일의 텍스트를 추출 오류 메시지를 출력한다.
후행 조건	시스템은 정상 처리된 조직 내부 데이터를 벡터 저장소에 저장한다.

ix) 조직 내부 데이터 삭제

유스케이스명	조직 내부 데이터 삭제
개요	사용자가 시스템에 저장된 조직 내부 데이터들 중 삭제를 원하는 파일을 삭제할 수 있으며, 시스템은 해당 데이터를 데이터베이스에서 제거한다.
관련 액터	사용자, 데이터베이스
선행 조건	사용자는 시스템에 로그인해야 한다.
이벤트 흐름	기본흐름: 1. 사용자는 저장된 조직 내부 데이터들 중에서 원하는 삭제를 원하는 파일을 선택한다. 2. 시스템은 파일명을 기반으로 데이터베이스에서 해당 데이터를 조회한다. 3. 시스템은 사용자가 삭제를 요청한 파일의 이름을 기반으로 데이터베이스에 삭제 쿼리를 수행해서 해당 이름의 데이터를 삭제한다. 4. 시스템은 사용자에게 삭제 완료 메시지를 출력한다. 대안흐름: 2-1. 선택한 파일이 데이터베이스에 존재하지 않는 경우 1. 시스템은 사용자에게 삭제 가능한 데이터가 없다는 메시지를 출력한다. 예외흐름: 3-1. 데이터베이스 삭제 중 오류가 발생한 경우 1. 시스템은 삭제 작업을 롤백하고 삭제 실패 메시지를 출력한다.
후행 조건	해당 이름의 조직 내부 데이터는 데이터베이스에서 완전히 제거된다.

x) 배포 명세 생성

유스케이스명	배포 명세 생성
개요	시스템은 사용자의 질의를 바탕으로, 내부 데이터 DB를 참고해서 각 설정 항목에 주석과 해설이 포함된 배포 파일을 새로 생성한다.
관련 액터	사용자, 데이터베이스, LLM
선행 조건	사용자는 시스템에 로그인해야 한다.
이벤트 흐름	기본흐름: 1. 사용자가 배포하고자 하는 서비스 설명 또는 약결합 개선이 필요한 배포 파일에 대한 설명을 자연어로 입력한다. 2. 시스템은 사용자 입력을 기반으로 데이터베이스에서 유사도를 기반으로 내부 데이터를 조회한다. 3. 조회된 내부 데이터, 사용자 입력을 기반으로 마이크로서비스 배포 명세와 배포 명세에 대한 설명을 생성하기 위한 프롬프트를 작성한다. 4. 작성된 프롬프트를 기반으로 LLM에 생성을 요청한다. 5. 시스템은 생성된 배포 명세와 설명을 사용자에게 제공한다. 예외흐름: 2-1. 데이터베이스 조회 중 오류가 발생한 경우 1. 시스템은 데이터베이스 연결 상태를 점검하고 오류 내용을 기록한다.
후행 조건	사용자는 시스템이 제시한 배포 명세 파일을 참고할 수 있다.

xi) 배포 명세 오류 수정

유스케이스명	배포 명세 오류 수정
개요	시스템은 사용자의 질의 및 기존 배포 명세를 바탕으로, 내부 데이터 DB를 참고해서 각 설정 항목에 주석과 해설이 포함된 배포 파일 수정안을 생성한다.
관련 액터	사용자, 데이터베이스, LLM
선행 조건	사용자는 시스템에 로그인해야 하고, 질의 시 기존 배포 명세를 업로드해야 한다.
이벤트 흐름	기본흐름: 1. 시스템은 데이터베이스에서 오류 상황과 관련된 자료를 검색한다. 2. 시스템은 수집된 정보들을 기반으로 배포 명세 수정 프롬프트를 생성한다. 3. LLM은 생성된 프롬프트를 기반으로 적절한 배포 명세 수정과 설명을 생성한다. 4. 시스템은 생성된 배포 명세와 설명을 사용자에게 제공한다. 예외흐름: 1-1. 데이터베이스 조회 중 오류가 발생한 경우 1. 시스템은 데이터베이스 연결 상태를 점검하고 오류 내용을 기록한다.
후행 조건	사용자에게 적용 가능한 수정안이 제공된다.

xii) 배포 지원 이력 조회

유스케이스명	배포 지원 이력 조회
개요	사용자가 시스템 내에서 이전에 수행한 서비스 배포 지원 요청 또는 개선 작업 등의 이력을 확인할 수 있다.
관련 액터	사용자, 데이터베이스
선행 조건	사용자가 로그인한 상태여야 한다.
이벤트 흐름	기본흐름: 1. 시스템은 데이터베이스에서 사용자의 작업 이력 조회한다. 2. 시스템은 조회한 이력들을 최신순으로 정렬한다. 3. 시스템은 이력 목록을 사용자에게 출력한다. 대안흐름: 1-1. 사용자의 작업 이력이 없을 경우 1. 시스템은 사용자에게 작업 이력이 없다는 내용을 출력한다.
후행 조건	사용자는 목록에서 특정 항목을 선택하여 상세 내용을 확인할 수 있다.

xiii) 배포 지원 이력 삭제

유스케이스명	배포 지원 이력 삭제
개요	사용자는 시스템 내에서 자신이 수행한 서비스 배포 지원 이력을 삭제할 수 있으며, 시스템은 해당 이력을 데이터베이스에서 제거한다.
관련 액터	사용자, 데이터베이스
선행 조건	사용자가 로그인한 상태여야 한다.
이벤트 흐름	기본흐름: 1. 시스템은 삭제할 배포 지원 이력 항목을 선택한다. 2. 시스템은 해당 이력이 존재하는지 확인한다. 3. 시스템은 선택된 이력을 데이터베이스에서 삭제한다. 4. 시스템은 배포 지원 이력 목록에서 해당 이력을 제거해서 삭제되었음을 사용자에게 보여준다. 대안흐름: 2-1. 선택한 이력이 데이터베이스에 존재하지 않는 경우 1. 시스템은 사용자에게 삭제할 이력이 없다는 메시지를 출력한다. 예외흐름: 3-1. 데이터베이스 삭제 처리 중 오류가 발생한 경우 1. 시스템은 삭제 작업을 롤백하고 삭제 실패 메시지를 출력한다.
후행 조건	삭제된 이력은 데이터베이스에서 제거되며, 배포 지원 이력 목록 화면에서 표시되지 않는다.

xiv) 로그 수집기 배포

유스케이스명	로그 수집기 배포
개요	사용자는 ELK 스택 중 로그 수집기(Filebeat, Logstash, Metricbeat)를 배포해야 하고, 시스템은 수집되는 로그를 벡터 DB로 저장한다.
관련 액터	사용자, 데이터베이스, 사용자 애플리케이션
선행 조건	사용자는 시스템에 로그인해야 한다.
이벤트 흐름	기본흐름: 1. 사용자는 로그 수집기 배포 파일에 들어갈 Logstash 포트, 적용할 네임스페이스명을 시스템에 입력한다 2. 시스템은 사용자가 시스템에 입력한 정보를 기반으로 사용자의 MSA 애플리케이션에서 시스템의 데이터베이스로 로그를 전송할 수 있는 로그 수집기(Filebeat, Logstash, Metricbeat) 배포 파일을 제공한다. 3. 사용자는 해당 배포 파일을 사용자 애플리케이션에 적용하여 로그 수집기를 구성한다. 예외흐름: 2-1. 로그 수집기 배포 파일 구성 중 오류가 발생한 경우 1. 시스템은 데이터베이스 연결 상태를 점검하고 오류 내용을 기록한다.
후행 조건	시스템은 사용자 애플리케이션 모니터링을 수행한다.

xv) 기존 배포 명세 업로드

유스케이스명	기존 배포 명세 업로드
개요	사용자는 기존 배포에 사용한 배포 명세 파일을 업로드할 수 있으며, 시스템은 해당 배포 명세 파일을 검증한 뒤 데이터베이스에 저장한다
관련 액터	사용자, 데이터베이스
선행 조건	사용자가 로그인한 상태여야 한다.
이벤트 흐름	기본흐름: 1. 사용자는 하나 이상의 배포 명세 파일을 선택하여 업로드한다. 2. 시스템은 파일의 확장자가 .yaml 또는 .yml인지 검증한다. 3. 시스템은 올바른 확장자 파일에서 텍스트를 추출한다. 4. 시스템은 배포 명세 파일을 데이터베이스에 저장한다. 5. 시스템은 업로드에 성공한 파일들의 이름을 기존 배포 명세 목록에 추가하여 사용자에게 보여준다. 예외흐름: 2-1. 파일 확장자가 .yaml 또는 .yml이 아닌 경우 1. 시스템은 지원하지 않는 파일 형식 오류 메시지를 출력하고 해당 파일은 저장 대상에서 제외한다. 3-1. 텍스트 추출 과정에서 오류가 발생한 경우 1. 시스템은 텍스트 추출 실패 메시지를 출력하고 해당 파일은 저장 대상에서 제외한다. 4-1. 데이터베이스 저장 과정에서 오류가 발생한 경우 1. 시스템은 저장을 롤백하고 사용자에게 저장 실패 메시지를 출력한다.
후행 조건	정상 처리된 배포 명세 파일은 데이터베이스에 저장되며, 업로드된 파일이 목록에 표시된다.

xvi) 에러 로그 모니터링

유스케이스명	에러 로그 모니터링
개요	시스템은 Elasticsearch로부터 주기적으로 에러 로그를 수집하고, 조직 내부 데이터에 명시된 해결 방안을 기반으로 개선된 배포 파일을 생성하여 사용자에게 제공해야 한다.
관련 액터	스케줄러, 데이터베이스, LLM
선행 조건	사용자가 로그인한 상태이며, 사용자 애플리케이션 로그 수집이 정상적으로 수행되어야 한다.
이벤트 흐름	기본흐름: 1. 시스템은 데이터베이스에서 로그인 된 사용자의 로그를 3분 주기로 조회한다. 2. 시스템은 해당 사용자의 로그에 대해 텍스트 기반 분석을 수행한다. 예외흐름: 2-1. 오류 로그를 발견한 경우 1. 시스템은 문제를 해결할 수 있는 오류 해결 방안을 사용자에게 제공한다.
후행 조건	없음

xvii) 운영 메트릭 모니터링

유스케이스명	운영 메트릭 모니터링
개요	시스템은 데이터베이스에 주기적으로 저장되는 애플리케이션의 운영 메트릭(CPU, 메모리 사용량)을 수집하고, 조직 내부 데이터에 정의된 리소스 정책을 기반으로 최적화된 배포 명세 파일을 생성하여 사용자에게 제공한다.
관련 액터	스케줄러, 데이터베이스, LLM
선행 조건	데이터베이스에 메트릭 데이터가 저장되고 있어야 하며, 조직 내부 데이터에 리소스 정책이 정의되어 있어야 한다.
이벤트 흐름	기본흐름: 1. 시스템은 3분 주기로 스케줄러를 실행하여 데이터베이스에서 최근 메트릭 데이터(CPU, 메모리 사용량)를 수집한다. 2. 시스템은 수집된 메트릭과 관련된 조직 리소스 정책을 검색한다. 3. 시스템은 조직 리소스 정책을 고려하여 최적화된 배포 명세를 생성한다. 4. 시스템은 생성된 배포 명세 파일을 데이터베이스에 저장하고, 모니터링 이력 목록에 추가한다.
후행 조건	최적화된 배포 명세 파일이 데이터베이스에 저장되고 모니터링 이력 목록에 추가된다.

xviii) 배포 명세 개선안 조회

유스케이스명	배포 명세 개선안 조회
개요	사용자는 시스템 수집된 로그 및 메트릭 기반의 배포 명세 개선 이력을 조회할 수 있다.
관련 액터	사용자, 데이터베이스
선행 조건	사용자가 로그인한 상태여야 한다.
이벤트 흐름	기본흐름: 1. 사용자가 조회할 모니터링 이력 항목을 선택한다. 2. 시스템은 데이터베이스에서 해당 이력의 ID로 상세 정보를 조회한다. 3. 시스템은 조회된 이력의 상세 내용을 사용자에게 보여준다. 예외흐름: 2-1. 선택한 이력이 데이터베이스에 존재하지 않는 경우 1. 조회 가능한 이력이 없다는 메시지를 출력한다. 2-2. 데이터베이스에서 해당 이력을 조회 중 오류가 발생한 경우 1. 시스템은 해당 이력 조회 실패 메시지를 출력한다.
후행 조건	조회된 이력의 상세 내용을 사용자가 확인할 수 있다.

xix) 배포 명세 개선안 삭제

유스케이스명	배포 명세 개선안 삭제
개요	사용자는 시스템에 저장된 모니터링 이력 중 특정 항목을 삭제할 수 있으며, 시스템은 해당 이력을 데이터베이스에서 제거한다.
관련 액터	사용자, 데이터베이스
선행 조건	사용자가 로그인한 상태여야 한다.
이벤트 흐름	기본흐름: 1. 사용자가 삭제할 모니터링 이력 항목을 선택한다. 2. 시스템은 해당 이력 존재 여부를 확인한다. 3. 시스템은 해당 이력을 데이터베이스에서 삭제한다. 4. 시스템은 모니터링 이력 목록에서 해당 이력을 제거하여 사용자에게 사라진 결과를 보여준다. 예외흐름: 2-1. 선택한 이력이 데이터베이스에 존재하지 않는 경우 1. 시스템은 삭제할 이력이 없다는 메시지를 출력한다. 3-1. 데이터베이스에서 해당 이력 삭제 오류가 발생한 경우 1. 시스템은 삭제 작업을 롤백하고 삭제 실패 메시지를 출력한다.
후행 조건	삭제된 이력은 데이터베이스에서 제거되고, 모니터링 이력 목록에 표시되지 않는다.

3.1.3. 클래스 다이어그램

i) 조직 계정 관리

그림 11은 조직 계정을 관리하기 위한 클래스 다이어그램이다.

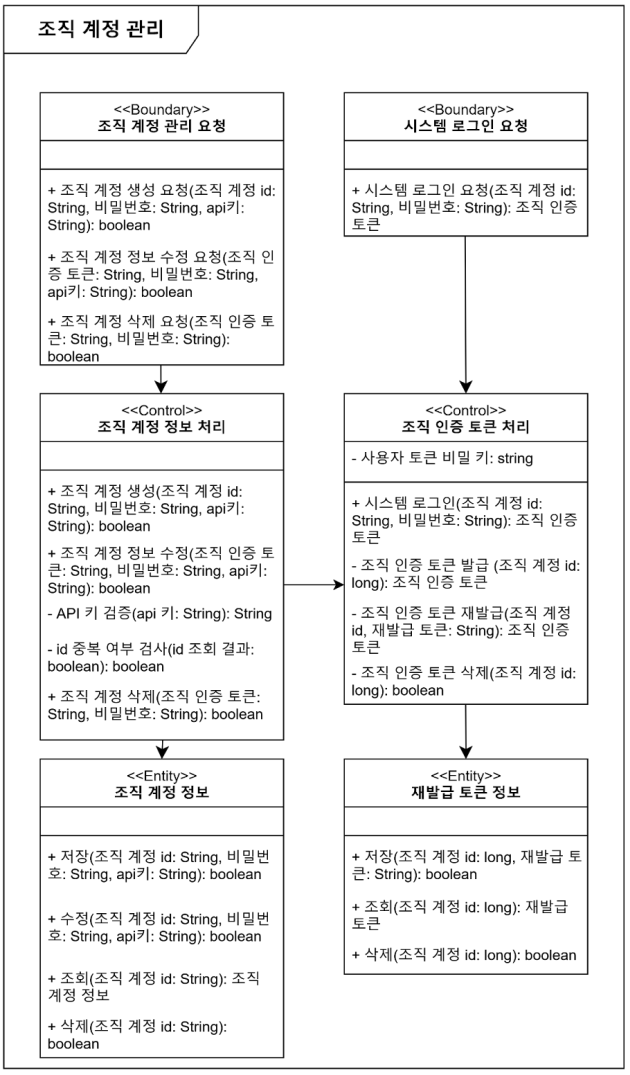


그림 11 조직 계정 관리 클래스 다이어그램

그림에 나타난 바와 같이, 조직 계정 관리 요청 클래스는 사용자가 조직 계정을 생성, 수정, 삭제 시 사용하는 클래스이다. 조직 계정 정보 처리 클래스는 사용자 요구사항에 따라 계정을 생성, 삭제, 수정한다. 시스템 로그인 요청 클래스는 사용자가 시스템에 로그인하기 위해 사용하는 클래스로, 조직 인증 토큰 처리 클래스를 통해 인증 토큰을 발급받는다. 조직 계정 정보 클래스는 시스템에 저장된 조직 계정 정보를 가지고 있고 재발급 토큰 정보 클래스는 사용자의 재발급 토큰 정보를 가지고 있다.

ii) MSA 프로젝트 관리

그림 12는 사용자가 시스템에 등록한 프로젝트를 관리하기 위한 클래스 다이어그램이다.

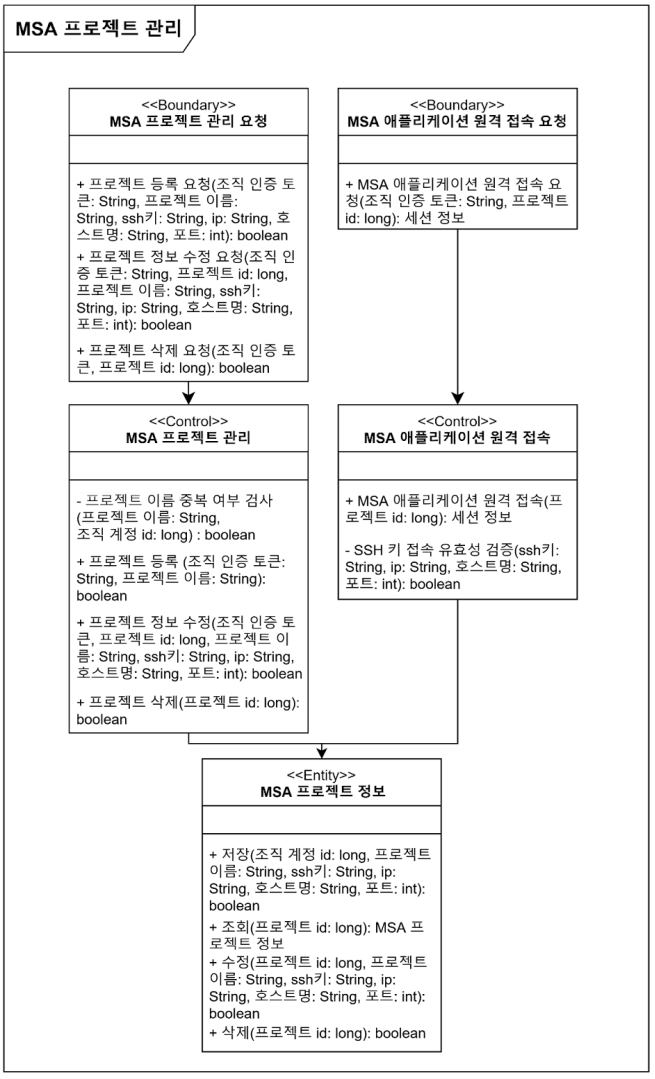


그림 12 MSA 프로젝트 관리 클래스 다이어그램

그림과 같이, MSA 프로젝트 관리 요청 클래스는 사용자가 MSA 애플리케이션을 관리하는 프로젝트를 등록, 정보 수정, 삭제 시 사용하는 클래스이다. MSA 프로젝트 관리에서 프로젝트를 관리하기 위한 주요 로직을 수행한다. MSA 애플리케이션 원격 접속 클래스는 MSA 애플리케이션 원격 접속 환경을 구성하는 데 사용되는 클래스이다. MSA 애플리케이션 원격 접속 클래스에서 유효성 검증 등 원격 접속을 위한 환경을 구축한 MSA 프로젝트 정보 클래스는 시스템에 등록된 MSA 프로젝트 정보를 관리한다.

iii) 조직 내부 데이터 관리

그림 13은 조직 내부 데이터를 관리할 수 있도록 구성한 클래스 다이어그램이다.

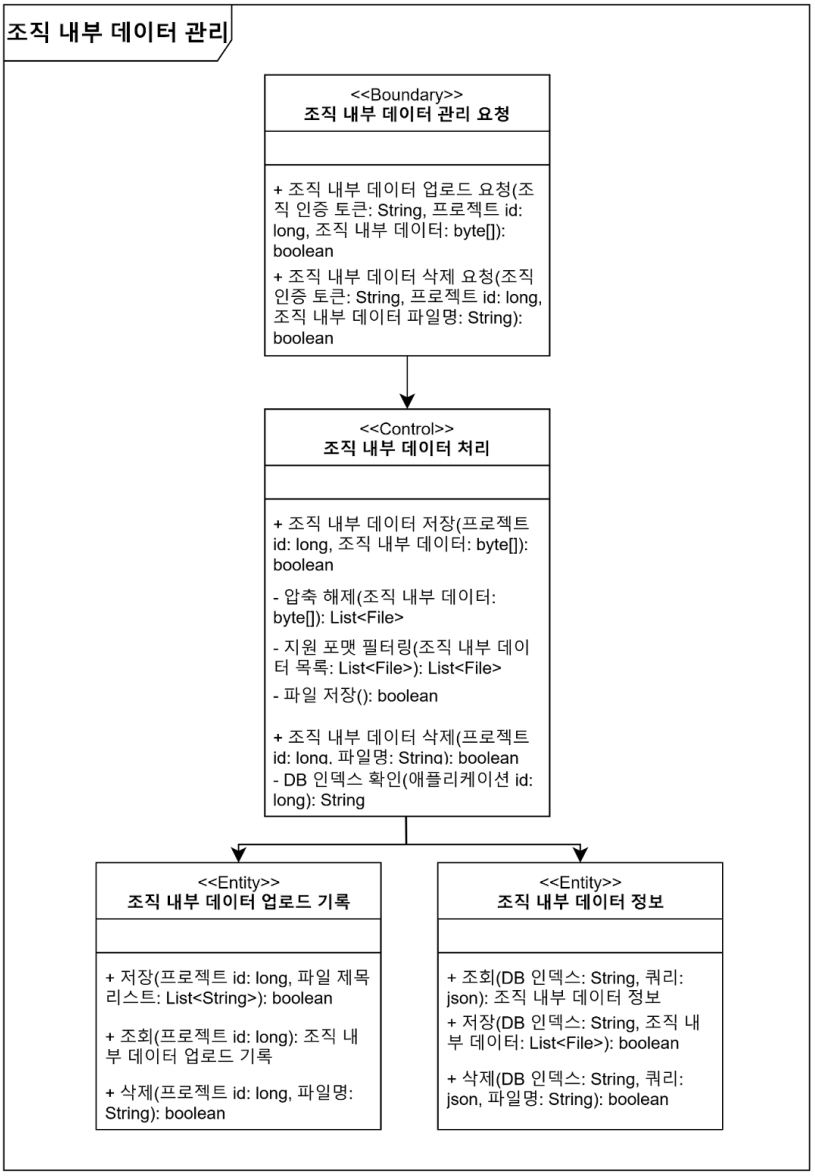


그림 13 조직 내부 데이터 관리 클래스 다이어그램

그림과 같이, 조직 내부 데이터 관리 요청 클래스는 조직 내부 데이터를 본 시스템에 업로드 및 삭제하기 위한 인터페이스 역할을 한다. 조직 내부 데이터 처리 클래스에서 조직 내부 데이터를 시스템에 저장하기 위한 주요 로직을 수행한다. 조직 내부 데이터 업로드 기록 클래스는 업로드된 조직 내부 데이터의 파일 제목을, 조직 내부 데이터 정보 클래스는 조직 내부 데이터의 파일 내용을 저장 및 관리하는 역할을 수행한다.

iv) RAG 기반 MSA 배포 지원

그림 14는 RAG 기반 MSA 배포 지원 기능을 구성한 클래스 다이어그램이다.

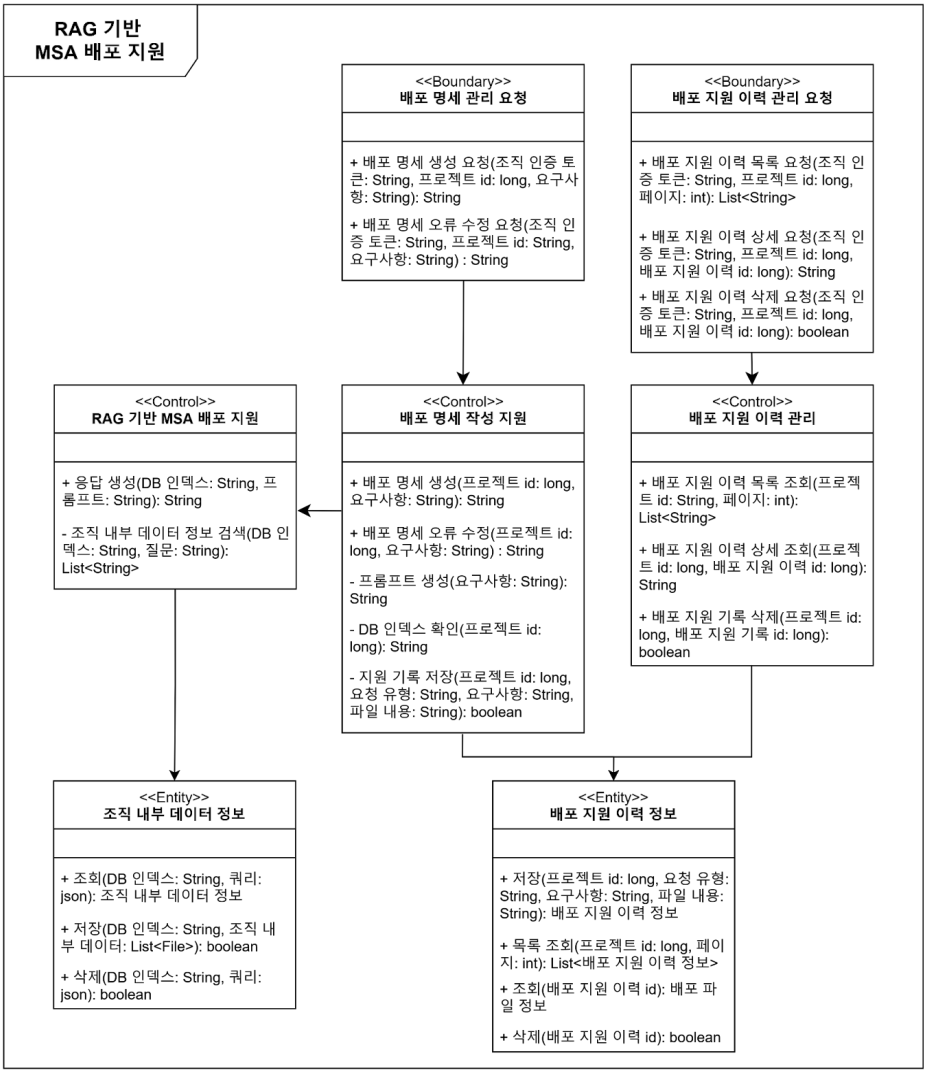


그림 14 RAG 기반 MSA 배포 지원 클래스 다이어그램

그림에 명시된 바와 같이, 배포 명세 관리 요청 클래스는 배포 명세 생성 및 오류 수정을 위한 인터페이스 역할을 한다. 배포 명세 작성 지원 클래스는 배포 명세 생성 및 오류 수정 로직을 수행하고 지원 이력 정보를 저장한다. RAG 기반 MSA 배포 지원 클래스는 조직 내부 데이터 정보를 통해 배포 명세를 생성한다.

배포 지원 이력 관리 요청 클래스는 배포 명세 작성 지원 클래스를 통해 생성된 배포 지원 이력 정보를 조회하는 기능을 수행한다. 배포 지원 이력 관리 클래스에서 조회 및 삭제를 위한 로직을 수행하고, 배포 지원 이력 정보 클래스에서 배포 지원 이력 정보를 관리한다.

v) 서비스 운영 모니터링

그림 15는 서비스 운영 모니터링 기능을 구성한 클래스 다이어그램이다.

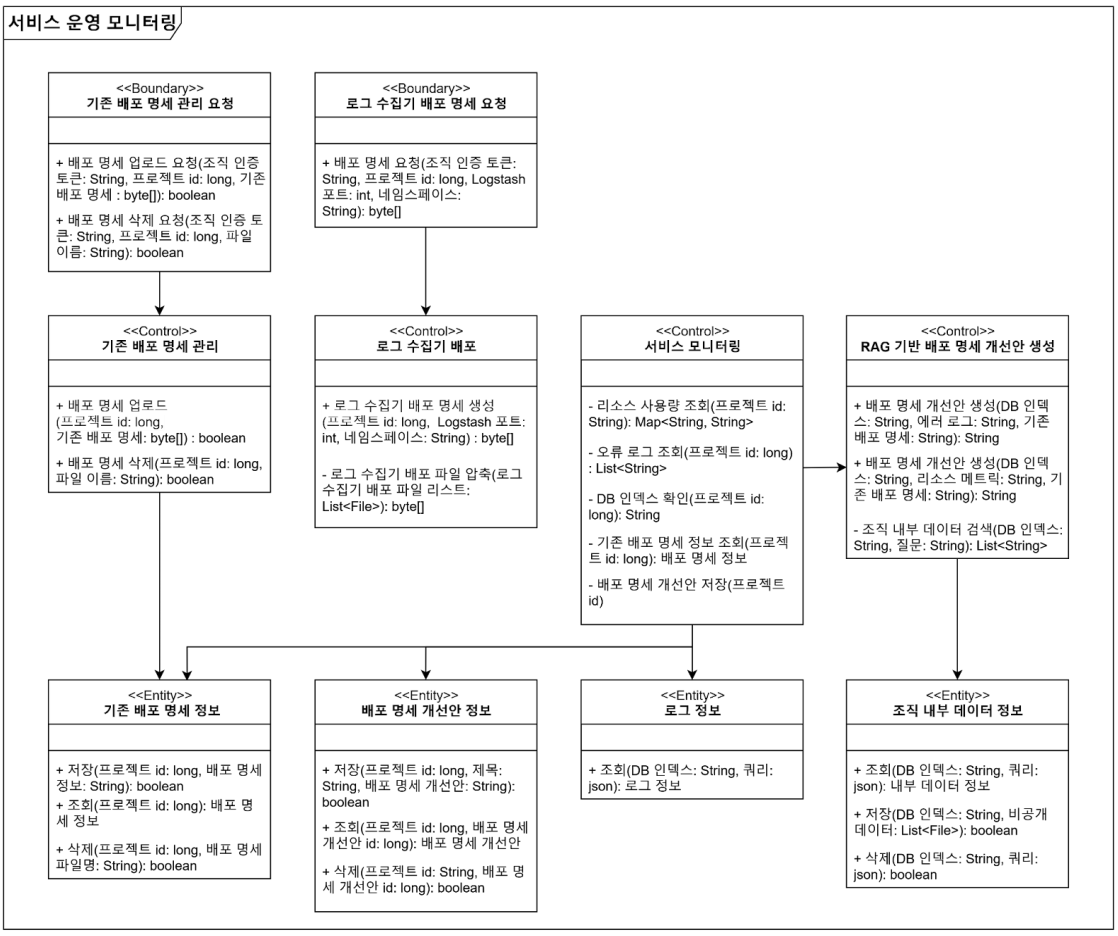


그림 15 서비스 운영 모니터링 클래스 다이어그램

그림과 같이, 기존 배포 명세 관리 요청 클래스는 판단의 기준이 되는 기존 적용된 배포 명세를 시스템에 업로드하거나 삭제하는 인터페이스이다. 기존 배포 명세 관리 클래스에서 구체적인 로직을 수행하고 기존 배포 명세 정보 클래스에서 저장, 조회, 삭제 기능을 담당한다.

로그 수집기 배포 명세 요청 클래스는 사용자의 MSA 애플리케이션에 적용 가능한 로그 수집기 배포 명세를 요청하는 인터페이스이다. 로그 수집기 배포 클래스에서 로그 수집기 배포 명세를 생성하는 로직을 담당한다.

서비스 모니터링 클래스는 주기적으로 프로젝트별 리소스 메트릭, 에러 로그, 기존 배포 명세를 수집해 배포 명세 개선을 요청하고 결과를 저장한다. 개선안 생성 클래스는 로그 및 메트릭 기반으로 내부 데이터를 검색해 RAG 방식으로 개선안을 도출하는 클래스이다.

vi) 배포 명세 개선안 관리

그림 16은 모니터링에서 생성된 배포 명세 개선안을 관리하는 클래스 다이어그램이다.

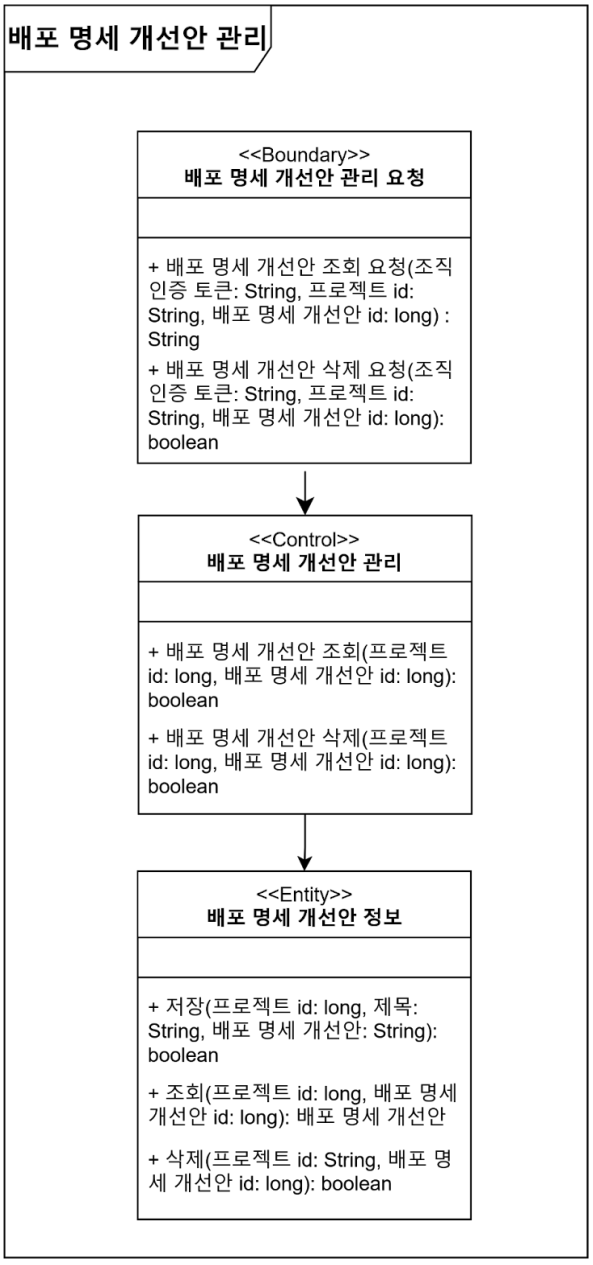


그림 16 배포 명세 개선안 관리 클래스 다이어그램

그림에서처럼, 배포 명세 개선안 관리 요청 클래스는 서비스 운영 모니터링 기능을 통해 생성된 배포 명세 개선안 정보를 조회하거나 삭제하는 인터페이스이다. 배포 명세 개선안 관리에서 조회 및 삭제를 위한 로직을 수행한다.

3.1.4. 시퀀스 다이어그램

i) 조직 계정 생성

그림 17은 조직 계정 생성 시퀀스 다이어그램이다.

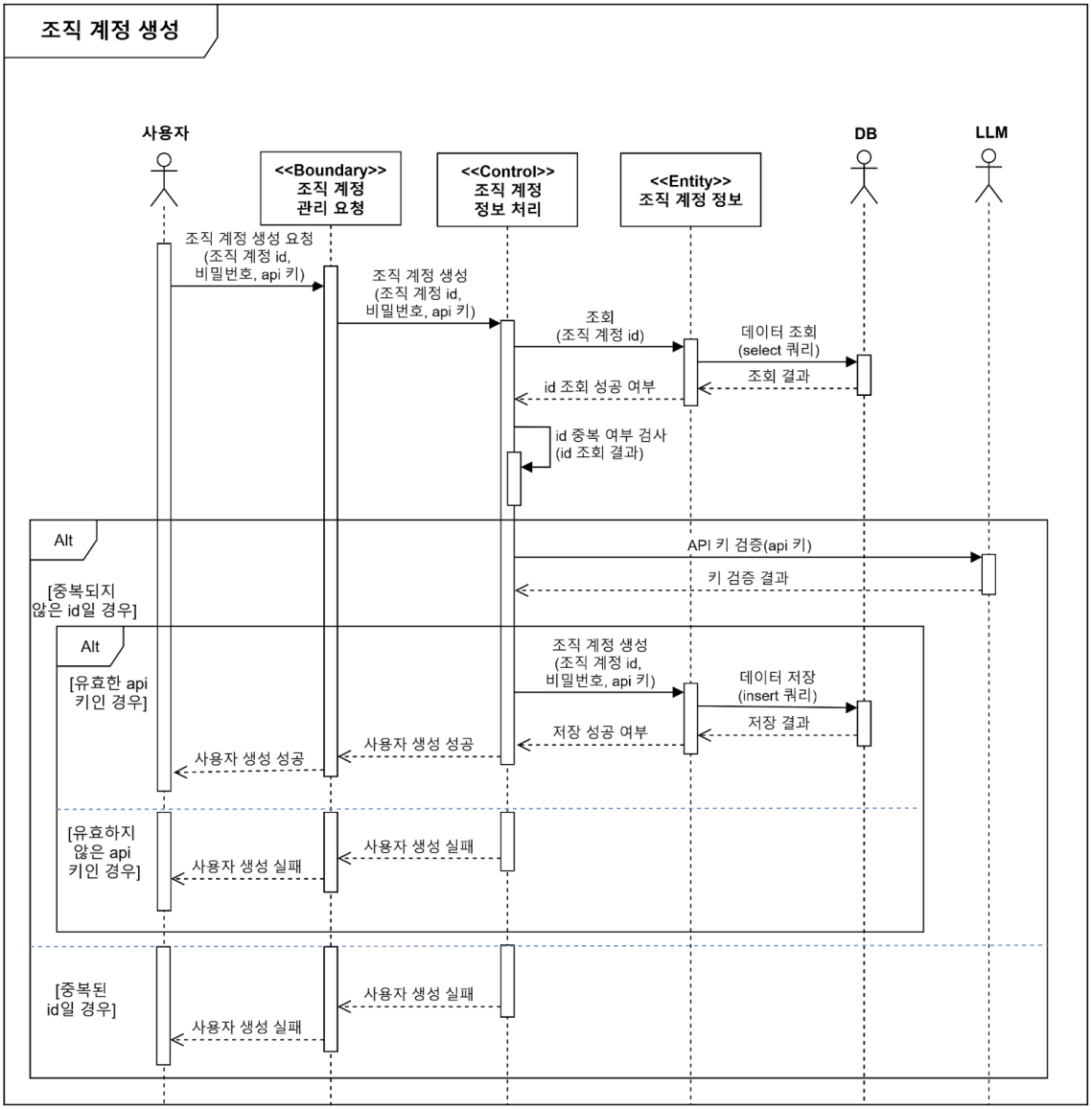


그림 17 조직 계정 생성 시퀀스 다이어그램

그림에 나타낸 바와 같이, 사용자는 id 와 비밀번호, API 키를 입력하고 회원가입을 요청한다. 시스템은 DB 를 조회하여 id 가 중복되는 id 인지 검사한다. 중복되지 않은 id 일 경우 API 키의 유효성을 검사한다. 유효한 API 키인 경우 사용자 정보를 DB 에 저장하고 회원가입 성공 메시지를 출력한다. id 가 중복되었거나 API 키가 유효하지 않을 경우 회원가입 실패 메시지를 출력한다.

ii) 조직 계정 삭제

그림 18은 조직 계정 삭제 시퀀스 다이어그램이다.

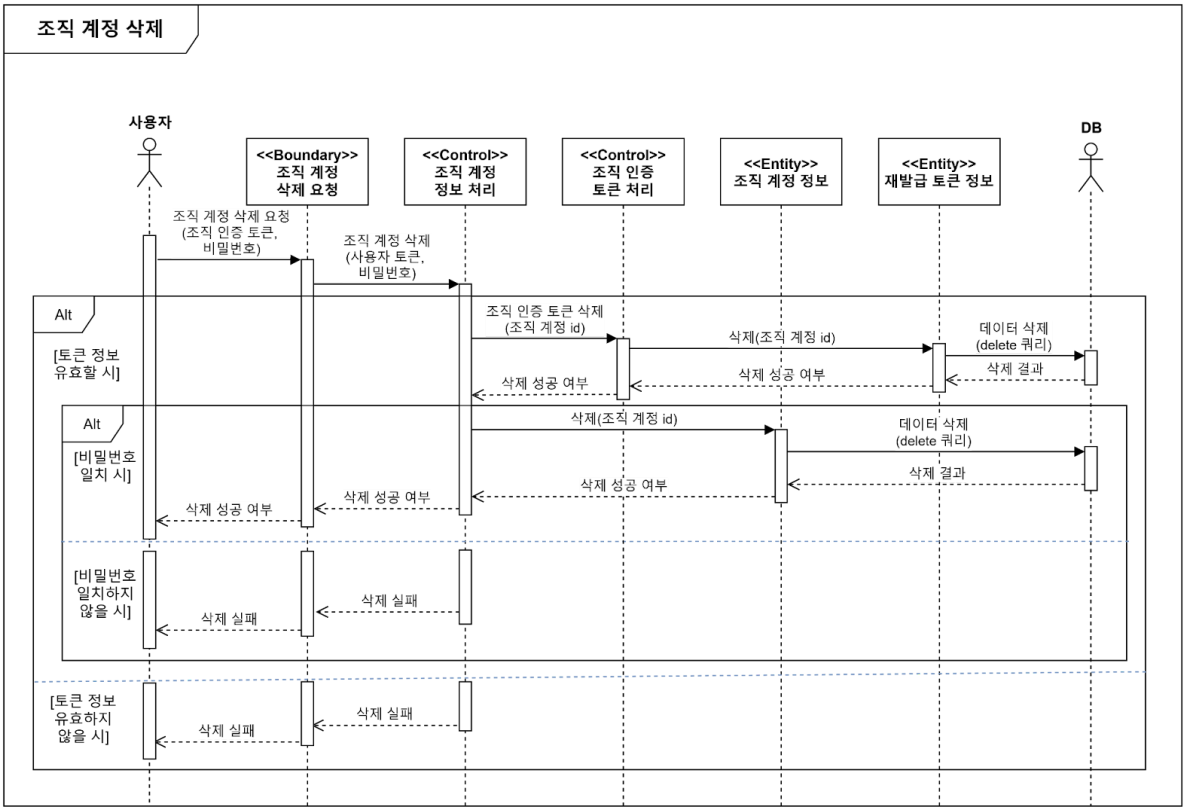


그림 18 조직 계정 삭제 시퀀스 다이어그램

그림에서처럼, 사용자는 사용자 토큰과 사용자 비밀번호를 통해 사용자 정보 삭제를 요청한다. 시스템은 토큰의 만료 여부를 판단하고 유효할 시 비밀번호 일치 여부를 확인한다. 일치할 경우 삭제 후 삭제 성공 응답을 반환한다. 비밀번호가 일치하지 않거나 토큰 정보가 유효하지 않은 경우, 삭제 실패 응답을 반환한다.

iii) 조직 계정 정보 수정

그림 19는 조직 계정 정보 수정 시퀀스 다이어그램이다.

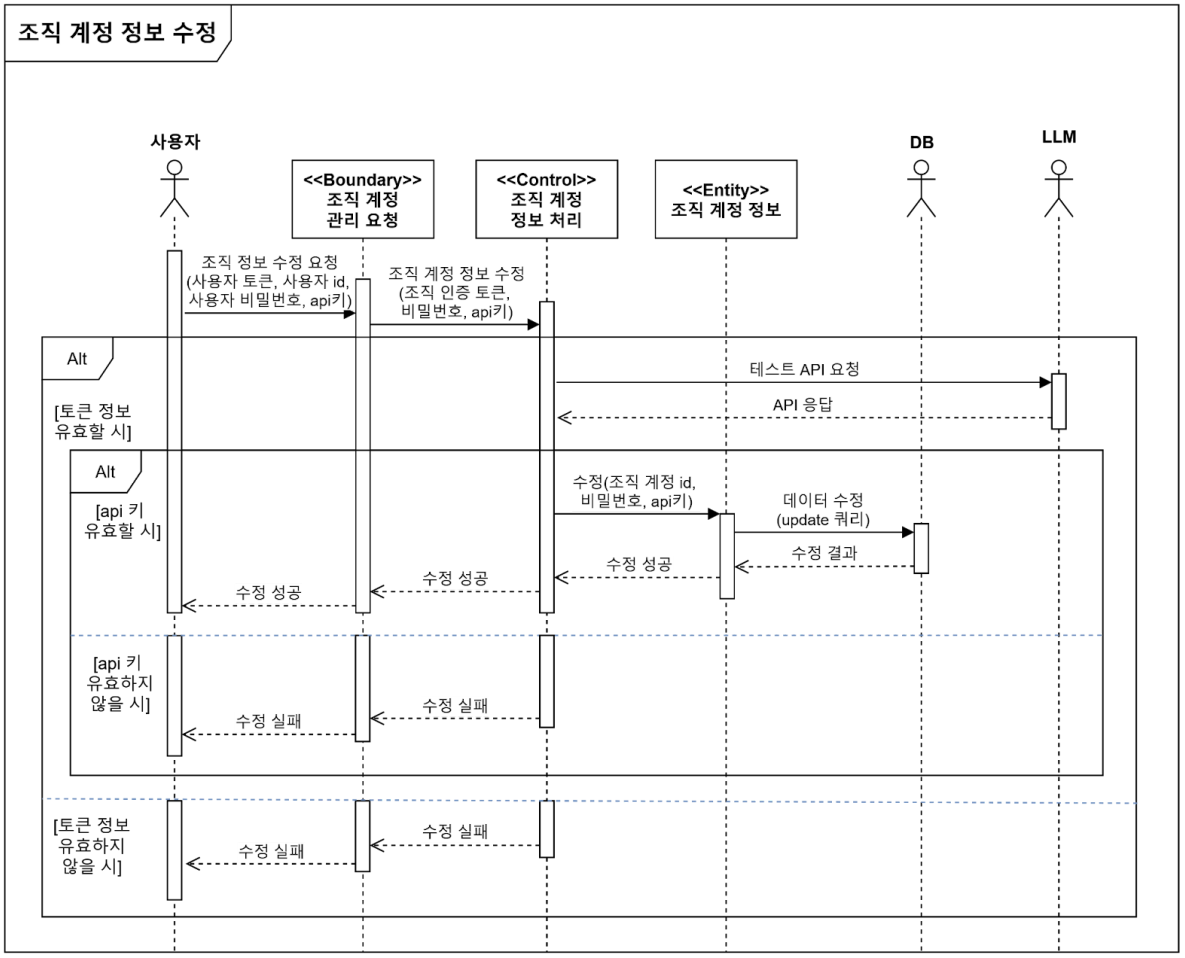


그림 19 조직 계정 정보 수정 시퀀스 다이어그램

그림에서 보듯이, 사용자는 사용자 토큰과 변경하고자 하는 내용의 사용자 id, 사용자 비밀번호, api 키를 입력하고 정보 수정을 요청한다. 시스템은 토큰 정보가 유효할 시 API 키 검증을 수행한다. API 키가 유효하다면 수정할 정보를 DB에 저장하고 성공 응답을 반환한다. API 키가 유효하지 않을 경우 실패 응답을 반환한다. 토큰 정보가 유효하지 않은 경우에도 실패 응답을 반환한다.

iv) 시스템 로그인

그림 20은 시스템 로그인 시퀀스 다이어그램이다.

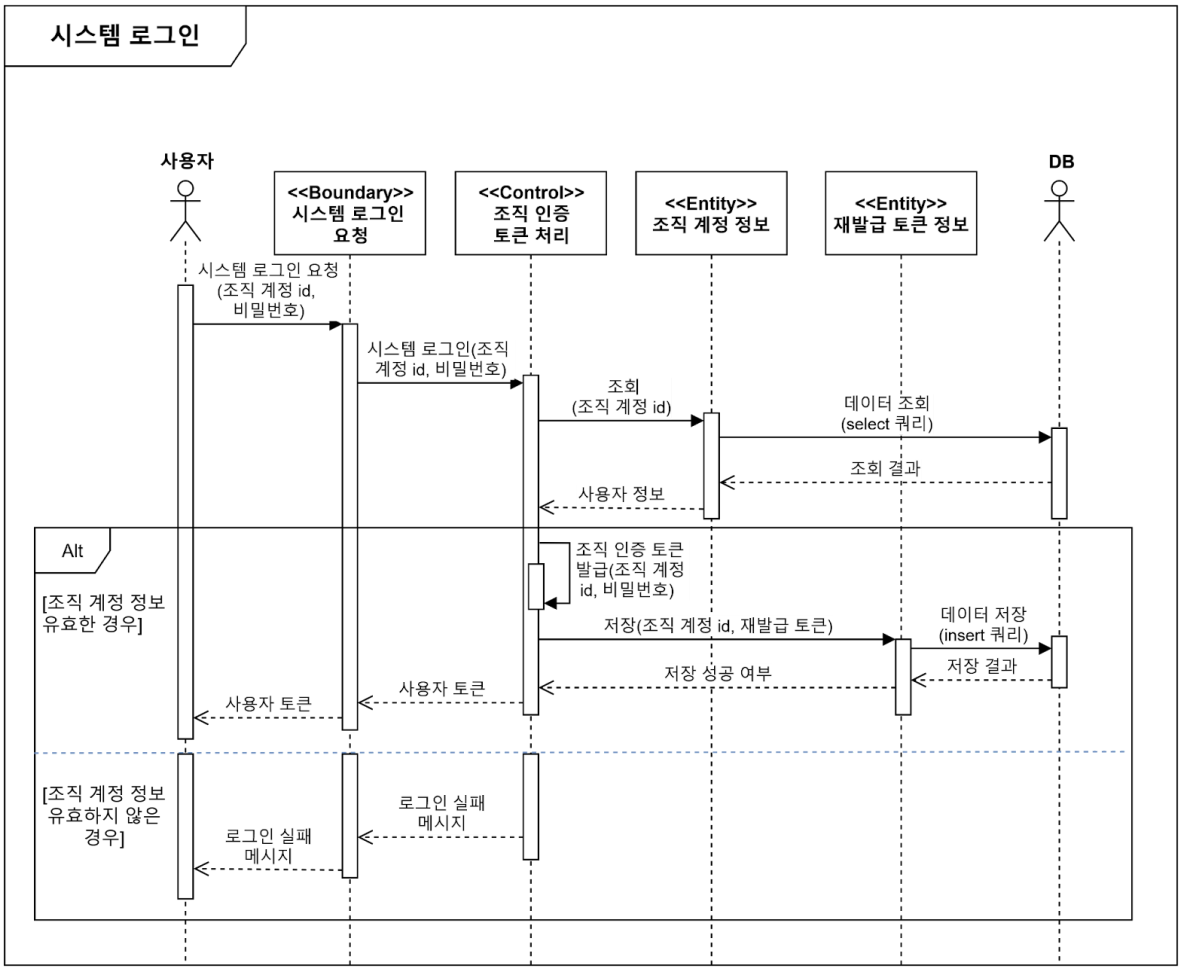


그림 20 시스템 로그인 시퀀스 다이어그램

그림에 따르면, 사용자는 id와 비밀번호를 입력하고 로그인 요청을 수행한다. 시스템은 DB 를 조회해서 유효한 사용자인지 판단한다. 유효한 사용자인 경우 사용자 토큰 처리에서 사용자 토큰을 발급한다. 이 때 재발급 토큰도 함께 발급하는데, 재발급 토큰은 DB 에 저장하고 사용자 토큰은 사용자에게 전달한다. 사용자 정보가 유효하지 않은 경우 로그인 실패 메시지를 출력한다.

v) MSA 프로젝트 생성

그림 21은 MSA 프로젝트 생성 시퀀스 다이어그램이다.

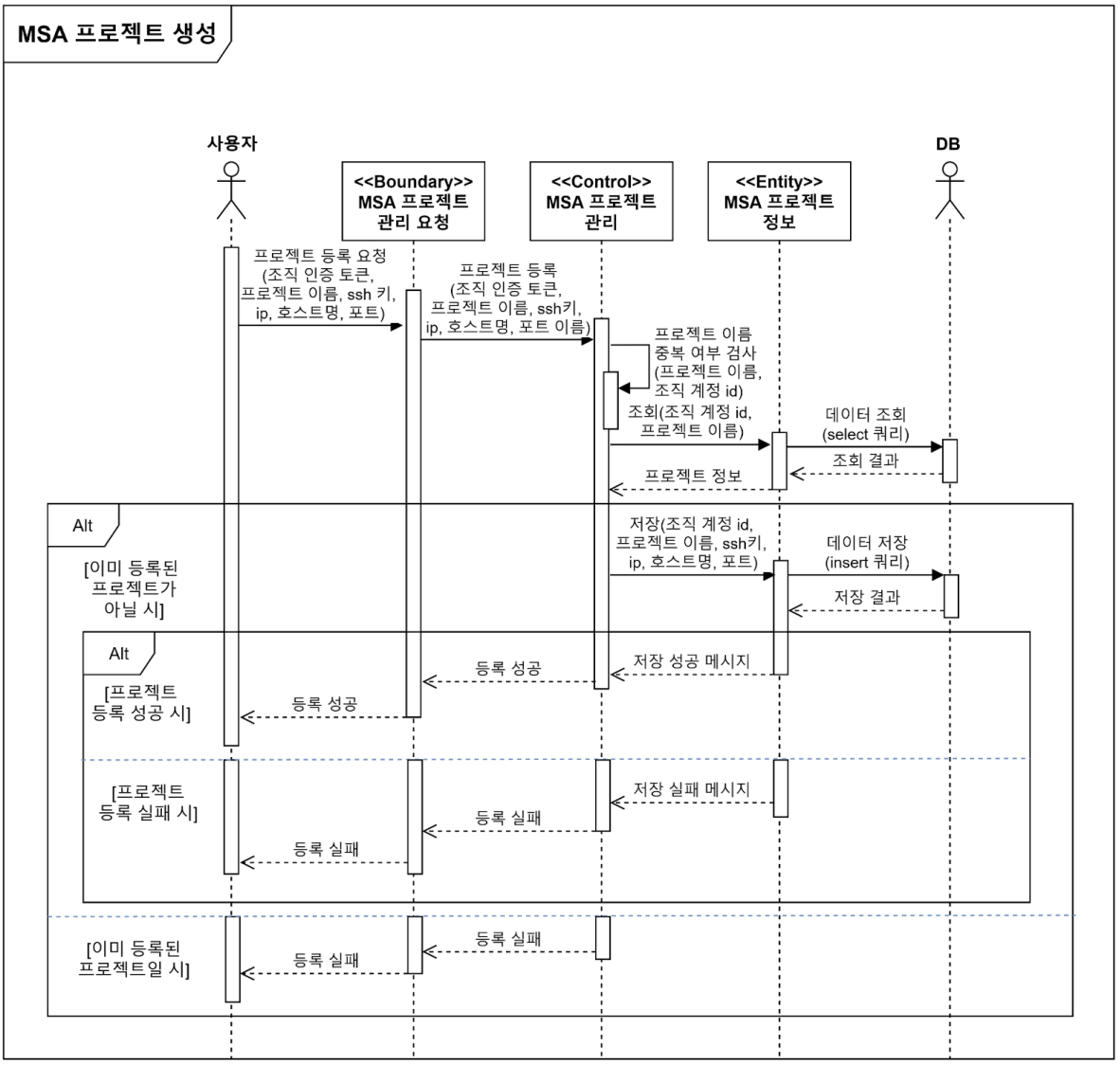


그림 21 MSA 프로젝트 생성 시퀀스 다이어그램

그림과 같이, 사용자는 조직 인증 토큰과 프로젝트 이름, ssh 키, ip, 호스트명, 포트를 입력하여 프로젝트를 생성한다. 토큰 정보가 유효할 시, 시스템은 DB를 조회해서 프로젝트 이름 중복 여부를 검사하고, 프로젝트 이름이 유효할 시, 입력받은 정보를 DB에 저장하고 생성 완료 메시지를 사용자에게 전달한다. 프로젝트 이름이 중복되거나 DB에 저장 실패 시, 프로젝트 등록 실패 메시지를 사용자에게 전달한다.

vi) MSA 프로젝트 정보 수정

그림 22는 MSA 프로젝트 정보 수정 시퀀스 다이어그램이다.

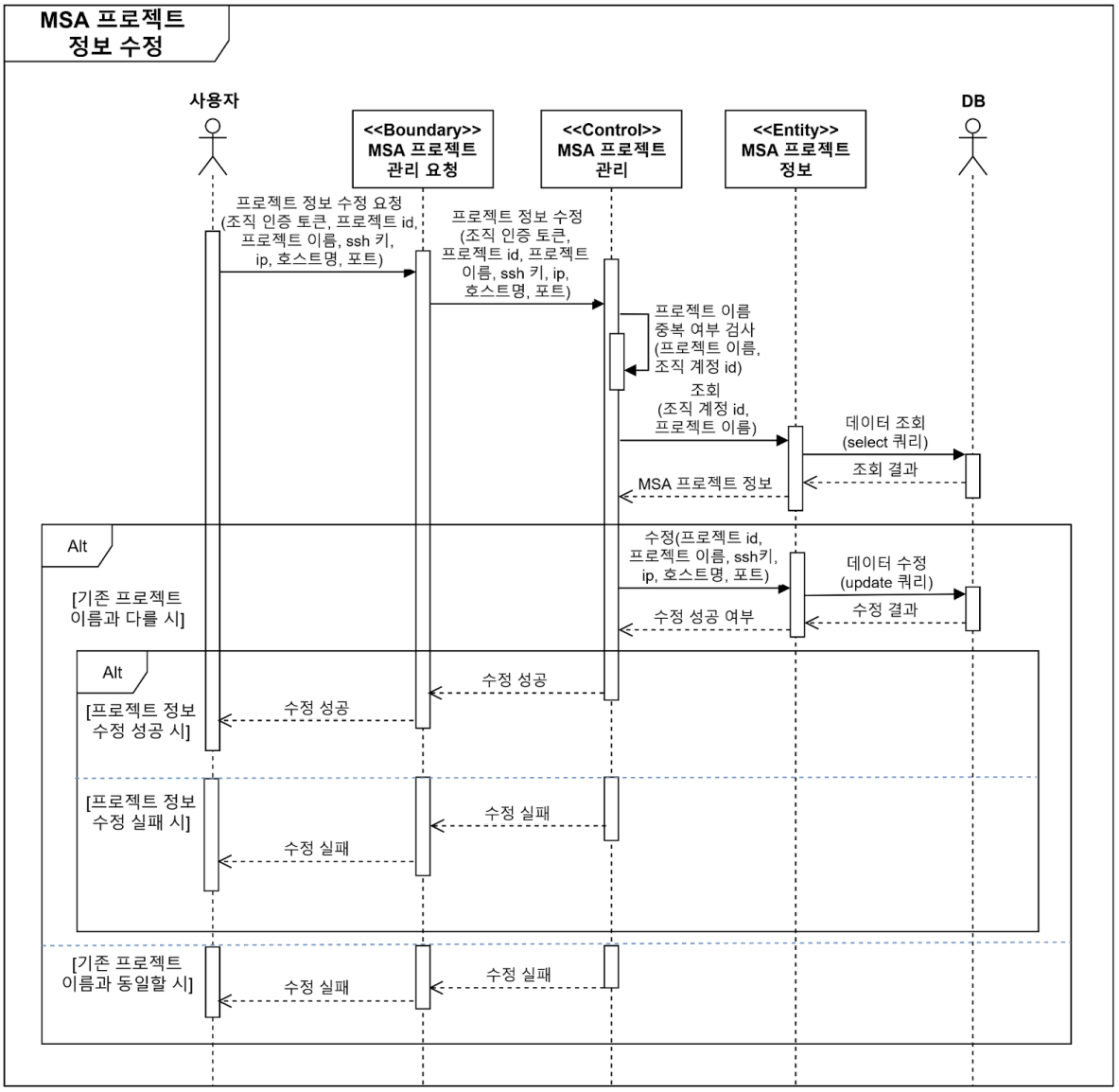


그림 22 MSA 프로젝트 정보 수정 시퀀스 다이어그램

그림과 같이, 사용자는 조직 인증 토큰과 프로젝트 이름, ssh 키, ip, 호스트명, 포트를 입력하여 프로젝트 정보 수정을 요청한다. 토큰 정보가 유효할 시, 시스템은 DB를 조회해서 프로젝트 이름 중복 여부를 검사하고, 프로젝트 이름이 유효할 시, 입력받은 정보를 바탕으로 해당 프로젝트 정보를 DB에 업데이트 하고 수정 완료 메시지를 사용자에게 전달한다. 프로젝트 이름이 중복되거나 DB에 업데이트 실패 시, 프로젝트 정보 수정 실패 메시지를 사용자에게 전달한다.

vii) MSA 프로젝트 삭제

그림 23은 MSA 프로젝트 삭제 시퀀스 다이어그램이다.

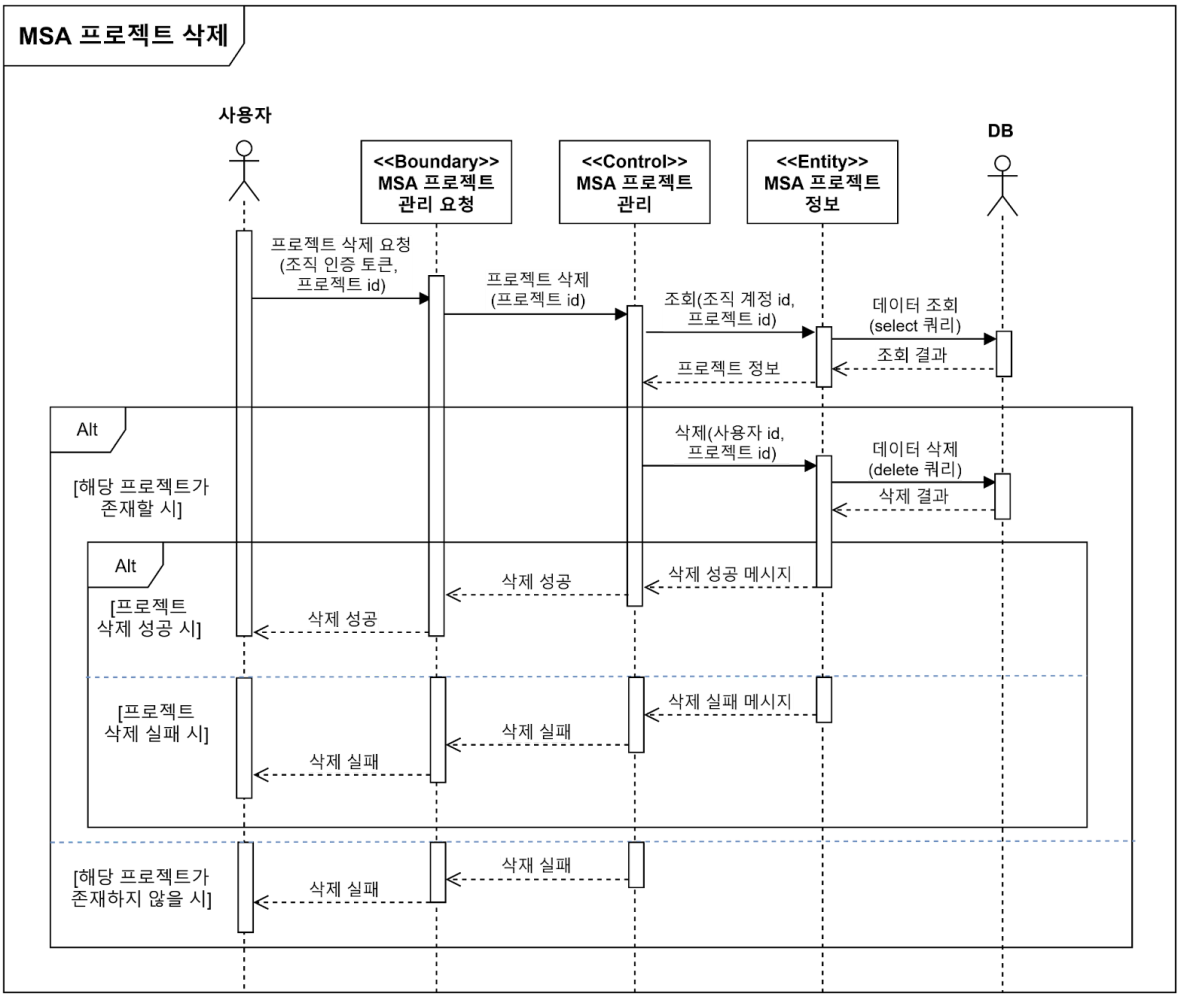


그림 23 MSA 프로젝트 삭제 시퀀스 다이어그램

그림에 제시된 바와 같이, 사용자는 조직 인증 토큰을 가지고 프로젝트 삭제 요청을 수행한다. 토큰 정보가 유효할 시, 시스템은 DB를 조회해서 해당 프로젝트 정보를 조회하고 DB에서 삭제 후 사용자에게 삭제 성공 메시지를 전달한다. 해당 프로젝트에 DB에 존재하지 않거나 삭제 중 오류가 발생 시, 삭제 실패 메시지를 전달한다.

viii) MSA 프로젝트 원격접속

그림 24는 MSA 프로젝트 원격 접속 시퀀스 다이어그램이다.

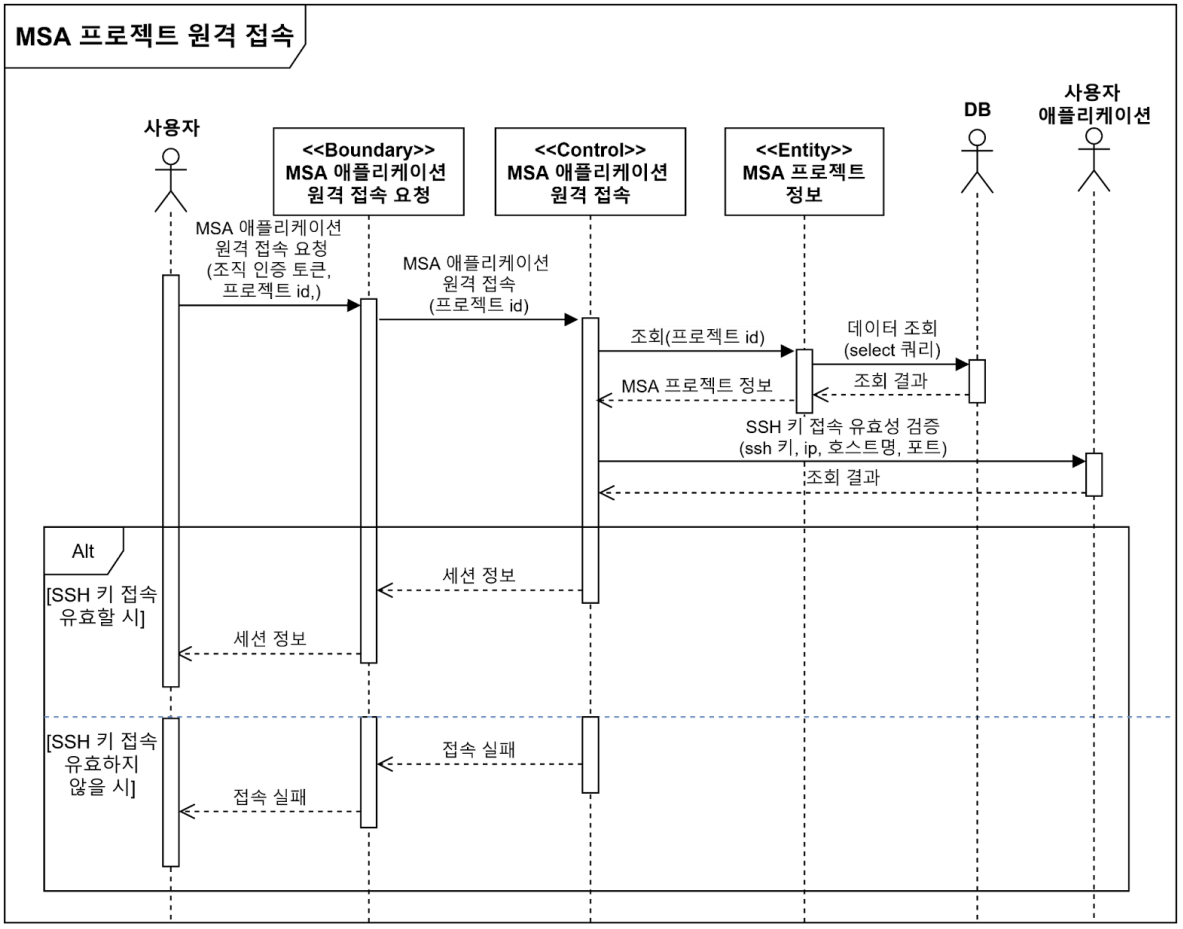


그림 24 MSA 프로젝트 원격 접속 시퀀스 다이어그램

그림에 제시된 바와 같이, 사용자는 특정 프로젝트의 MSA 애플리케이션에 대해 원격 접속을 요청한다. 시스템은 DB에서 해당 프로젝트의 SSH 키, pem 파일, IP 주소, 접속 계정명 등 접속 정보를 조회하여 유효성을 검증한다. 정보가 유효할 경우, 시스템은 해당 정보로 원격 서버와 연결을 시도한다. 연결에 시도하면 시스템은 사용자에게 원격 접속 세션을 제공하여 웹 터미널을 통해 프로젝트 환경에 접근할 수 있도록 한다. 만약 접속 정보가 유효하지 않거나 인증에 실패하는 경우, 시스템은 사용자에게 오류 메시지를 출력하고 접속을 중단한다.

ix) 조직 내부 데이터 업로드

그림 25는 조직 내부 데이터 업로드 시퀀스 다이어그램이다.

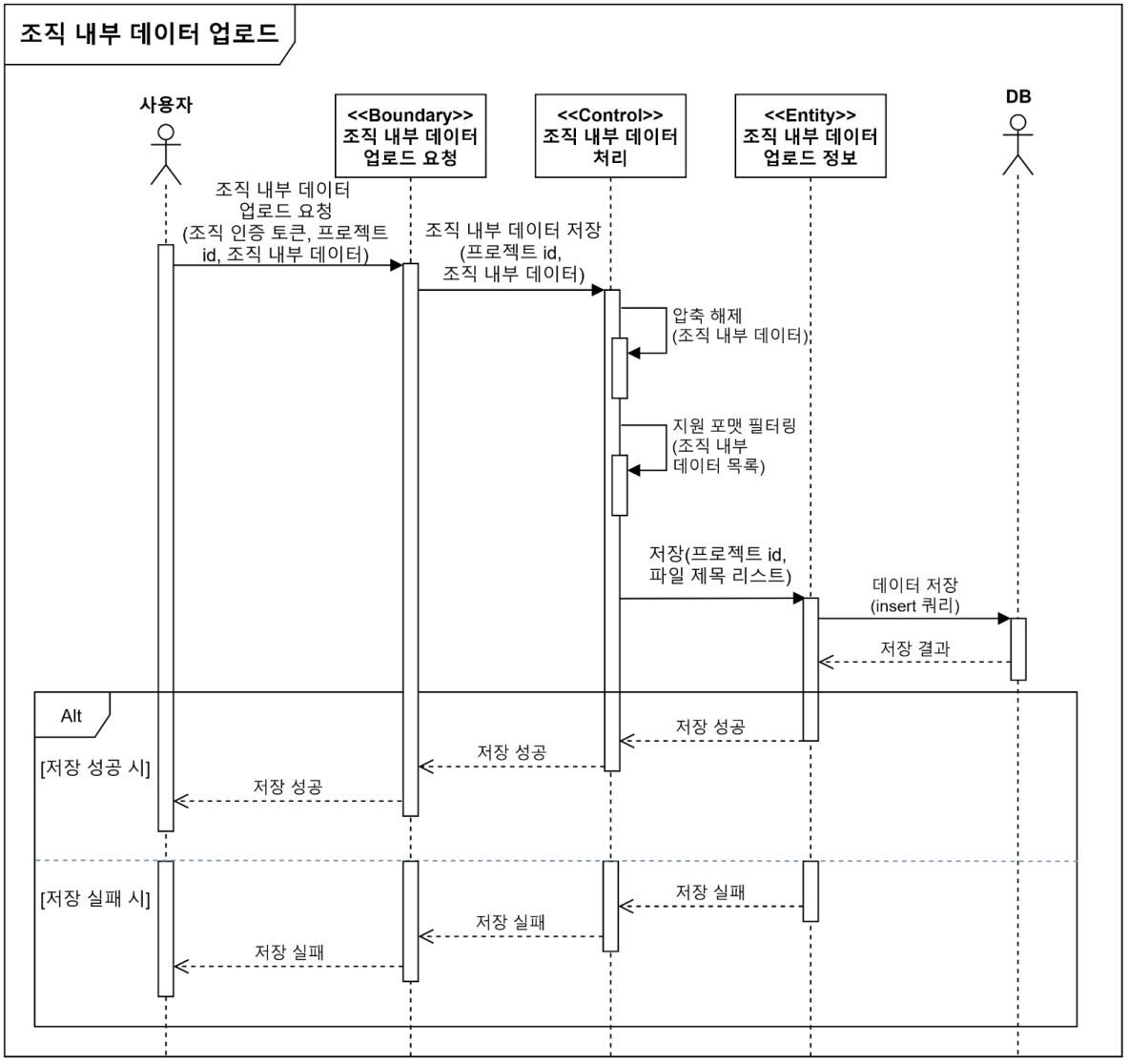


그림 25 조직 내부 데이터 업로드 시퀀스 다이어그램

그림에 명시된 바와 같이, 사용자는 조직 내부 데이터를 .zip 파일 형태로 시스템에 업로드한다. 시스템은 전달받은 파일의 압축을 해제하고, 실행 파일(.exe, .sh, .bash 등)을 제외한 유효한 포맷의 파일을 필터링한다. 이후 필터링 된 파일의 내용을 UTF-8 형식의 텍스트로 추출하고 추출된 텍스트 내용을 벡터 임베딩하여 데이터베이스에 저장한다. 저장에 성공하면 사용자에게 성공 메시지를, 과정 중 오류가 발생하면 실패 메시지를 전달한다.

x) 조직 내부 데이터 삭제

그림 26은 조직 내부 데이터 삭제 시퀀스 다이어그램이다.

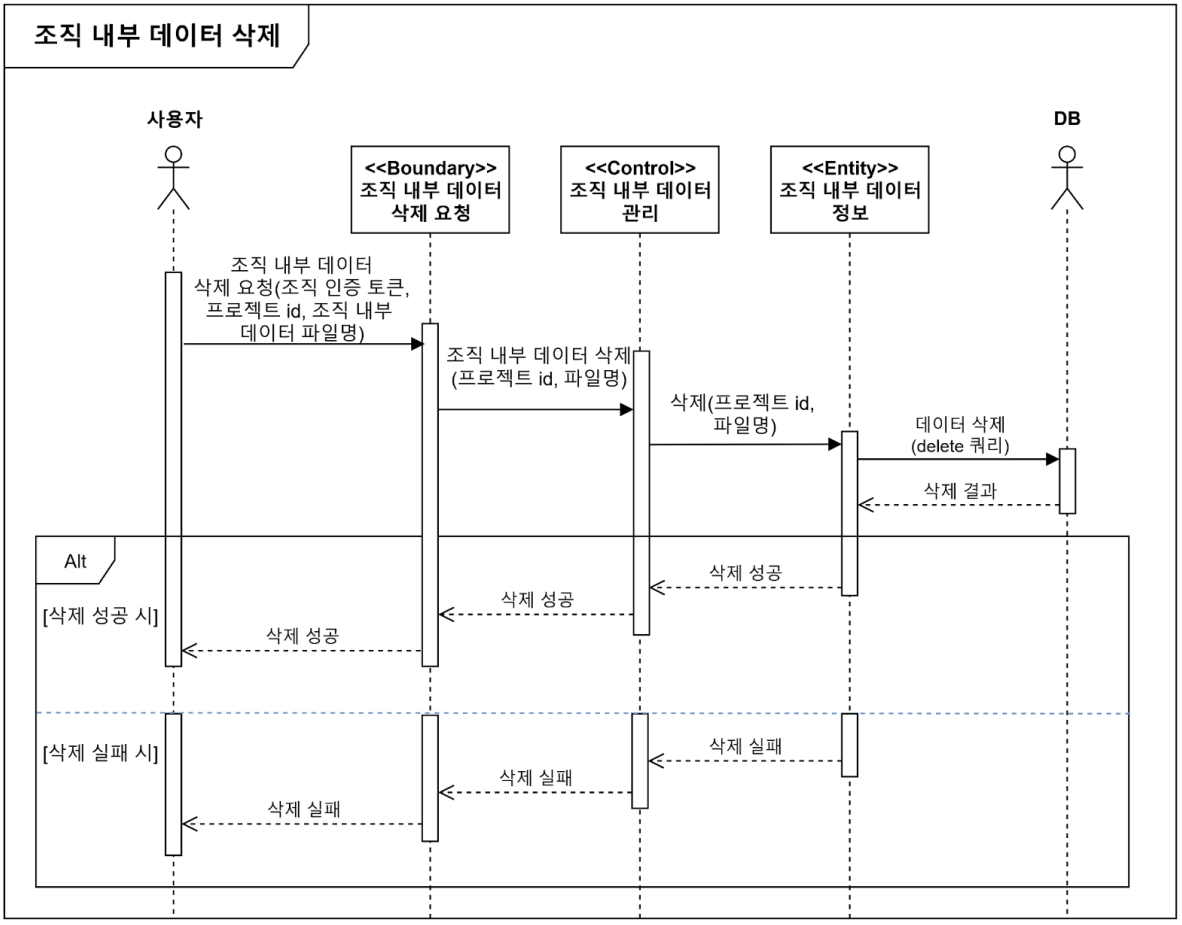


그림 26 조직 내부 데이터 삭제 시퀀스 다이어그램

그림과 같이, 사용자는 시스템에 저장된 조직 내부 데이터 목록에서 삭제를 원하는 파일을 선택하여 삭제를 요청한다. 시스템은 DB에서 삭제 쿼리를 실행하여 해당 파일의 데이터를 제거하고 사용자에게 삭제 성공 메시지를 전달한다. 삭제에 실패하면 사용자에게 삭제 실패 메시지를 전달한다.

xi) 배포 명세 생성

그림 27은 배포 명세 생성 시퀀스 다이어그램이다.

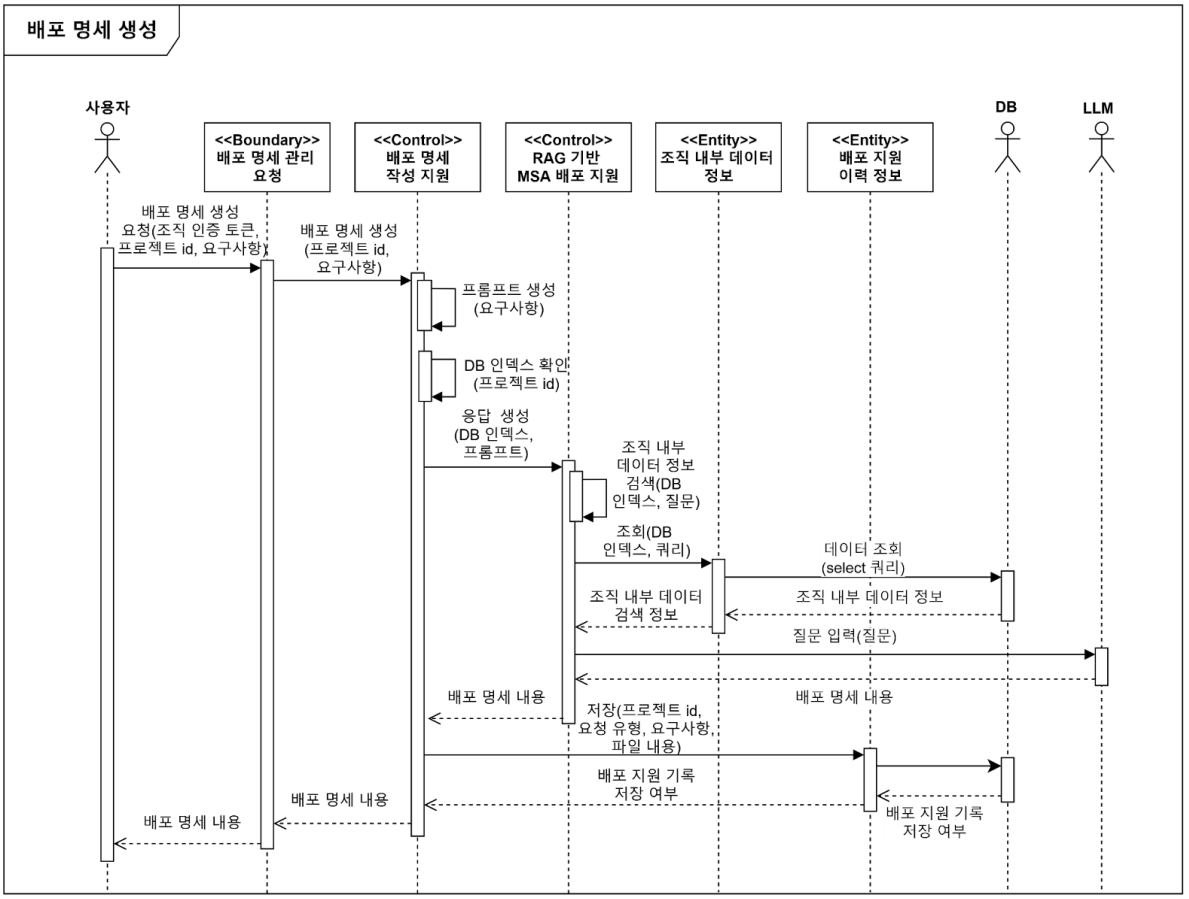


그림 27 배포 명세 생성 시퀀스 다이어그램

그림에 명시된 바와 같이, 사용자가 질의 유형을 배포 생성으로 설정하고 배포할 서비스에 대한 설명을 자연어로 입력하여 생성을 요청하면, 시스템은 질의 유형을 기반으로 LLM에게 전달할 프롬프트를 생성한다. 그 다음 시스템은 해당 프로젝트의 조직 내부 데이터가 저장되어 있는 프로젝트 id 인덱스를 DB에서 유사도 검색하여 배포 명세 생성을 위한 조직 내부 데이터를 조회한다. 시스템은 이렇게 조회된 조직 내부 데이터와 사용자 요청에 기반하여 생성된 프롬프트를 함께 LLM에 전달하여 배포 명세 생성을 요청한다. LLM으로부터 생성된 배포 명세 내용을 전달받은 시스템은 해당 내용을 사용자에게 출력해서 보여줌과 함께 DB에 이력으로 저장한다.

xii) 배포 명세 오류 수정

그림 28은 배포 명세 오류 수정 시퀀스 다이어그램이다.

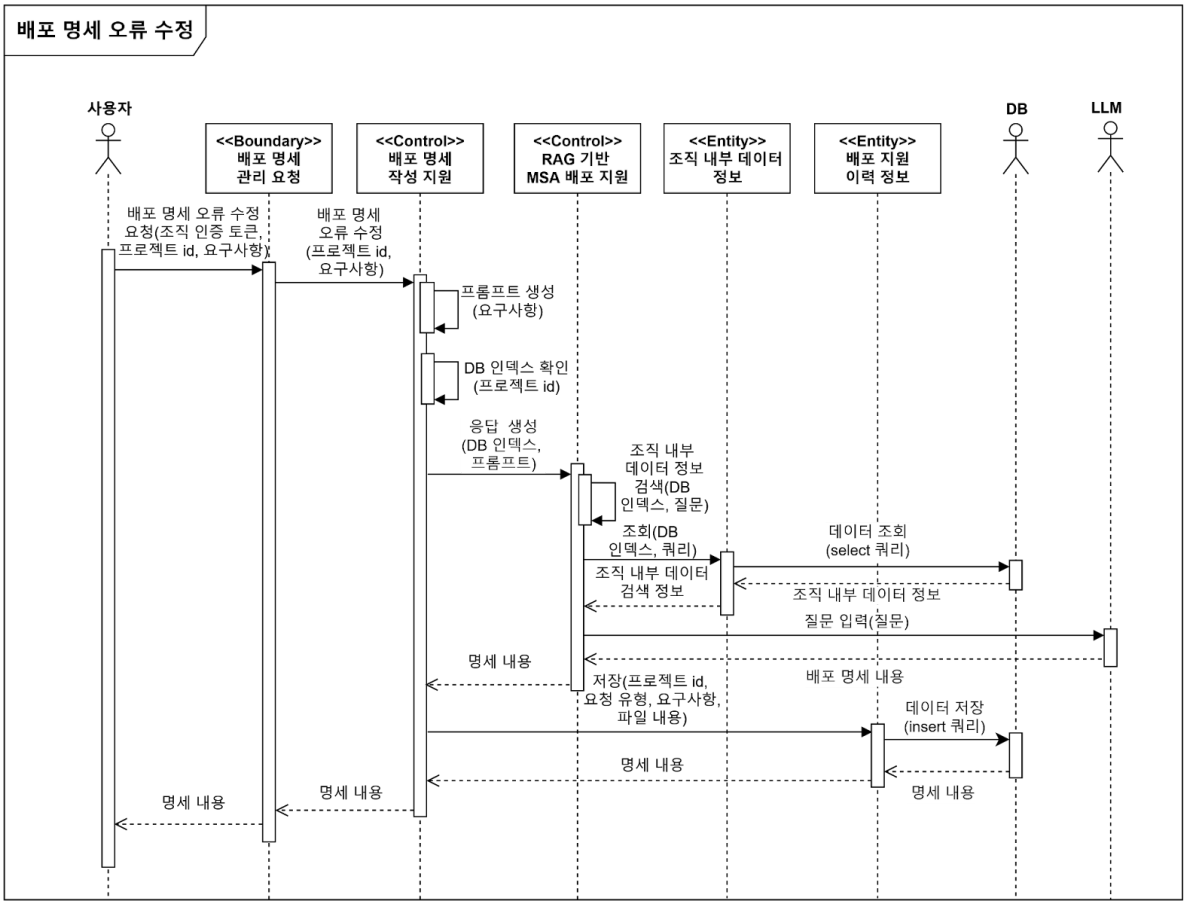


그림 28 배포 명세 오류 수정 시퀀스 다이어그램

그림에서처럼, 사용자가 질의 유형을 오류 수정으로 설정한 후, 오류가 발생한 상황이나 수정 요구사항을 자연어로 입력하고 배포에 오류가 발생하는 배포 명세 파일을 시스템에 업로드하여 배포 명세 오류 수정 요청을 보낸다. 시스템은 사용자의 자연어 질의와 업로드한 배포 명세 파일의 내용을 요구사항으로 묶고, 질의 유형을 바탕으로 LLM에게 전달할 프롬프트를 생성한다. 이후 조직 내부 데이터가 저장되어 있는 프로젝트 id 인덱스를 DB에서 조회하여 오류 수정에 필요한 조직 내부 데이터를 조회한다. 시스템은 이렇게 조회된 데이터와 생성된 프롬프트를 종합하여 LLM에 배포 명세 수정안 생성을 요청한다. LLM으로부터 수정된 배포 명세 내용을 전달받으면, 시스템은 해당 내용을 사용자에게 출력하여 보여줌과 함께 DB에 이력으로 저장한다.

xiii) 배포 지원 기록 조회

그림 29는 배포 지원 이력 조회 시퀀스 다이어그램이다.

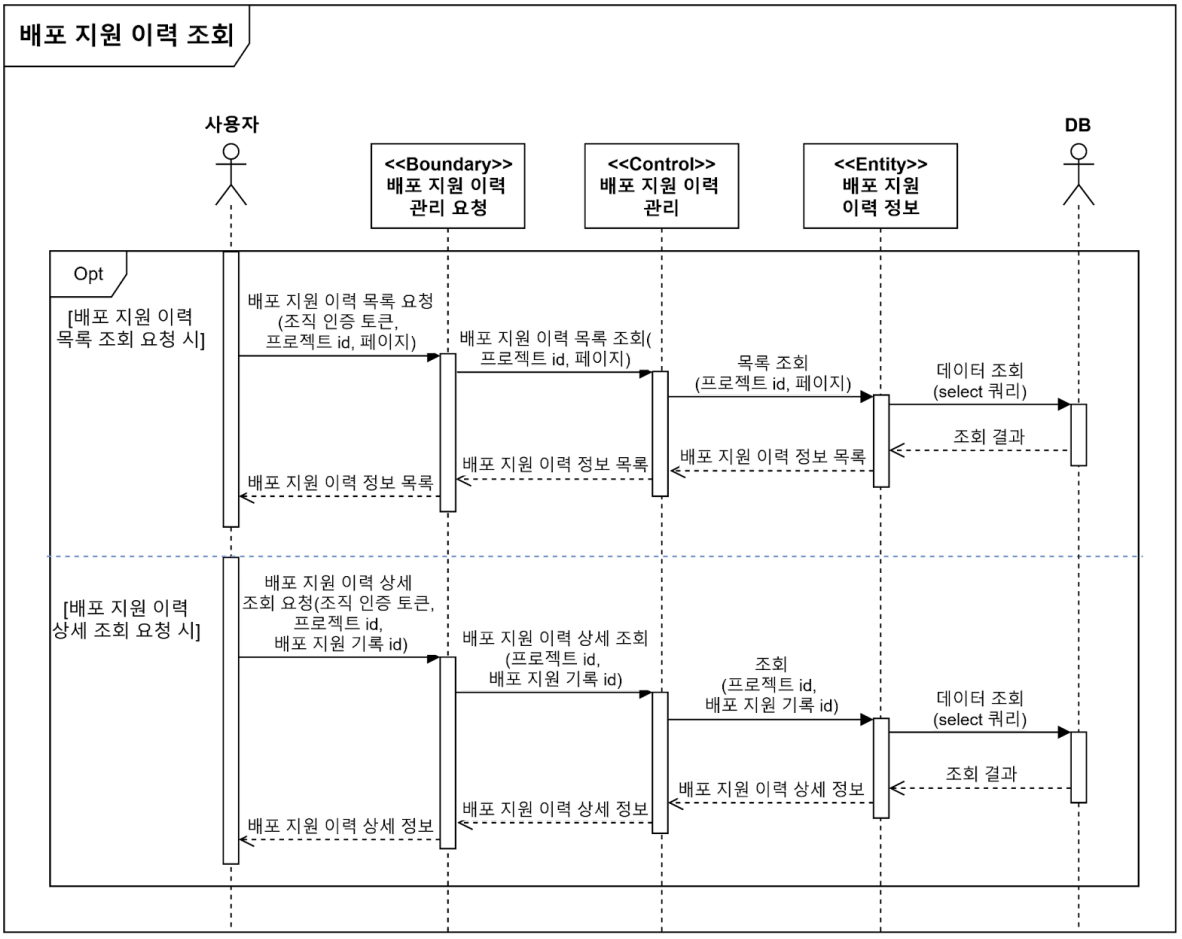


그림 29 배포 지원 이력 조회 시퀀스 다이어그램

그림 내용과 같이, 사용자는 배포 지원 이력을 조회할 프로젝트를 선택하고 조회를 요청한다. 시스템은 해당 프로젝트의 배포 지원 이력 목록을 데이터베이스에서 조회한다. 시스템은 조회에 성공한 경우 사용자에게 조회 성공 결과를 보여주고 실패 시 조회 실패 결과를 출력한다.

xiv) 배포 지원 기록 삭제

그림 30은 배포 지원 이력 삭제 시퀀스 다이어그램이다.

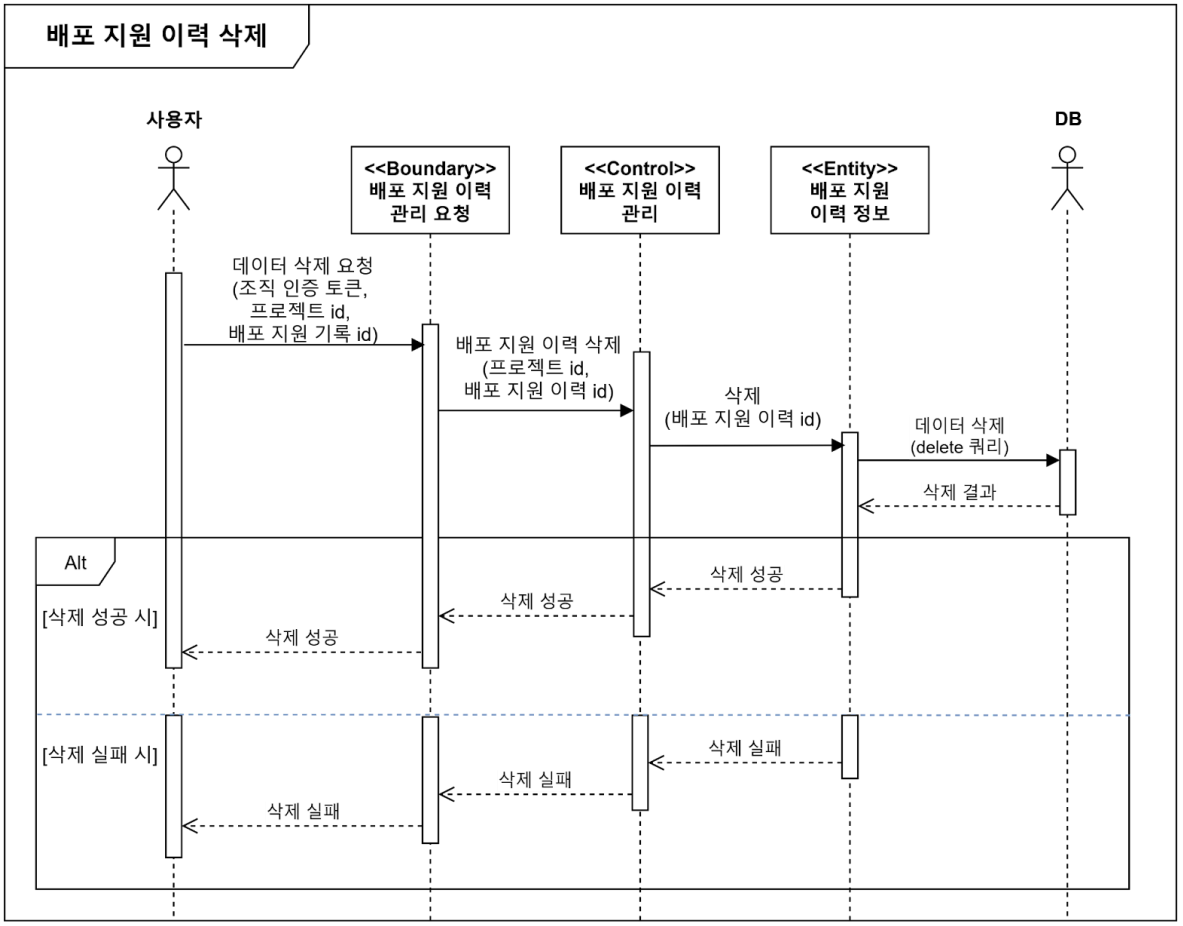


그림 30 배포 지원 이력 삭제 시퀀스 다이어그램

그림에서처럼, 사용자는 삭제할 배포 지원 기록을 선택하고 삭제를 요청한다. 시스템은 해당 배포 지원 기록을 데이터베이스에서 삭제한다. 시스템은 삭제에 성공한 경우 사용자에게 삭제 성공 결과를 보여주고 실패 시 삭제 실패 결과를 출력한다.

xv) 로그 수집기 배포

그림 31은 로그 수집기 배포 시퀀스 다이어그램이다.

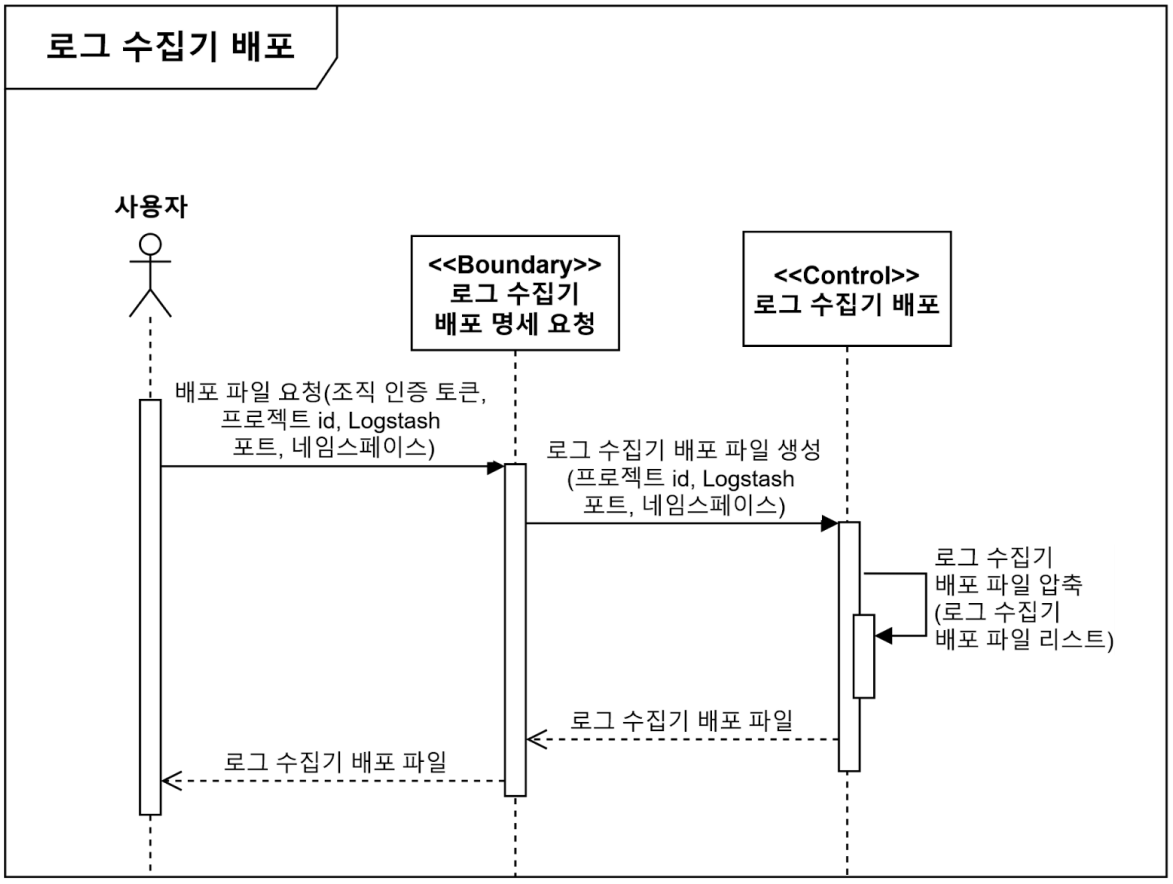


그림 31 로그 수집기 배포 시퀀스 다이어그램

그림에서처럼, 사용자는 로그를 수집할 대상 애플리케이션의 네임스페이스와 Logstash 포트 번호 등 필요한 정보를 시스템에 입력한다. 시스템은 사용자가 입력한 정보를 바탕으로, 사용자 애플리케이션 환경에 맞게 구성된 로그 수집기(Filebeat, Logstash, Metricbeat) 배포 파일을 생성하여 사용자에게 제공한다. 사용자는 이 배포 파일을 자신의 애플리케이션에 적용하여 로그 수집기 배포를 완료한다.

xvi) 기존 배포 명세 업로드

그림 32는 기존 배포 명세 업로드 시퀀스 다이어그램이다.

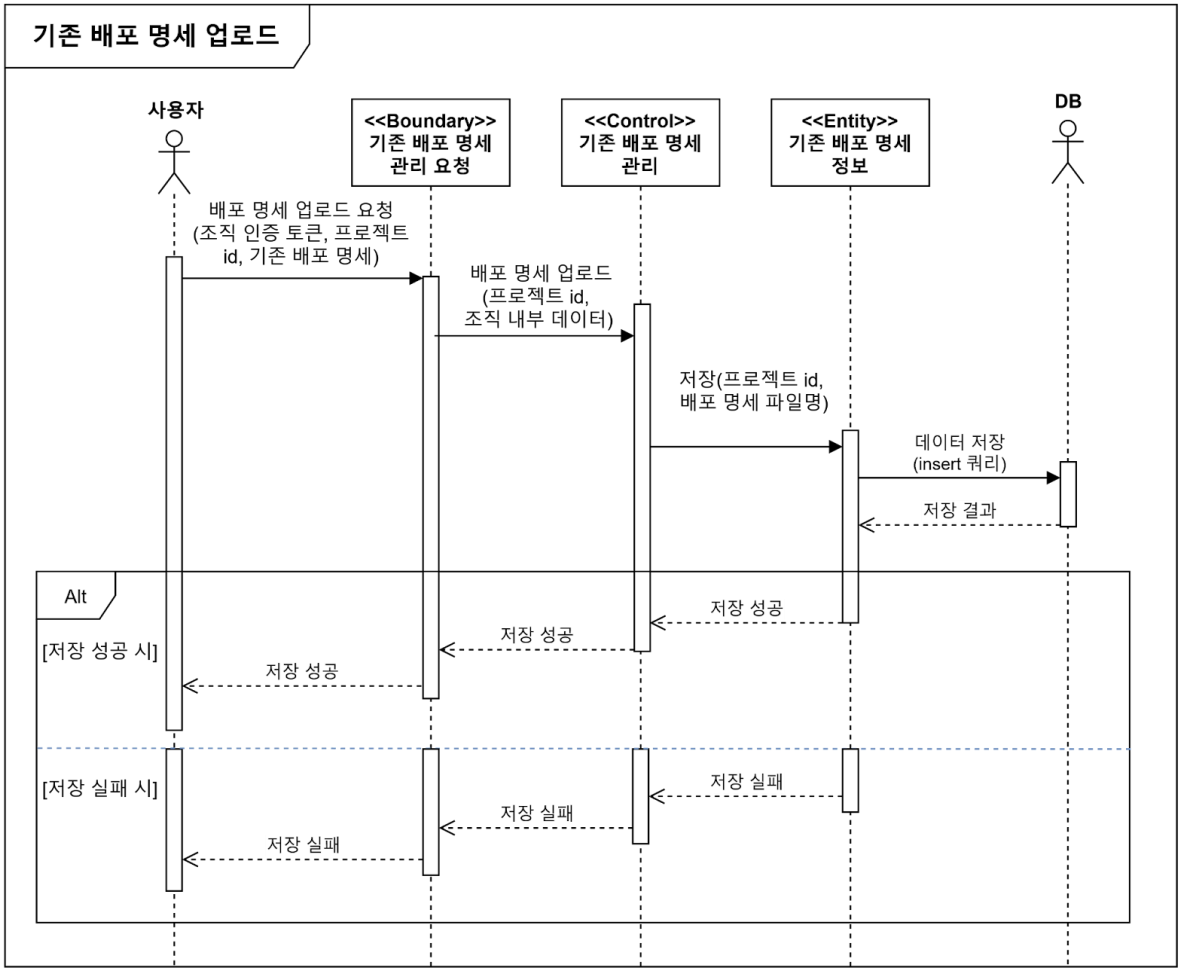


그림 32 기존 배포 명세 업로드 시퀀스 다이어그램

그림에서처럼, 사용자는 기존 프로젝트에 업로드할 기존 배포 명세를 선택하고 업로드를 요청한다. 시스템은 배포 명세를 데이터베이스에서 저장한다. 시스템은 저장에 성공한 경우 사용자에게 저장 성공 결과를 보여주고 실패 시 저장 실패 결과를 출력한다.

xvii) 기존 배포 명세 삭제

그림 33은 기존 배포 명세 삭제 시퀀스 다이어그램이다.

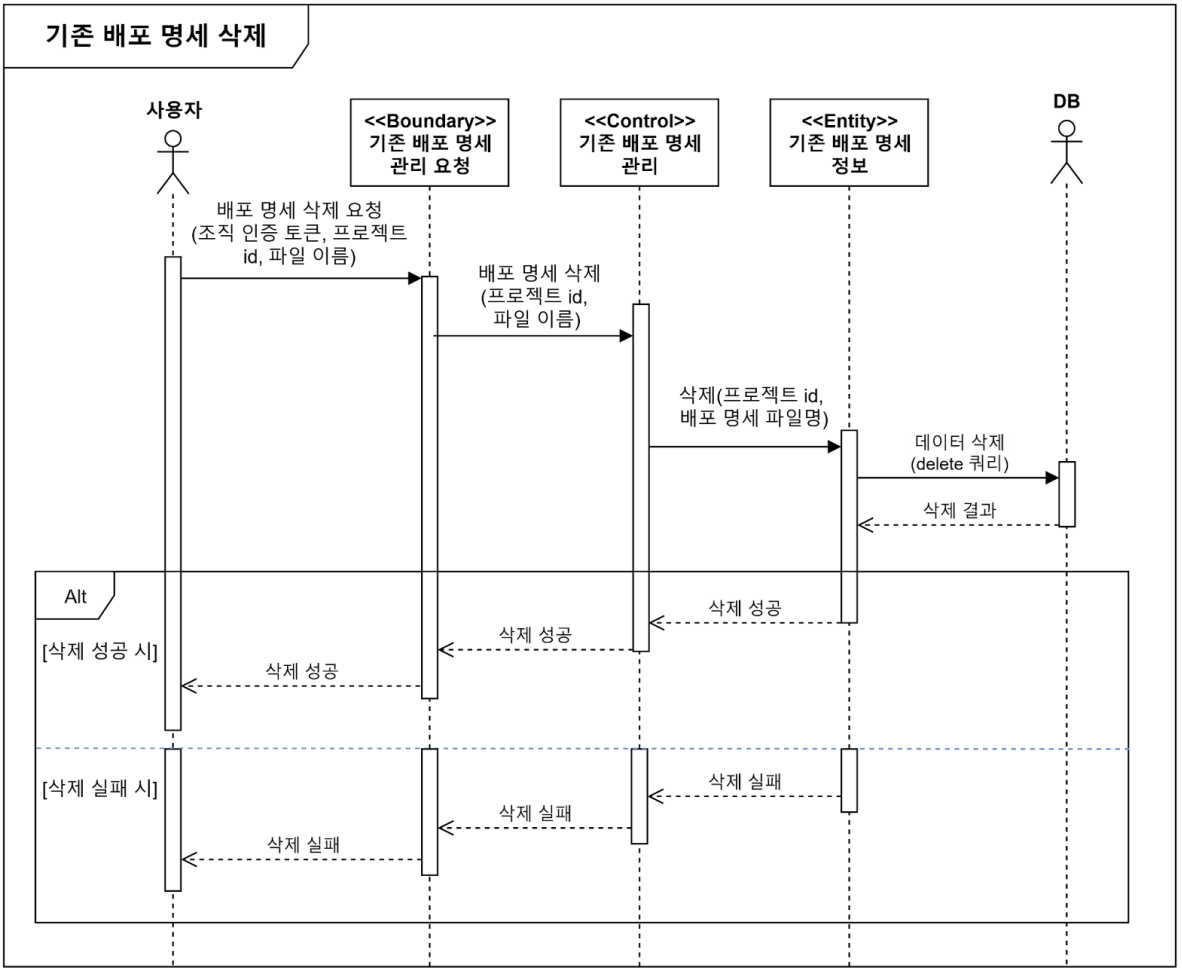


그림 33 기존 배포 명세 삭제 시퀀스 다이어그램

그림과 같이, 사용자는 삭제할 배포 명세를 선택하고 삭제를 요청한다. 시스템은 해당 배포 명세를 데이터베이스에서 삭제한다. 시스템은 삭제에 성공한 경우 사용자에게 삭제 성공 결과를 보여주고 실패 시 삭제 실패 결과를 출력한다.

xviii) 에러 로그 모니터링

그림 34는 에러 로그 모니터링 시퀀스 다이어그램이다.

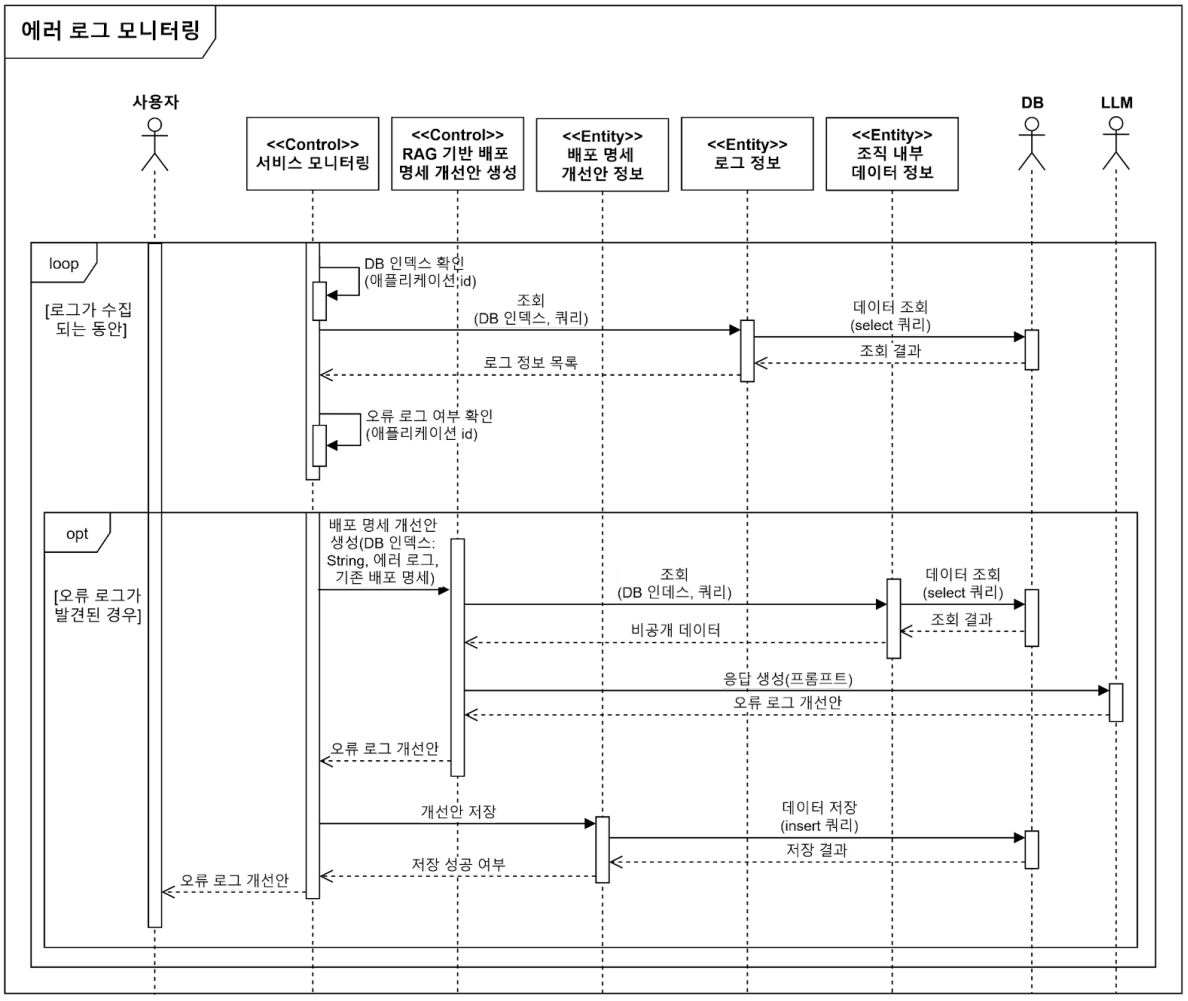


그림 34 에러 로그 모니터링 시퀀스 다이어그램

그림에 제시된 바와 같이, 시스템은 스케줄러에 의해 3분 주기로 DB에서 최신 로그를 조회하고 에러 로그가 있는지 지속적으로 확인한다. 만약 오류 로그가 발생하면 RAG 기술을 사용하여 개선된 배포 명세 파일을 생성한다. 시스템은 조직 내부 데이터가 저장된 프로젝트 id 인덱스를 DB에서 조회하고 발견된 오류 로그 정보와 함께 LLM에 전달하여 배포 명세 개선안을 생성하도록 요청한다. 생성된 배포 명세 개선안은 DB에 저장되며, 모니터링 페이지의 배포 명세 개선안 목록에서 해당 이력을 확인 및 조회 할 수 있다.

xix) 운영 메트릭 모니터링

그림 35는 운영 메트릭 모니터링 시퀀스 다이어그램이다.

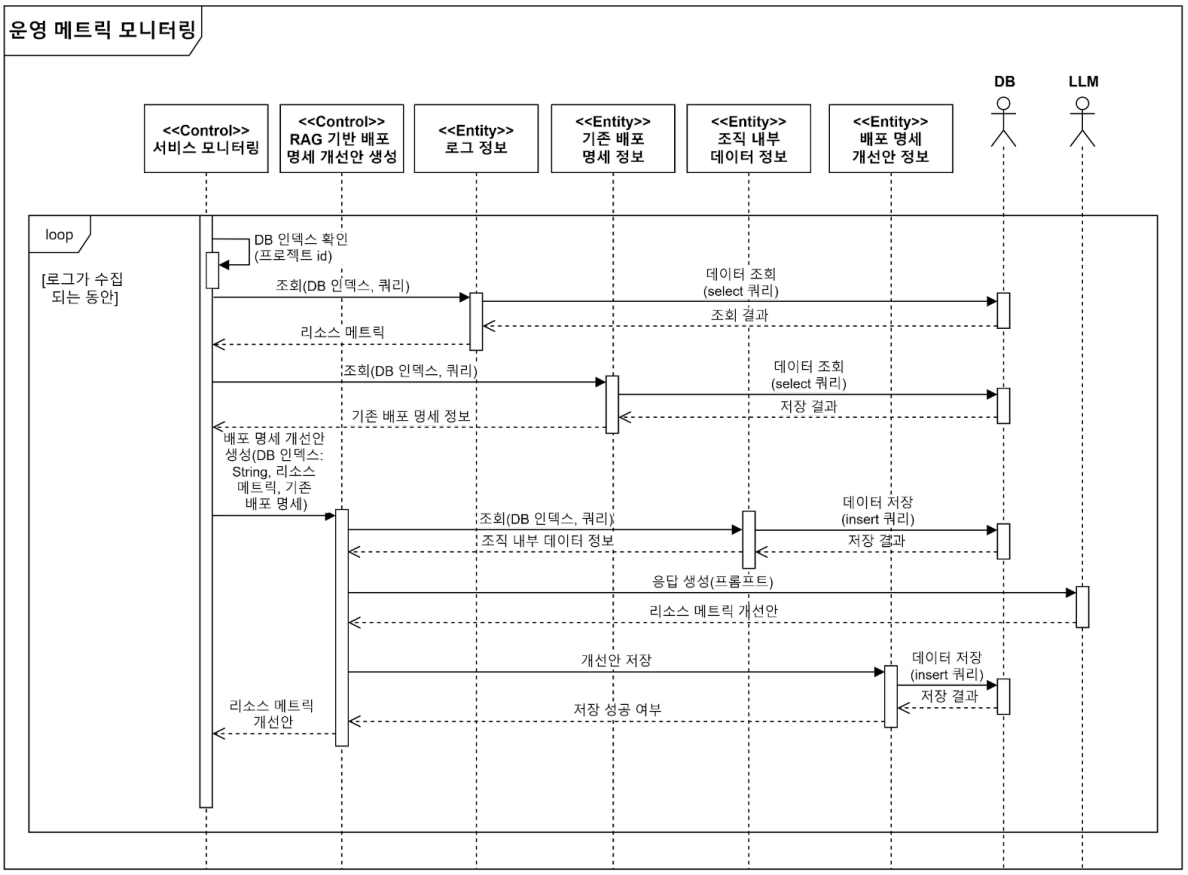


그림 35 운영 메트릭 모니터링 시퀀스 다이어그램

그림에서처럼, 시스템은 스케줄러에 의해 3분 간격으로 DB에서 CPU, 메모리 사용량과 같은 운영 메트릭 데이터를 조회한다. 이를 통해 시스템은 시스템에 등록된 프로젝트의 Pod의 CPU/메모리 수집된 데이터를 바탕으로 리소스 사용량과 조직 내부 데이터에 정의된 리소스 정책을 비교하여 권장 리소스 설정 값을 도출한다. 이 운영 메트릭 조회 결과는 모니터링 이력으로 DB에 저장된다. 시스템은 조직의 리소스 정책 데이터와 현재 메트릭 분석 결과를 종합하여 LLM에 전달하고, 조직 내부 데이터 내 리소스 할당 정책을 기반으로 리소스 개선안(requests, limits)이 포함된 배포 명세 개선안을 생성하도록 요청한다. 생성된 개선안은 이력으로 저장되어 사용자에게 제공된다.

xx) 배포 명세 개선안 조회

그림 36은 배포 명세 개선안 조회 시퀀스 다이어그램이다.

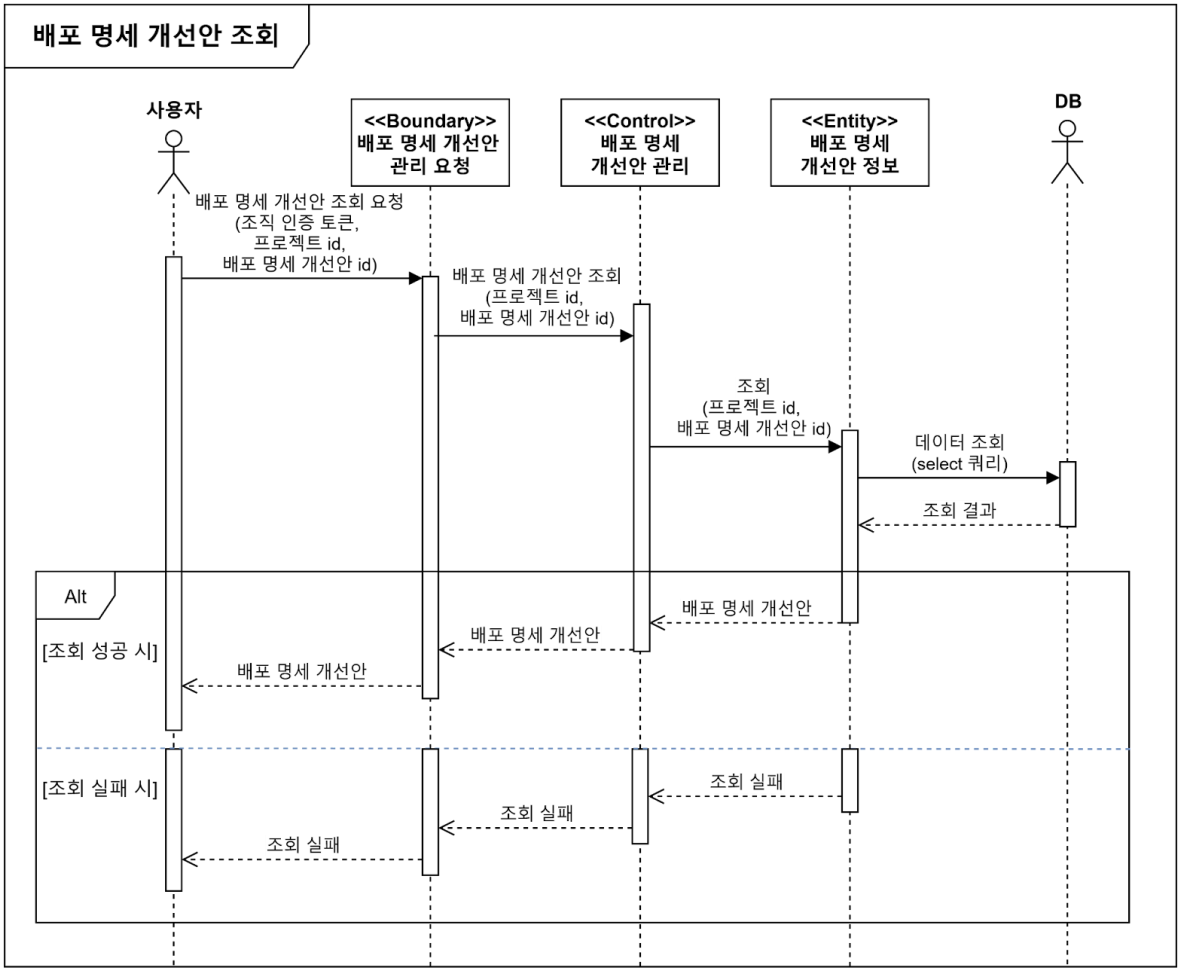


그림 36 배포 명세 개선안 조회 시퀀스 다이어그램

그림에서처럼, 사용자는 배포 명세 개선안을 조회할 프로젝트를 선택하고 조회를 요청한다. 시스템은 해당 프로젝트의 배포 명세 개선안 목록을 데이터베이스에서 조회한다. 시스템은 조회에 성공한 경우 사용자에게 조회 성공 결과를 보여주고 실패 시 조회 실패 결과를 출력한다.

xxi) 배포 명세 개선안 삭제

그림 37은 배포 명세 개선안 삭제 시퀀스 다이어그램이다.

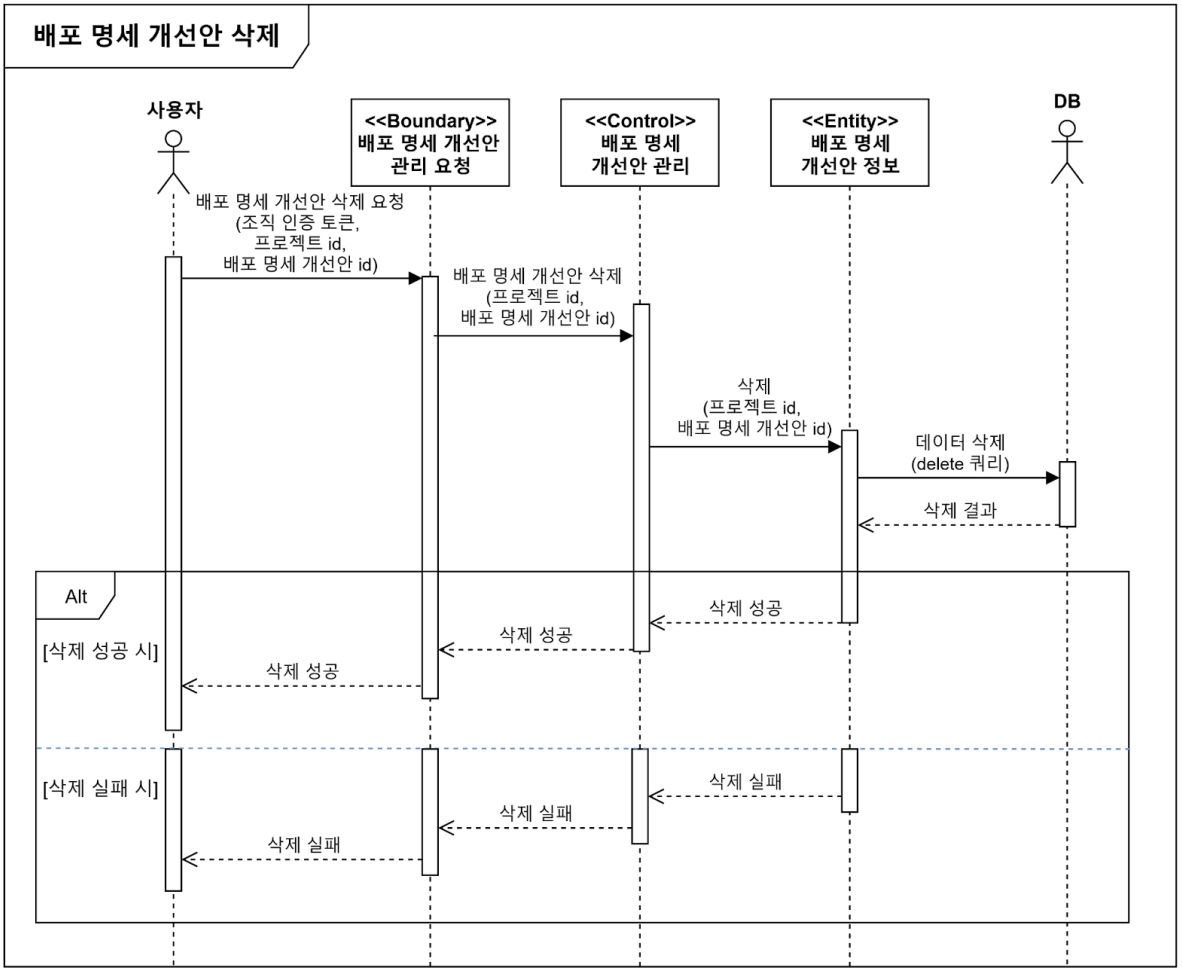


그림 37 배포 명세 개선안 삭제 시퀀스 다이어그램

그림 내용과 같이, 사용자는 삭제할 개선안 항목을 선택하고 삭제를 요청한다. 시스템은 해당 개선안을 데이터베이스에서 삭제한다. 시스템은 삭제에 성공한 경우 사용자에게 삭제 성공 결과를 보여주고 실패 시 삭제 실패 결과를 출력한다.

3.2. 시스템 구성

3.2.1. 시스템 구성도

그림 38은 전체 시스템이 어떻게 구성되어 있는지 나타낸다. 제안하는 시스템은 MSA 운영·관리 부담을 줄이는 것을 목표로, 조직 계정 관리, MSA 프로젝트 관리, 조직 내부 데이터 관리, RAG 기반 MSA 애플리케이션 배포 지원, 서비스 운영 모니터링 기능을 제공한다.

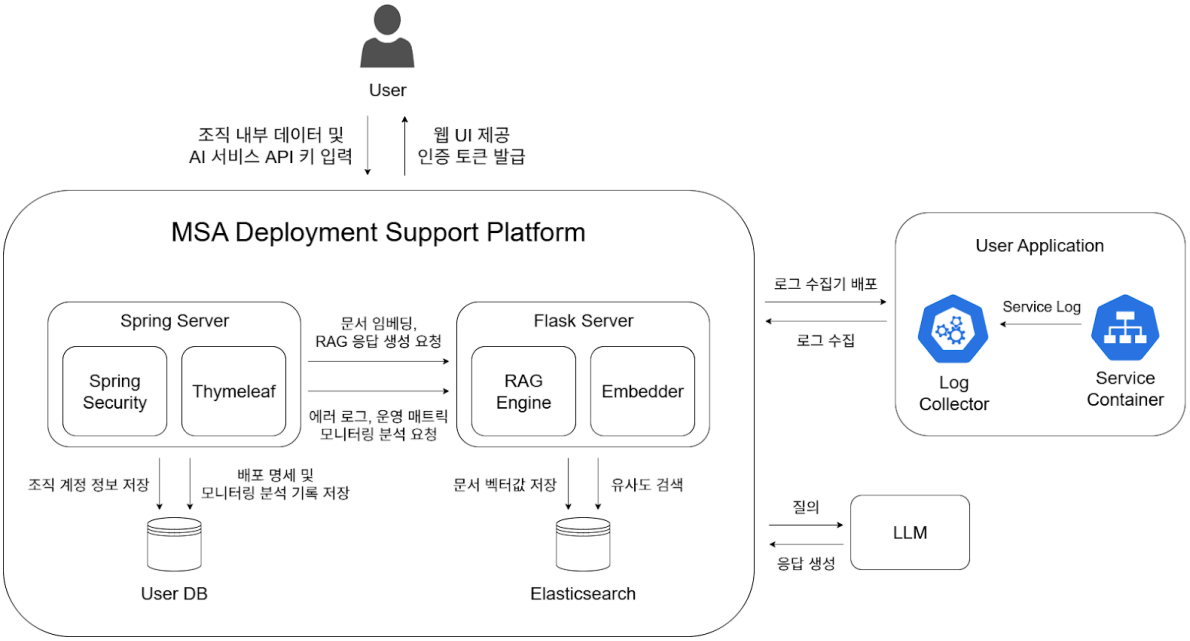


그림 38 시스템 구성도

그림에 나타낸 바와 같이, 조직 계정 관리는 사용자의 시스템 회원가입 시 LLM 사용을 위한 API 키를 검증하고 로그인 및 계정 삭제를 수행하는 기능이다. MSA 프로젝트 관리는 MSA 배포 환경 접속을 위한 IP 주소와 SSH 인증 정보를 관리하는 기능이다. 조직 내부 데이터 관리는 RAG의 검색(Retriever) 단계에 필요한 조직 내부 데이터를 임베딩하여 Elasticsearch에 저장 및 조직 내부 데이터 삭제를 수행한다. RAG 기반 MSA 애플리케이션 배포 지원은 Elasticsearch에 저장되어 있는 조직 내부 데이터를 유사도 검색하여 사용자 요청을 반영한 배포 명세 생성 및 수정을 수행한다. 서비스 운영 모니터링은 사용자에게 로그 수집기를 제공하여 사용자 애플리케이션에 로그 수집 환경을 구성할 수 있도록 하고, 애플리케이션에서 발생하는 로그와 메트릭을 수집하여 배포 명세를 개선하는 역할을 수행한다.

3.2.2. 개발 환경

표 3 시스템 개발 환경 구성도

Container Orchestration Tool	 
Backend	 
Frontend	
RAG	
Log Collector	

표 4 개발 환경

개발환경	사용기술
소프트웨어 형상관리(SCM)	Github
컨테이너 오케스트레이션 도구	Docker, Kubernetes
Back-end	Spring Boot, Flask
Front-end	HTML, CSS, JavaScript
RAG	LangChain
로그 수집	Elasticsearch, Logstash, Filebeat

3.3. RAG를 활용한 컨테이너 기반 마이크로서비스 운영 환경 관리 지원 시스템

3.3.1. 조직 계정 관리

제안하는 시스템의 RAG 기반 답변 생성 기능을 이용하기 위해서는 LLM API 키가 필요하다. 시스템은 사용자별로 API 키를 관리하기 위해 회원가입 기능을 제공한다. 사용자는 계정 생성 시 API 키를 입력하고 시스템은 LLM 제공자의 API 키 유효성 검증 URL에 HTTP 요청을 보내서 사용자의 API 키의 유효성을 검증한다.

그림 39는 사용자가 입력한 Gemini API키를 Google APIs가 제공하는 API 키 유효성 검증 URL에 HTTP 요청을 보내 유효성을 검증받은 화면을 보여준다.

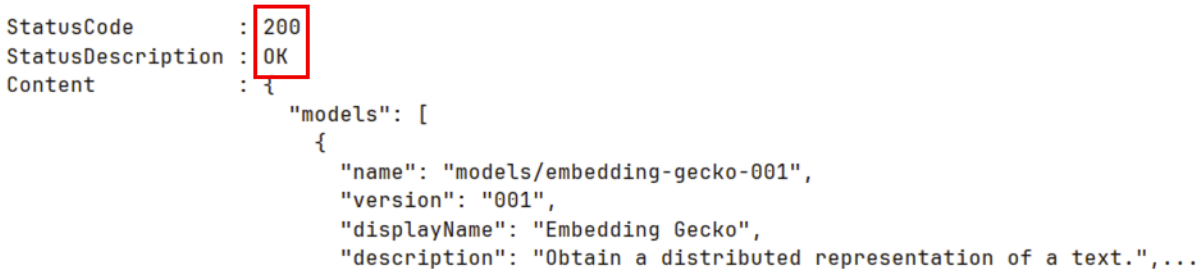


그림 39 API 키 검증 성공 응답 메시지

그림에 나타낸 바와 같이, 사용자는 회원가입 시 추가적으로 아이디와 비밀번호를 입력하여 시스템 회원가입을 진행한다. 회원가입 성공 시 생성된 계정으로 로그인하여 시스템의 기능을 이용할 수 있다.

3.3.2. MSA 프로젝트 관리

시스템은 사용자의 애플리케이션을 체계적으로 관리하기 위한 프로젝트 관리 기능을 제공한다. 각 프로젝트는 하나의 MSA 관리 단위에 대응하며, 해당 서비스의 배포 명세 작성에 필요한 조직 내부 데이터를 저장하고 관리한다. 또한 애플리케이션에서 수집되는 로그에 대해서는 Elasticsearch의 인덱스(index) 단위로 분리해서, 로그 데이터를 프로젝트 단위로 저장·조회할 수 있도록 한다. 이를 통해 사용자는 각 프로젝트별로 배포 명세, 실행 환경, 운영 로그를 일관성 있게 관리할 수 있으며, 서비스별 데이터의 구분과 추적 가능성이 확보된다.

추가적으로 프로젝트는 사용자 애플리케이션에 원격 접속 기능을 제공하여, 사용자가 웹 인터페이스를 통해 MSA 애플리케이션 배포 환경에 직접 접속하고 제어할 수 있도록 한다. 이를 위하여 시스템은 사용자가 프로젝트를 생성할 때 IP 주소, SSH 인증 정보가 담긴 pem 파일, 포트 번호, 서버 사용자 이름의 입력을 요구한다.

그림 40은 사용자가 웹 인터페이스를 통해 사용자 애플리케이션 환경에 접속하는 과정을 나타낸다.

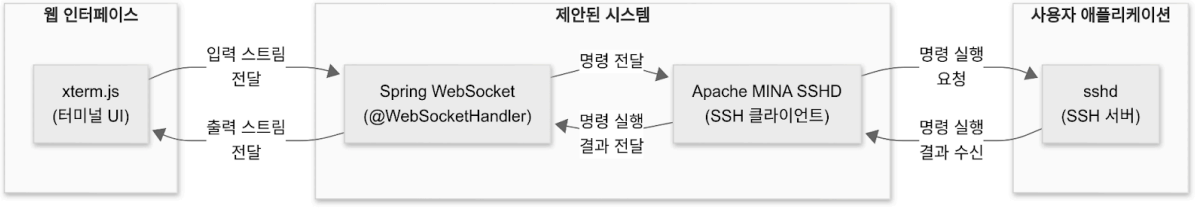


그림 40 웹 인터페이스를 통한 프로젝트 원격 접속

그림에서처럼, 시스템은 Spring 환경에서 SSH 통신 기능을 제공하는 라이브러리인 Apache MINA SSHD를 사용하여 MSA 애플리케이션의 서버 환경으로 SSH 연결을 수행한다. SSH 인증이 성공하면, 원격 접속 상태를 관리하기 위한 고유한 세션 ID가 생성된다. 웹 터미널 구축 라이브러리인 xterm.js를 통해 웹 브라우저 환경에서 SSH 세션을 터미널 형태로 제공한다. 이를 통해 사용자는 웹 화면에서 터미널을 통해 애플리케이션 상태 확인 및 제어 작업을 수행할 수 있다.

3.3.3. 조직 내부 데이터 관리

i) 조직 내부 데이터 분류

컨테이너 기반 MSA를 배포하기 위해 작성되어야 하는 배포 명세는 애플리케이션의 실행 환경을 어떻게 구성할지 구체적으로 지시하는 여러 필드로 구성된다. 조직의 목적에 부합하는 애플리케이션을 운용하기 위해서는 이러한 필드의 값을 정의하는 조직 내부 데이터가 필수적이다. 이러한 조직 내부 데이터는 MSA 정보 요소와 적용 목적을 명확하게 하기 위해 분류될 필요가 있다. 각 서비스의 API 명세, 설정, 의존 관계 등이 명확히 관리되지 않으면 서비스 간의 숨겨진 의존성으로 인해 작은 변경이 예기치 못한 장애로 번지거나, 개발 환경과 운영 환경의 불일치를 초래하여 시스템 전체의 복잡성을 높이기 때문이다[11].

그림 41은 이러한 조직 내부 데이터가 어떻게 분류되고, RAG 기술을 활용하여 최종적으로 쿠버네티스 배포 명세로 변환되는지 전체 과정을 시각적으로 보여준다.

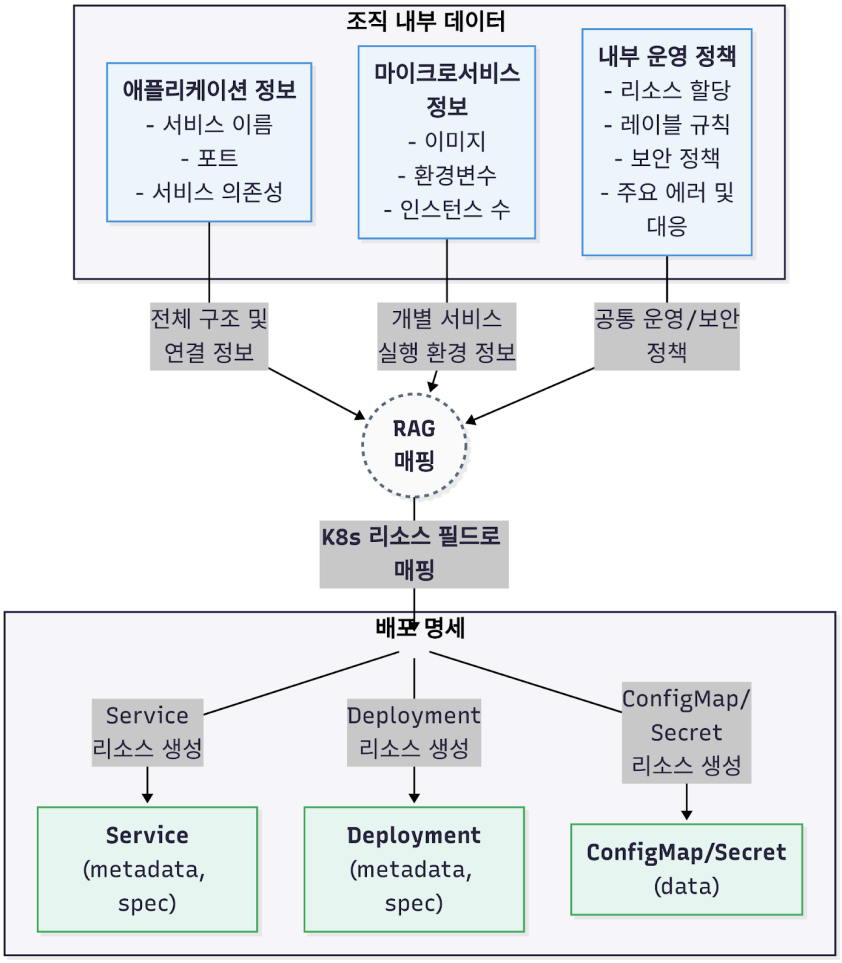


그림 41 조직 내부 데이터를 활용한 배포 명세 생성 과정

그림에서 볼 수 있듯이, 시스템은 세 가지 유형으로 분류된 조직 내부 데이터를 입력으로 사용한다. 애플리케이션 정보는 서비스 이름, 포트, 서비스 간 의존성과 같이 서비스의 전체 구조 및 연결 정보를 정의한다. 마이크로서비스 정보는 컨테이너 이미지, 환경 변수, 인스턴스 수 등 개별 서비스의 구체적인 실행 환경을 정의한다. 내부 운영 정책은 리소스 할당량, 레이블 규칙, 보안 정책, 주요 에러 및 대응 방안 등 조직 전반의 운영 원칙과 기준을 포함한다.

이렇게 분류된 정보들은 RAG 매핑을 통해 각 데이터의 목적에 맞게 쿠버네티스 리소스의 필드에 매핑된다. 이 과정을 통해 최종적으로 Service, Deployment, ConfigMap/Secret 등의 구체적인 리소스를 정의하게 된다. 이러한 접근 방식은 추상적인 조직의 요구사항과 정책을 실제 배포 가능한 구체적인 결과물로 변환하는 과정을 보여준다.

표 5는 분류된 내부 데이터의 항목들이 배포 명세의 어떤 필드에 반영되는지 나타낸다.

표 5 내부 데이터 항목과 배포 명세 필드 연관관계

구성 항목 / 정보 요소	데이터 항목	배포 명세 필드	데이터 항목 정의
애플리케이션 정보	서비스 이름	metadata.name	애플리케이션을 구분하기 위한 고유 이름
	내부 서비스 포트	spec.ports	서비스 간 통신에 사용되는 포트
	의존 서비스	data	애플리케이션을 구성하는 마이크로서비스 목록
	외부 노출 방식	spec.type	외부에서 서비스에 연결하기 위한 공개 방식
마이크로서비스 정보	컨테이너 이미지 및 버전	spec.template.spec .containers.image	배포에 사용되는 이미지 경로 및 버전
	마이크로서비스 포트	spec.template.spec .containers.ports	서비스의 내부 통신용 포트
	환경 변수	spec.template.spec .containers.env	서비스 동작에 필요한 설정 값 목록

	볼륨 구성	spec.template.spec.volumes, containers.volumeMounts	서비스의 데이터 저장 공간
	실행 인스턴스 수	spec.replicas	서비스의 동시 실행 개수
조직 운영 정책	레이블 규칙	metadata.labels, spec.selector	조직 내 레이블 작성 규칙
	리소스 할당 기준	spec.template.spec.containers.resources	서비스가 사용할 컴퓨터 자원에 대한 조직의 내부 규정
	보안 및 운영 정책	spec.template.spec.securityContext	모든 서비스의 공통 보안 설정 규칙
	주요 에러 해결 방안	-	배포 및 운영 중 발생 가능한 오류에 대한 해결 방안

표에 나타난 바와 같이, 이러한 분류는 배포 명세의 각 필드가 요구하는 데이터 범위와 직접적으로 대응되며, 시스템은 분류된 데이터를 자동으로 참조하여 배포 명세를 생성한다. 이를 통해 사용자는 분산된 내부 문서를 일일이 수집하지 않고도, 필요한 정보가 정확하게 반영된 일관성 있는 배포 명세를 작성할 수 있다.

ii) 조직 내부 데이터 임베딩

사용자는 생성한 프로젝트에 배포 명세 작성에 필요한 조직 내부 데이터를 업로드한다. 시스템은 데이터를 임베딩한 후 해당 프로젝트를 인덱스로 설정하여 Elasticsearch에 저장한다. 그림 42는 사용자가 project-1에 업로드한 문서가 벡터화되어 저장된 모습을 나타낸다.



그림 42 내부 문서가 임베딩되어 저장된 모습

그림과 같이 내부 문서가 임베딩 된 이후 시스템은 사용자 질의를 바탕으로 유사도 검색을 수행하여 관련된 문서를 자동으로 참조하여 배포 명세를 생성할 수 있다. 임베딩된 데이터는 file_name을 통해 삭제할 수 있다.

3.3.4. RAG 기반 MSA 애플리케이션 배포 지원

3.3.3절에서 정의한 내부 데이터를 활용하여 실제 배포 명세를 생성하고 수정하는 시스템의 핵심 기능이다. 이 기능을 사용자에게 제공하기 위해 본 시스템은 사용자의 자연어 질의를 통해 배포 명세의 생성을 요청하거나 기존 명세의 오류 수정을 요구할 수 있는 대화형 인터페이스를 제공한다. 사용자가 배포 명세 생성 혹은 수정을 요청하면 시스템은 RAG 파이프라인을 통해 조직 내부 데이터를 참조하여 배포 명세 파일을 생성한다.

RAG 파이프라인은 LangChain Expression Language(LCEL)을 기반으로 구축하였다. LCEL은 파이프(|) 연산자를 사용하여 RAG의 각 단계를 직관적으로 연결하는 LangChain의 선언적 체인 구성 방식이다. 이는 복잡한 파이프라인을 간결하게 정의할 수 있고, 스트리밍 및 비동기 실행을 지원하여 성능 최적화에 유리하다. 이 체인은 각 단계가 파이프로 연결되어 순차적으로 실행된다. 전체 과정은 컨텍스트 생성 및 프롬프트 구성과 답변 생성 및 최종 반환 두 단계로 구성된다.

그림 43은 본 시스템이 사용자 질의에 대한 답변 생성을 LLM에 요청하기 위한 최종 프롬프트를 구성하는 과정을 나타낸 흐름도이다. 사용자의 질의가 입력되면, Vector 저장소에서 관련 문서를 검색하여 컨텍스트를 생성한다. 최종적으로 해당 컨텍스트와 사용자의 질의를 포함하는 프롬프트가 생성된다.

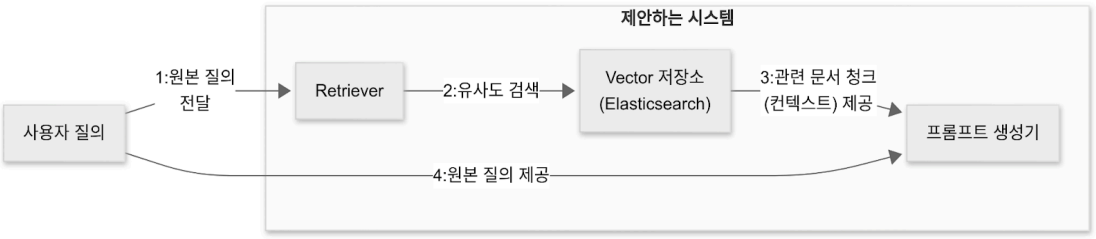


그림 43 컨텍스트 생성 및 프롬프트 구성 과정

그림 44는 시스템이 생성된 프롬프트를 통해 답변 생성하는 과정을 나타낸 흐름도이다. 이전 단계에서 완성된 프롬프트가 외부 LLM에 전달되면 LLM이 답변을 생성한다. 생성된 답변은 웹 UI에 렌더링 됨으로써 사용자에게 최종답변으로 제공되며, 사용자가 나중에 해당 이력을 확인할 수 있도록 시스템 DB에 저장한다.



그림 44 답변 생성 과정

하지만 연구 과정에서, 단순히 RAG로 조직 내부 데이터를 제공하는 것만으로는 실제 배포 환경에서 유효한 결과물을 얻기 어렵다는 문제점을 발견했다. LLM은 검색된 데이터를 기반으로 응답을 생성함에도 불구하고, YAML 문법 오류를 포함하거나 조직의 고유한 운영 정책(예: Secret 리소스는 Base64 인코딩 후 data필드만 사용)을 위반하는 등 유효하지 않은 결과물을 생성하는 경향이 있었다.

따라서 제안하는 시스템은 이러한 LLM의 한계를 극복하고 생성 결과의 신뢰성을 확보하기 위해, 4.1절의 정량평가에서 확인된 배포 실패 이력을 바탕으로 LLM에게 전달되는 최종 프롬프트를 점진적으로 개선하는 과정을 거쳐 생성 정확도를 제고하였다. 최종 프롬프트(P)는, 시스템의 배포 명세 생성 규칙과 구조를 정의하는 프롬프트 템플릿(T)에 요청마다 내용이 바뀌는 동적 변수인 사용자 질의(Q)만을 주입하거나 질의와 조직 내부 데이터 문맥(C)을 함께 주입하여 완성된다. 본 연구에서는 최종 프롬프트를 구성하는 데 사용되는 세 가지 요소인 사용자 질의(Q), 검색된 조직 내부 데이터 문맥(C), 프롬프트 템플릿(T)를 다음과 같이 정의하였다.

i) 사용자 질의 (Q)

1) Q1 (단순 질의)

기능 흐름이나 필요한 서비스 목록 없이,최종 목표만 단순하게 요청하는 수준이다. 실제 사용된 질의는 다음과 같다.

“카카오톡 선물하기 MSA 배포하기 위한 yaml 파일 작성해줘"

2) Q2 (상세 질의)

배포하려는 애플리케이션의 전체 기능 흐름과 이를 구현하는 데 필요한 모든 마이크로서비스 구성요소를 명시적으로 전달한다. 여기서 기능 흐름은 사용자의 행동 시나리오를 의미하며, 구성요소 명시는 해당 시나리오를 실행하는 데 필요한 모든 서비스의 목록을 지정한다. 실제로 사용된 질의는 다음과 같다.

> "선물하기 서비스를 구축하려고 하거든? 실제 배포가 가능해야 할 텐데 유저 등록, 관리도 가능해야 하고 상품 조회, 사용자 위시리스트 조회랑 주문하기, 주문하기 시 카카오 메시지로 메시지 보내는 것까지 기능적으로 가능해야 해. 이걸 구현하려면 user-service, product-service, wish-service, order-service, gift-service, api-gateway가 모두 필요해. 그러려면 배포 파일을 어떻게 구성해야 할까?"

ii) 조직 내부 데이터 문맥 (C)

그림 45는 조직 내부 데이터 문맥의 예시로, order-service 기본 정보를 나타낸다.

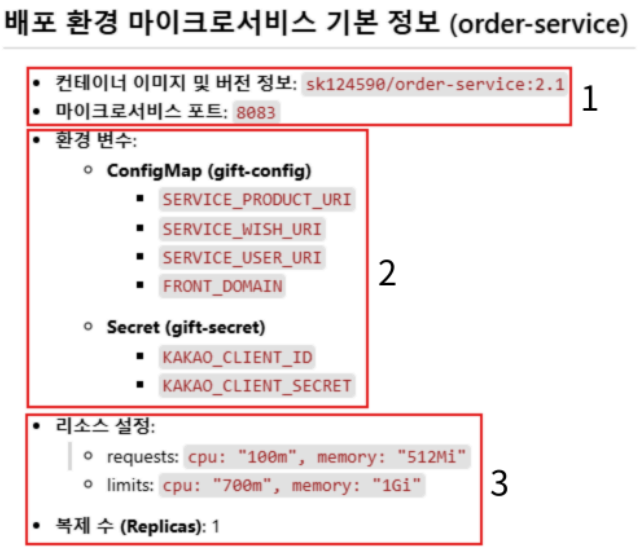


그림 45 조직 내부 데이터 문맥에 포함되는 order-service 기본 정보

배포명세가 생성될 때, 1을 참조하여 order-service에서 사용할 컨테이너 이미지와 포트를 지정하고, 2를 참조하여 의존성 주입에 필요한 환경변수를 설정하며, 3을 참조하여 리소스 할당량과 실행할 인스턴스(Replica) 수를 명시한다.

iii) 프롬프트 템플릿 (T)

1) T1 (기본)

그림 46은 T1 프롬프트 템플릿을 나타낸다. 이 프롬프트 템플릿은 LLM에게 쿠버네티스 전문가로서의 페르소나를 부여하고, 배포 명세를 생성하기 위한 기본적인 작성 규칙으로 구성된다.

```
yaml_generation_prompt = PromptTemplate(
    input_variables=["query"],
    template="""
        당신은 클라우드 애플리케이션 배포에 능숙한 Kubernetes 전문가입니다.
        아래의 문맥과 사용자 질의를 기반으로 Kubernetes 클러스터에 실제로 배포 가능한 YAML 명세를 생성하세요.
```

쿠버네티스 전문가(Persona) 역할 부여

```
        요구사항:
        {query}

        작성 규칙:
        - 전체 결과를 **YAML 형식**으로 출력하세요.
        - 각 필드의 역할을 **간단한 주석** 으로 설명하세요.
        - 기본적인 리소스 설정을 포함하세요:
        - `replicas`, `containerPort`, `resources.limits/requests` 등
        - 필요한 경우 다음과 같은 리소스도 함께 생성하세요:
        - `Service`, `Deployment`, `ConfigMap`, `Ingress`, `PersistentVolumeClaim`
        - 생성된 YAML은 실제 배포 가능한 형식이어야 합니다.
        - 하나의 YAML 안에 여러 리소스가 필요한 경우 `---` 로 구분해서 함께 정의하세요.
```

LLM이 생성할 결과물의 형식(YAML)과 반드시 포함해야 할 내용(주석, 리소스 종류 등)을 규칙으로 명시

```
    """
```

그림 46 T1의 프롬프트 템플릿

2) T2 (RAG 문맥 교정 절대 규칙)

그림 47은 T2 프롬프트 템플릿을 나타낸다.

```
당신은 클라우드 애플리케이션 배포에 능숙한 Kubernetes 전문가입니다.
당신의 주요 목표는 '문맥'에 설명된 모든 마이크로서비스에 대한 완전하고 배포 가능한 Kubernetes
YAML 명세 세트를 생성하는 것입니다.

'문맥'의 내용을 반드시 준수해서 사용자 질의에 맞는 Kubernetes YAML 명세와 그 출처를 JSON
형식으로 생성하세요.

### 절대 규칙 ###
1. '문맥'은 유일한 신뢰 가능한 정보 출처입니다. 사전 지식과 문맥이 충돌하는 경우, 반드시
문맥의 내용을 우선해야 합니다.
2. '문맥'에 명시된 리소스 종류, 포트, 이미지 버전, 레이블 규칙, 리소스 할당량 등 모든 세부
사항을 정확하게 반영해야 합니다.
3. '문맥'에 설명된 민감 정보를 바탕으로 단일 `apiVersion: v1, kind: Secret` 리소스를 먼저
생성해야 합니다.
4. 이 민감 정보가 필요한 다른 모든 리소스(예: Deployment)는 생성된 Secret을 `secretKeyRef`
또는 `envFrom`을 사용하여 참조해야 합니다. Deployment에 Secret 값을 직접 포함하지 마세요.
5. '문맥'에 언급된 모든 마이크로서비스(예: api-gateway, user-service, product-service 등)에
대해 Deployment와 Service를 반드시 생성해야 합니다. 하나도 빠뜨리면 안 됩니다.
```

배포 명세의 항목에 매핑될 값을 정확히 반영하기 위한 문맥 교정 절대 규칙

```
문맥:
{context}
```

최종 프롬프트에서 조직 내부 데이터 문맥이 들어가는 부분

```
사용자 질의:
{query}
```

그림 47 문맥과 절대 규칙이 추가된 T2 프롬프트 템플릿

그림에 나타낸 바와 같이, 이 프롬프트 템플릿은 답변 생성 과정에서 참조할 조직 내부 데이터가 들어가는 문맥 항목을 포함한다. 추가적으로 이 조직 내부 데이터 문맥을 참조하여 배포 명세의 항목에 매핑될 값을 정확히 반영하기 위해 문맥 교정 절대 규칙을 정의하였다.

3) T3 (명시적 규칙)

그림 48은 T3 프롬프트 템플릿을 나타낸다. 이 프롬프트 템플릿에는 YAML 구조 절대 규칙 (metadata.data 금지), 컨텍스트 정책 준수 규칙(Secret 리소스에 Base64로 인코딩된 문자열인 경우에만 data필드 사용)의 명시적 규칙이 추가되었다.

```
5. '문맥'에 언급된 모든 마이크로서비스에 대해 Deployment와 Service를 반드시 생성해야
합니다. 하나도 빠뜨리면 안 됩니다.

### YAML 구조 절대 규칙 ###
1. `ConfigMap`과 `Secret`의 `data` 필드는 절대로 `metadata` 필드의 하위에 위치해서는
안 됩니다. `apiVersion`, `kind`, `metadata`, `data`는 모두 동일한 레벨의 필드여야
합니다. 이 규칙은 쿠버네티스 문법의 기본이며 `metadata.data` 형태는 문법 오류입니다.
```

문법 오류를 억제하는 YAML 구조 절대 규칙을 명시

```
### 컨텍스트 정책 준수 규칙 ###
1. '문맥' 내의 '조직 내부 정책'에 명시된 규칙을 **반드시** 준수해야 합니다.
2. 예를 들어, 'Secret 관리' 정책에서 `data` 필드를 사용하고 Base64 인코딩을
요구했다면, `절대로 `stringData`를 사용하거나 평문 텍스트를 사용해서는 안 됩니다.`
```

조직 내부 정책 위반 오류를 억제하는 컨텍스트 정책 준수 규칙을 명시

```
문맥:
{context}
```

그림 48 YAML 구조 절대 규칙이 추가된 T3의 프롬프트 템플릿

그림과 같이, 프롬프트 템플릿에는 YAML 구조 절대 규칙 (metadata.data 금지), 컨텍스트 정책 준수 규칙(Secret 리소스에 Base64로 인코딩된 문자열인 경우에만 data필드 사용)의 명시적 규칙이 추가되었다.

다음은 배포 실패 이력 기반의 최종 프롬프트 개선 과정을 나타낸다.

- P1(베이스라인) = T1(Q1)

가장 기본적인 페르소나 및 작성규칙을 조합한 초기 프롬프트로, 필수 마이크로서비스가 누락되는 구조적 문제와 컨테이너 이미지 경로, 포트 번호 등 모든 설정 값이 일반적인 예시 값으로 채워지는 내용 오류를 식별하였다. 또한 리소스 생성 범위가 명확하지 않아 일부 리소스가 누락되면서 올바른 배포 명세 구조가 형성되지 않는 문제가 관찰되었다.

- P2(리소스 생성 범위 명확화를 통한 리소스의 구조적 누락 문제 해결) = T1(Q2)

P1에서 도출한 문제를 해결하기 위해, 생성 대상 리소스를 명확히 지정하는 상세 질의(Q2)를 도입했다. 이를 통해 구조적 누락 문제는 해결할 수 있었으나, 조직 내부 데이터를 참조하지 못해 리소스 항목에 정확한 값을 할당하지 못하는 문제가 관찰되었다.

- P3(조직 내부 데이터 문맥 적용) = T2(Q2 + C)

P2에서 리소스 생성 범위 문제 해결을 확인했으므로, 배포 명세 파일의 각 항목에 값을 정확하게 할당할 수 있는지를 검증하기 위해 조직 내부 데이터 문맥(C)을 적용했다. 추가적으로 RAG를 활용한 문맥을 정확하게 반영하기 위한 지침을 추가하여 프롬프트 템플릿을 구성하였다. 테스트 결과, 모든 리소스에 내부 데이터에서 지정한 값이 올바르게 주입되었지만, LLM이 조직의 암묵적인 정책(예: Secret 리소스 에서 Base64 인코딩 값은 data필드에 정의해야 함)을 위반하거나 YAML 문법 오류(metadata.data 사용)를 발생시키는 문제가 관찰되었다.

- P4(배포 명세 이력 데이터에 기반한 명시적 규칙 추가) = T2(Q2 + C)

이전 단계(P3)에서 생성된 배포 명세 이력 데이터에서 발견된 문제(문법 오류, 정책 위반 문제)를 해결하기 위해 설계되었다. YAML 구조 절대 규칙(metadata.data 금지)과 Secret 리소스 사용 규칙(Base64 인코딩 값은 data 필드만 사용)을 포함한 명시적 규칙이 추가된 템플릿(T2)을 적용함으로써, 문법 오류와 정책 위반을 억제할 수 있었다.

이처럼 본 시스템은 점진적 고도화를 통해 설계된 사용자 질의, 조직 내부 데이터, 템플릿의 조합을 통해 최종 프롬프트를 구성함으로써 LLM이 생성하는 답변의 예측 불가능성을 제어하고 신뢰도 높은 배포 명세 생성을 지원한다. 이러한 설계 요소들이 배포 성공률에 미치는 영향에 대한 정량적 평가는 4.1.절에서 상세히 서술한다.

3.3.5. 서비스 운영 모니터링

시스템은 MSA 환경에서 발생하는 에러 로그와 메트릭 데이터를 수집하고 분석하여 운영 상태를 모니터링하고, RAG 모델을 통해 배포 명세 개선안을 도출하는 동적 개선 기능을 제공한다. 이 기능은 로그 수집, 로그 분석 요청, 개선안 생성 및 저장 세 단계로 구성된다.

i) 로그 수집

그림 49는 사용자 MSA 애플리케이션의 로그 수집 과정을 나타낸다.

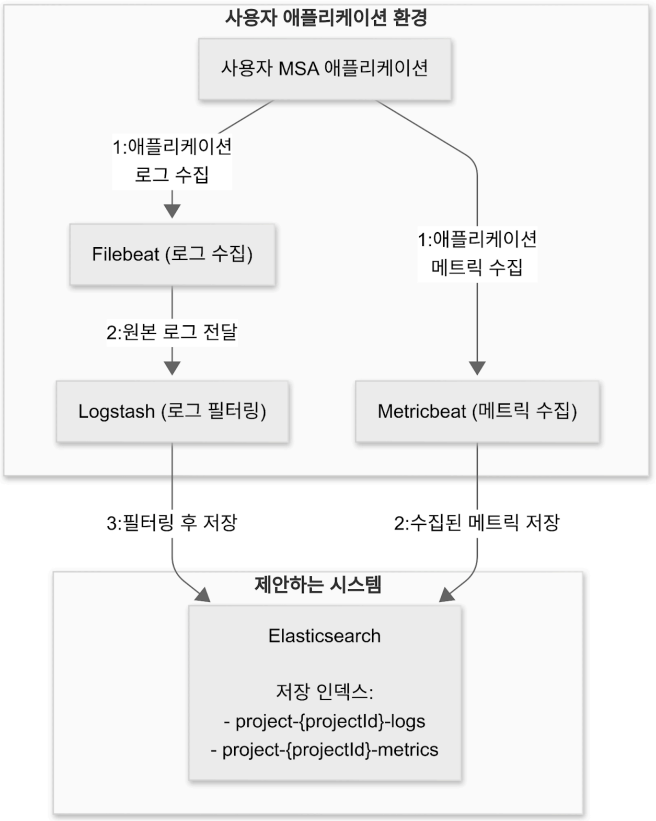


그림 49 로그 수집 과정

그림과 같이, 사용자 애플리케이션 환경에 배포된 로그 수집기(Filebeat, Metricbeat)를 통해 로그 및 메트릭 데이터를 수집한다. Filebeat는 컨테이너에서 발생하는 로그를 수집해서 Logstash로 전송한다. Logstash는 필터링(원본 로그를 로그 레벨 등 의미 단위로 분해) 규칙에 따라 로그를 정제하고 시스템의 Elasticsearch로 전송한다. Metricbeat는 CPU/메모리 등 리소스 메트릭을 수집하여 시스템의 Elasticsearch로 전송한다. 시스템은 Elasticsearch 내부 인덱스를 프로젝트 ID 별로 분리하여 각 프로젝트의 로그 데이터와 메트릭을 독립적으로 관리한다.

ii) 로그 분석 요청

그림 50은 로그 분석 요청 과정을 나타낸다.

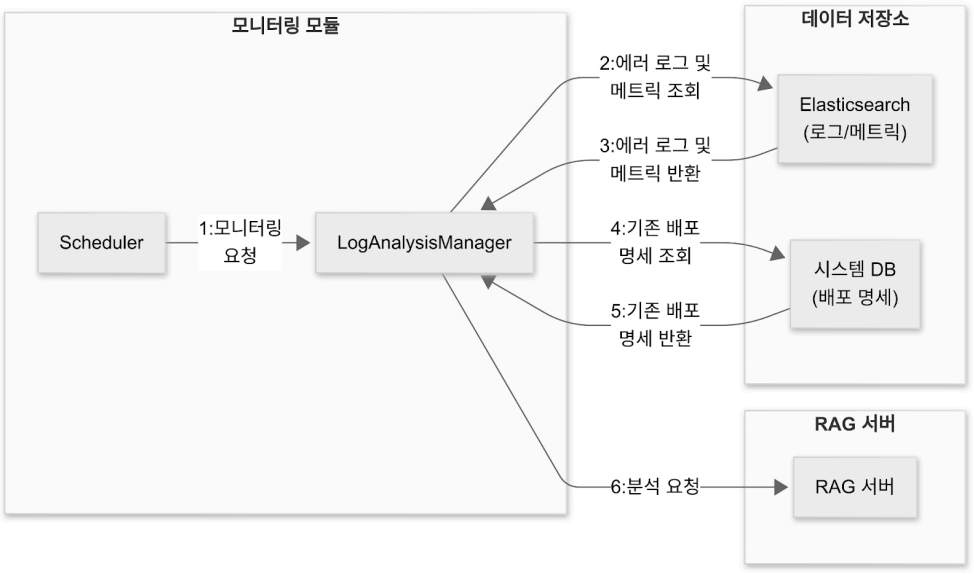


그림 50 로그 분석 요청

그림에서처럼, 로그 분석 모듈 LogAnalysisManager는 Spring의 스케줄러를 통해 일정 주기로 분석 작업을 수행한다. 이 모듈은 Elasticsearch에서 에러 로그와 리소스 메트릭을 조회하고 시스템 DB에서 기존 배포 명세를 조회한다. 이후 조회된 데이터를 조합하여 RAG 서버에 에러 로그 분석 요청과 리소스 분석 요청을 수행한다.

iii) 개선안 생성 및 저장

그림 51은 개선안 생성 및 저장 과정을 나타낸다.

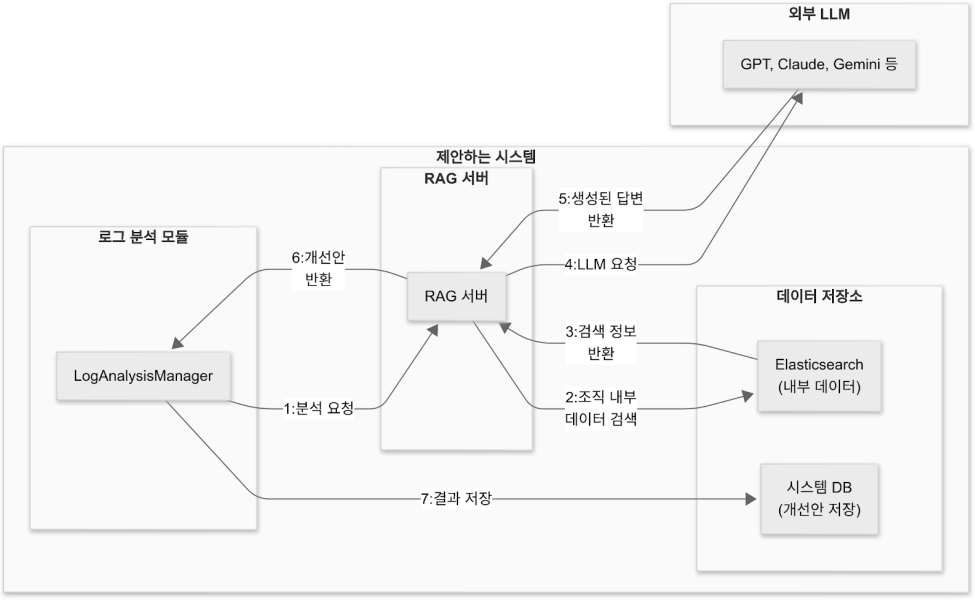


그림 51 개선안 생성 및 저장 과정

그림에 나타낸 바와 같이, RAG 서버는 LogAnalysis Manager로부터 받은 분석 요청 데이터를 기반으로 Elasticsearch에 저장된 조직 내부 데이터를 검색하여 필요 정보(컨텍스트)를 구성하고, 이를 외부 LLM에 전달하여 배포 명세 개선안을 생성한다. 생성된 개선안은 시스템 데이터베이스에 저장되어 사용자가 언제든지 조회 가능하다.

4. 연구 결과 분석 및 평가

4.1. 사례 연구 내용

본 사례 연구에서는 MSA로 구축된 "선물하기 서비스" 애플리케이션의 배포 및 운영 과정을 통해 제안 시스템의 효용성을 검증한다. 해당 애플리케이션을 사례로 선정한 이유는 다음과 같다.

- 서비스 간 의존성을 반영한 배포 명세 생성 정확도 평가 가능

선물하기 서비스는 인증(user-service), 상품(product-service), 주문(order-service)을 포함한 6개의 마이크로서비스로 구성되어 있으며, 각 서비스는 다른 서비스 간 의존 관계를 가진다. 예를 들어, 사용자가 상품을 주문하는 경우, 요청은 gift-app에서 시작해 api-gateway를 거쳐 order-service로 전달된다. 이때 order-service는 다시 user-service에서 사용자 정보를 인증하고 product-service에서 상품 재고를 확인하는 등 유기적으로 상호작용하는 구조를 가진다. 이러한 구조는 복잡한 서비스 관계를 정확히 해석하여 배포 명세에 자동으로 반영하는 본 시스템의 핵심 기능을 효과적으로 보여줄 수 있다.

- 운영 정책을 반영한 배포 명세 생성 정확도 평가 가능

안정적인 서비스 제공을 위해 조직의 보안 및 운영 정책을 배포 명세에 일관되게 적용하는 것이 매우 중요하다. 예를 들어, 보안 정책에 따라 소셜 로그인 클라이언트 시크릿(KAKAO_CLIENT_SECRET) 값이나 서비스 간 통신용 비밀 키(JWT_SECRET)와 같은 민감 정보는 반드시 Base64로 인코딩하여 Secret 리소스의 data 필드로 관리해야 하며, 운영 정책 측면에서는 고가용성을 위해 모든 마이크로서비스가 최소 2개 이상의 복제본(replicas)을 유지하고 안정적인 성능을 위해 CPU 및 메모리의 requests와 limits 값을 명시해야 한다. 이처럼 복합적인 조직 정책들을 시스템이 올바르게 반영하는지 보여줌으로써, 정책 기반의 명세 생성이 유효하게 이루어지는지 입증할 수 있다.

본 사례 연구는 조직 내부 데이터를 활용해 해당 "선물하기 서비스" 애플리케이션의 Kubernetes 배포 명세를 생성하고 수집된 로그 및 리소스 메트릭을 기반으로 개선안을 제안하는 과정을 보임으로써, 제안하는 시스템이 가진 기능의 실제 환경에서의 정확성과 실효성을 검증하는 것을 목적으로 한다.

4.1.1. 선물하기 서비스의 마이크로서비스 구성

선물하기 서비스는 사용자가 상품을 주문하고 주문 메시지를 보낼 수 있는 MSA 애플리케이션이다. 이 애플리케이션은 다음과 같은 6개의 마이크로서비스로 구성된다.

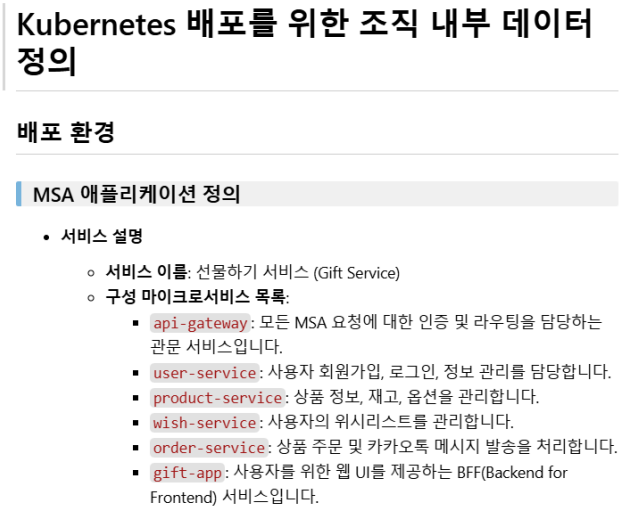
- gift-app: 사용자를 위한 웹 UI를 제공하는 서비스
- api-gateway: 모든 MSA 요청에 대한 인증 및 라우팅을 담당하는 서비스
- user-service: 사용자 회원가입, 로그인, 정보 관리를 담당하는 서비스
- product-service: 상품 정보, 재고, 옵션을 관리하는 서비스
- wish-service: 사용자의 위시리스트를 관리하는 서비스
- order-service: 상품 주문 및 주문 완료 메시지 발송을 처리하는 서비스

4.1.2. 선물하기 서비스 운영 조직 내부 데이터 구성

안정적인 MSA 배포 및 운영을 위해, 본 시스템은 조직 내부 데이터를 활용한다. 이때 데이터는 Kubernetes 배포 명세의 핵심 구성요소(Metadata, Spec 등)와 직접적으로 매핑될 수 있도록, 다음과 같은 세 가지 기준으로 구조화하여 정의되었다.

i) MSA 애플리케이션 정의

그림 52는 조직 내부 데이터 중 MSA 애플리케이션 정의 문서 중, 각 서비스가 어떤 기능을 수행하는지에 대한 설명을 나타낸다.



Kubernetes 배포를 위한 조직 내부 데이터 정의

배포 환경

MSA 애플리케이션 정의

- **서비스 설명**
  - **서비스 이름**: 선물하기 서비스 (Gift Service)
  - **구성 마이크로서비스 목록**:
    - `api-gateway`: 모든 MSA 요청에 대한 인증 및 라우팅을 담당하는 관문 서비스입니다.
    - `user-service`: 사용자 회원가입, 로그인, 정보 관리를 담당합니다.
    - `product-service`: 상품 정보, 재고, 옵션을 관리합니다.
    - `wish-service`: 사용자의 위시리스트를 관리합니다.
    - `order-service`: 상품 주문 및 카카오톡 메시지 발송을 처리합니다.
    - `gift-app`: 사용자를 위한 웹 UI를 제공하는 BFF(Backend for Frontend) 서비스입니다.

그림 52 MSA 애플리케이션 정의 문서 (서비스 설명)

그림과 같이, MSA 애플리케이션 정의 문서는 3.3.3에서 정의한 서비스 이름, 의존 서비스, 내부 서비스 포트, 외부 노출 방식 정보 등 애플리케이션의 전체 아키텍처 정보를 포함한다. 예를 들어, "선물하기 서비스"에서는 6개의 마이크로서비스 중 gift-app은 사용자 요청을 받는 BFF(Backend For Frontend)역할을 수행하기 위해 LoadBalancer 타입을 통해 외부에 노출되고, order-service가 user-service와 product-service의 내부 주소를 필요로 한다는 의존 관계를 명시한다. 이 정보는 Kubernetes Service 리소스와 각 Pod에 주입될 환경 변수를 생성하는 근거가 된다.

ii) 마이크로서비스 기본 정보

그림 53은 마이크로서비스 기본 정보 문서 중, user-service에 대한 기본 정보를 나타낸다.

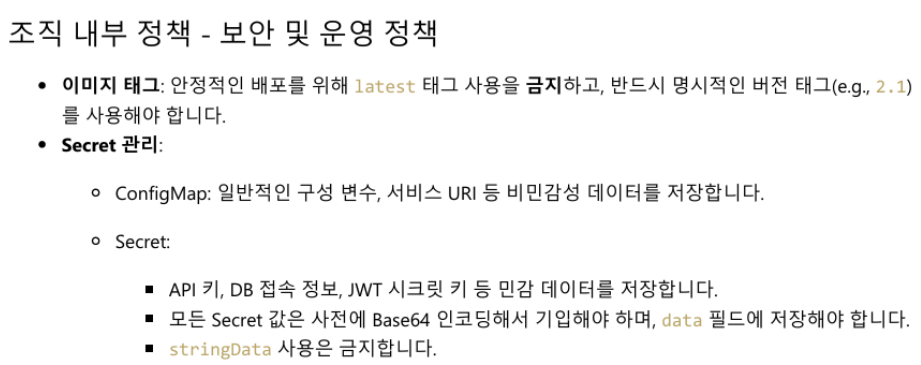


그림 53 마이크로서비스 기본 정보 문서 (user-service)

그림 내용과 같이, 마이크로서비스 기본 정보 문서에는 3.3.3에서 정의한 컨테이너 이미지 및 버전, 마이크로서비스 포트, 환경 변수, 볼륨 구성, 실행 인스턴스 수 정보 등 각 서비스 실행에 필요한 항목을 구체화한다. 이는 Kubernetes Deployment 리소스의 spec을 구성하는 가장 기본 데이터가 된다. 예를 들어 user-service의 경우, sk124590/user-service:2.1 이미지를 사용하고 8084 포트를 개방하며, JWT_SECRET과 같은 필수 환경변수를 주입받아 단일 인스턴스(replicas: 1)로 실행되도록 정의한다.

iii) 조직 내부 정책

그림 54는 조직 내부 정책 문서 중, 보안 및 운영 정책에 대한 내용을 나타낸다.


조직 내부 정책 - 보안 및 운영 정책

- **이미지 태그**: 안정적인 배포를 위해 `latest` 태그 사용을 **금지**하고, 반드시 명시적인 버전 태그(e.g., `2.1`)를 사용해야 합니다.
- **Secret 관리**:
  - ConfigMap: 일반적인 구성 변수, 서비스 URI 등 비민감성 데이터를 저장합니다.
  - Secret:
    - API 키, DB 접속 정보, JWT 시크릿 키 등 민감 데이터를 저장합니다.
    - 모든 Secret 값은 사전에 Base64 인코딩해서 기입해야 하며, `data` 필드에 저장해야 합니다.
    - `stringData` 사용은 금지합니다.

그림 54 조직 내부 정책 문서 (보안 및 운영 정책)

그림에서처럼, 조직 내부 정책 문서에는 3.3.3에서 정의한 레이블 규칙, 리소스 할당 기준, 보안 및 운영 정책, 주요 에러 해결 방안 등 모든 서비스가 일관되게 준수해야 할 조직의 표준과 규칙을 포함한다. "선물하기 서비스"의 경우, 보안 정책에 따라 KAKAO_CLIENT_SECRET 같은 민감 정보는 Base64로 인코딩하여 Secret 리소스의 data필드를 사용하도록 강제하고, 안정적인 배포를 위해 이미지 latest 태그 사용을 금지하고, 명시적인 이미지 버전 태그를 사용하도록 한다. 또한 일관된 리소스 관리를 위해 레이블 규칙에 따라 모든 서비스에 app: gifting-service 레이블을 부착해야 하며, 서비스 안정성을 위해 리소스 할당 기준에 따라 모든 컨테이너에 CPU와 메모리의 request와 limits을 설정해야 한다는 규칙이 포함된다.

4.1.3. 제안된 시스템을 활용한 배포 및 운영 시나리오

i) 배포 명세 생성

그림 55는 벡터화되어 Elasticsearch에 저장된 조직 내부 데이터 예시이다.

k _index	project-1
# _score	1
t metadata.content_type	text/plain
t metadata.file_name	배포 환경 마이크로서비스 기본 정보 (gift-app).md
t text	## 배포 환경 마이크로서비스 기본 정보 (gift-app) - **컨테이너 이미지 및 버전 정보**: `sk124590/gift-app:2.1` - **마이크로서비스 포트**: `8085` - **환경 변수**: - **ConfigMap (gift-config)** - `API_GATEWAY_URI` - `KAKAO_REDIRECT_URI` - **Secret (gift-secret)** - `JWT_SECRET` - `KAKAO_CLIENT_ID` - `KAKAO_CLIENT_SECRET` - **리소스 설정**: - requests: `cpu: "100m", memory: "512Mi"` - limits: `cpu: "500m", memory: "1Gi"` - **복제 수 (Replicas)**: 1
vector	[-0.022087497636675835, -0.01677463203668594 4, 0.01712522655725479, -0.02001089416444301 6, -0.02268081158399582, 0.01386199425905943, -0.03435833007097244, 0.015143015421926975, 0.002824988216161728, -0.013929416425526142,

그림 55 벡터화되어 Elasticsearch에 저장된 조직 내부 데이터

그림과 같이, 먼저 배포 담당자는 시스템에 조직 내부 데이터를 업로드 하여 배포 명세 생성 시 참조할 지식베이스를 구축한다. 이 데이터는 텍스트 임베딩 모델을 통해 벡터로 변환된 후, Elasticsearch 벡터 저장소에 저장된다. 이 과정을 통해 자연어 질의에 대한 유사도 검색을 수행할 수 있는 기반을 갖추게 된다.

그림 56은 조직 내부 데이터 관리 페이지의 데이터 업로드 및 삭제 인터페이스이다.

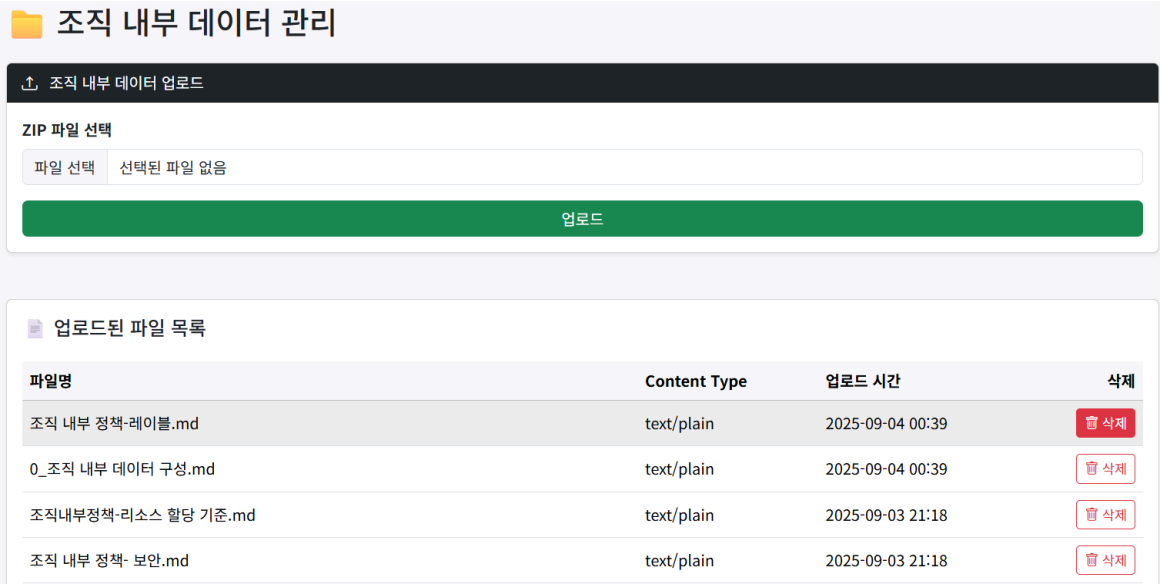


그림 56 데이터를 삭제하고 최신화 할 수 있는 조직 내부 데이터 관리 페이지

그림에 나타낸 바와 같이, 제안하는 시스템은 조직 내부 데이터 관리 페이지를 통해, 조직 정책이 변경될 경우 기존 데이터를 삭제하고 새로운 문서를 업로드하는 것만으로 데이터를 즉각적으로 최신화할 수 있다. 이는 재학습에 많은 시간과 비용이 소요되는 파인튜닝이나, 토큰 제한으로 인해 많은 문맥을 반영하기 어려운 프롬프트 엔지니어링 방식과 비교되는 RAG의 장점이다.

그림 57은 Langsmith 시각화 도구의 추적 결과를 나타낸다.

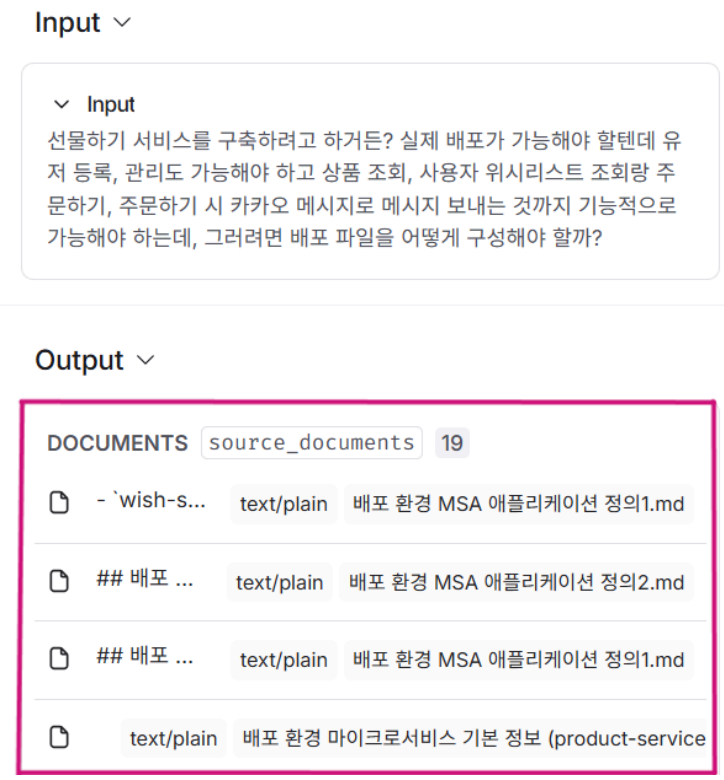


그림 57 Langsmith를 통한 RAG의 답변 생성 출처 추적

그림에서처럼, 지식베이스 구축 후, 배포 담당자는 자연어로 배포 명세 생성을 요청하고 시스템은 사용자 질의를 바탕으로 Elasticsearch 벡터 저장소에서 가장 관련성 높은 내부 데이터 청크들을 실시간으로 검색한다. RAG 기술은 MSA 애플리케이션 정의, 마이크로서비스 기본 정보 등의 문서를 핵심 근거로 활용했음을 알 수 있다. 이는 제안하는 시스템이 어떤 정보를 바탕으로 결과물을 생성했는지 투명하게 추적할 수 있어, 답변의 신뢰도를 높이는 중요한 장점이다.

그림 58은 시스템을 통해 생성된 조직의 정책과 요구사항을 준수하는 배포 명세 파일을 나타낸다.

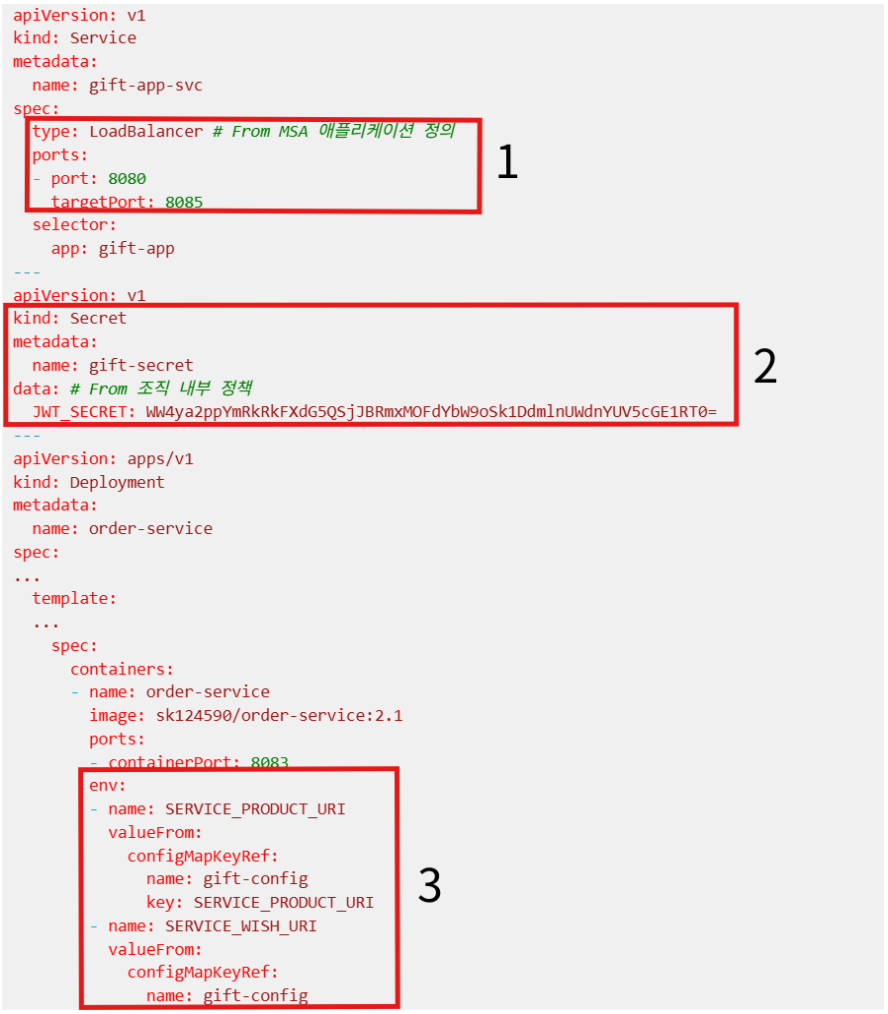


그림 58 시스템을 통해 생성된 조직의 정책과 요구사항을 준수하는 배포 명세 파일 일부

그림과 같이, 결과적으로 조직의 정책과 요구사항을 준수하는 배포 명세 파일이 생성되었다. 1에서 gift-app 서비스는 LoadBalancer로 올바르게 노출되었음을 확인할 수 있고, 2에서 모든 Secret 값은 data 필드를 사용하여 정책을 준수하였음을 확인할 수 있고, 3에서 각 서비스는 의존 관계에 따라 필요한 서비스 URI 환경변수를 정확히 주입받았음을 확인할 수 있다.

그림 59, 60은 원격 접속 세션을 통한 배포 화면과 배포 후 정상 작동하는 선물하기 서비스 화면이다.

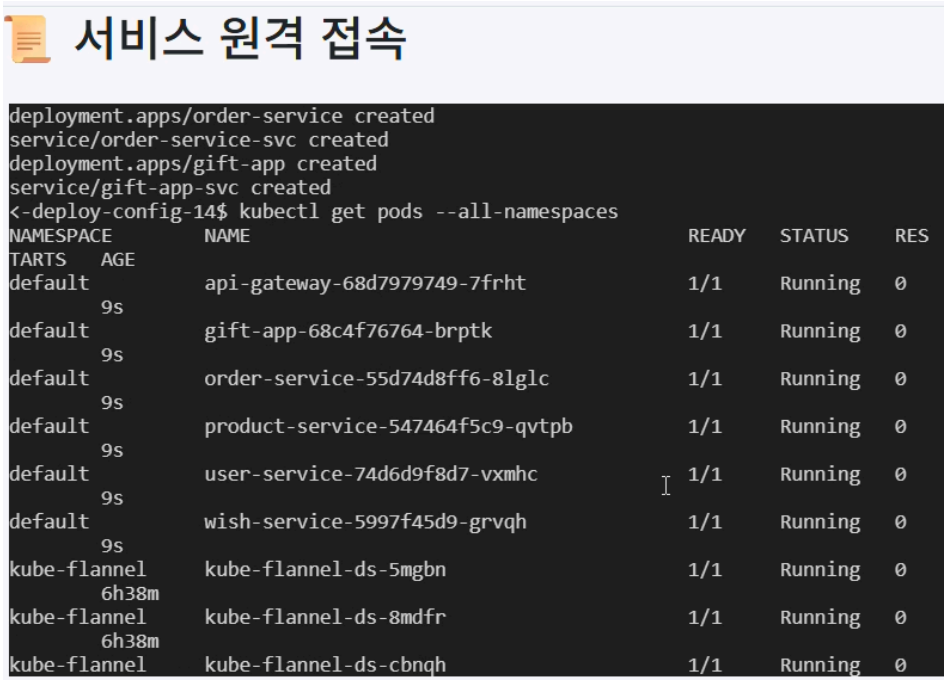


그림 59 원격 접속 세션을 통한 배포 화면



그림 60 배포 후 정상 작동하는 선물하기 서비스 MSA 애플리케이션 화면

그림에서처럼, 최종적으로 생성된 배포 명세를 그림 59와 같이 원격 접속 세션을 통해 배포한 결과, 그림 60과 같이 선물하기 서비스 MSA 애플리케이션이 정상 작동함을 확인할 수 있다.

ii) 서비스 운영 모니터링

초기 배포가 완료된 후, 시스템은 동적 모니터링 및 개선 기능을 수행한다. 로그 수집기를 통해 수집된 데이터에 대해 RAG를 통해 조직 내부 데이터를 근거로 구체적인 배포 명세 개선안을 사용자에게 제안한다.

그림 61은 로그 수집기 배포 후 선물하기 서비스의 모니터링 이력 조회 화면이다.

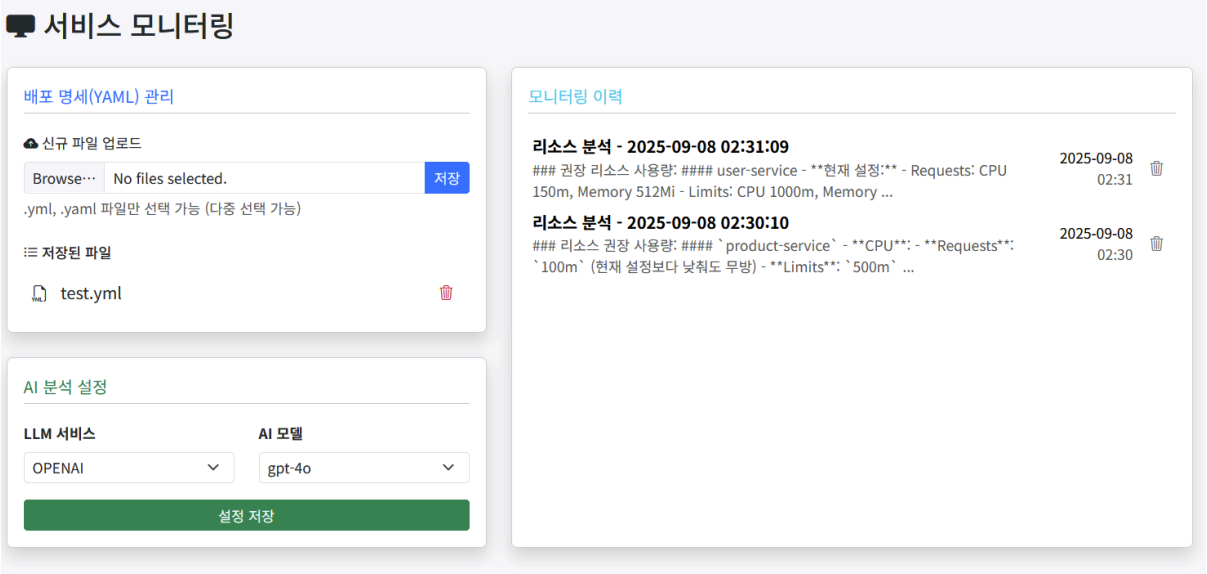


그림 61 로그 수집기 배포 후 선물하기 서비스 MSA 애플리케이션 모니터링 화면

그림에서처럼, 사용자는 모니터링 이력 목록에서 시스템이 주기적으로 분석한 결과를 확인할 수 있다. 모니터링 이력 항목 클릭 시 상세 내용도 확인할 수 있다.

그림 62는 모니터링 상세 이력 화면의 일부분이다.

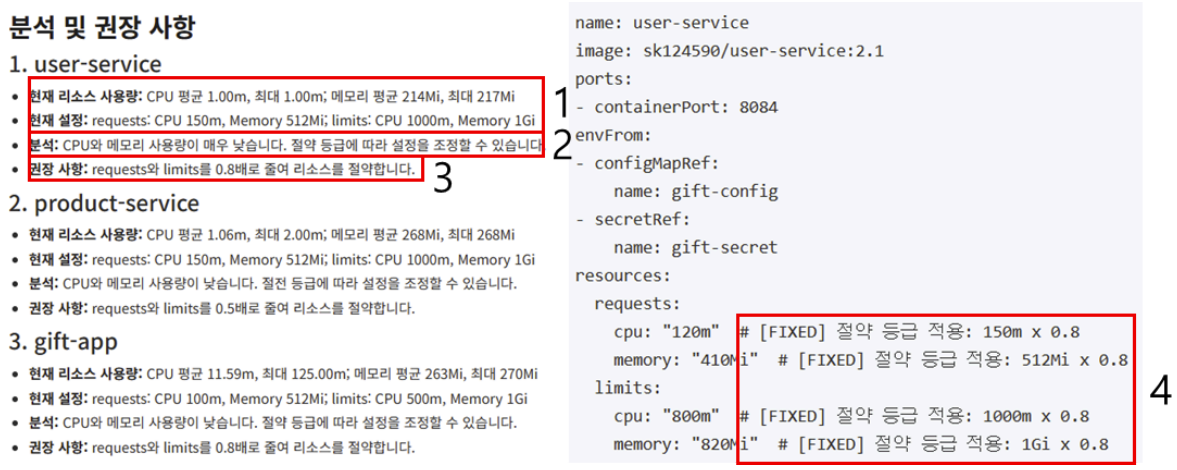


그림 62 모니터링 상세 이력 화면 일부

그림과 같이, 사용자는 시스템이 로그 수집 이후 RAG 기반으로 분석한 내용을 모니터링 화면에서 배포 명세 개선안 형태로 조회할 수 있다.

그림 62의 개선안은 다음과 같은 과정으로 도출되었다. 1에서 확인할 수 있듯이, 시스템은 사용자 애플리케이션의 메트릭 정보 및 기존 반영된 배포 명세를 조회해서 현재 리소스 사용량과 현재 리소스 설정 값을 확인한다. 2에서 시스템은 user-service의 현재 리소스 사용량이 설정 값(requests: CPU 150m, Memory 512 Mi)에 비해 낮다고 판단한다. 판단 결과를 바탕으로, 3을 통해 시스템은 조직 내부 정책에 정의된 '절약 등급'을 적용하여 리소스 사용량을 줄일 것을 제안한다. 4에서 확인할 수 있듯이, 최종적으로 requests와 limits 값을 기존의 0.8배로 조정하는 구체적인 배포 명세 수정안을 제안한다. 주석을 통해 어떤 변경 사항이 적용되었는지 명시해서 사용자가 변경 내용을 쉽게 이해할 수 있도록 구성한다.

이러한 개선안이 어떤 근거로 생성되었는지는 RAG가 참조한 문서를 통해 확인할 수 있다. 그림 63은 바로 위 그림 62의 개선안을 생성하기 위해, RAG가 실제로 참조한 조직 내부 데이터 문서 목록을 LangSmith 대시보드를 통해 확인한 결과를 보여준다.



그림 63 개선안 도출에 사용된 조직 내부 데이터 및 유사도 순위

그림에서처럼, 1에서 확인할 수 있듯이, 가장 높은 유사도를 보인 문서가 조직 내부 정책 중 리소스 할당 기준 문서임을 확인할 수 있다. 해당 문서의 절약 등급 정책 내용을 살펴보면, 2에 리소스가 필요 이상 낭비될 때 현재 requests, limits 값을 조정하라는 규칙이 명시되어 있다. 시스템은 실제 사용량 데이터에 대해 이 기준을 근거로 그림 62에 제시된 개선안을 도출했음을 확인할 수 있다.

4.2. 정량평가

본 평가는 3.3.4절에서 정의한 최종 프롬프트(P1~P4)의 실질적인 성능을 입증하는 것을 목표로 한다. 평가 척도로서 Google DevOps 성능 평가 (DORA)의 최고 수준인 Elite 등급의 변경 실패율(Change Failure Rate) 15% 이하(즉, 배포 성공률 85% 이상)를 목표 기준으로 설정했다. 이 기준을 달성하기 위해, 최종 프롬프트의 구성 요소인 템플릿(T)과 변수(Q, C)가 배포 성공률에 미치는 영향을 관찰하고 성공률 향상 요인을 도출하였다.

테스트에 사용한 MSA 애플리케이션은 상품을 위시리스트에 담고, 위시리스트에 담긴 상품을 주문하면 카카오로 로그인한 사용자에게 주문한 상품에 대한 문자를 보내는 선물하기 애플리케이션이다. 배포 성공 여부는 선물하기 애플리케이션의 모든 기능(로그인, 상품 조회, 위시리스트 추가, 주문, 메시지 전송 등)이 정상 작동하는 것을 기준으로 판정하였다.

4.2.1. 비교군 설정

아래 표 6은 비교 평가에 사용된 최종 프롬프트(P1~P4)의 검증 목표를 보여준다.

표 6 비교군 설정

최종 프롬프트	검증 목표
P1 (Non-RAG 베이스라인)	Non-RAG 환경에서 단순 질의의 성능 측정
P2 (Non-RAG + 상세 질의)	Non-RAG 환경에서 상세 질의의 구조 생성 능력 검증
P3 (RAG 적용)	RAG 적용 시 조직 내부 데이터 반영 정확성 능력 검증
P4 (RAG + 명시적 규칙)	명시적 규칙의 문법 및 정책 준수 효과 검증

해당 비교군 설계의 목표는 상세 질의의 효과(P1→P2), RAG 적용의 효과(P2→P3), 명시적 규칙 추가의 효과(P3→P4)를 단계적으로 검증하는 것이다. P1과 P2는 조직 내부 데이터(C)를 참조하지 않는(Non-RAG) 상황을, P3와 P4는 RAG를 통해 데이터를 참조하는 상황을 가정하여 각 요소가 배포 향상률에 미치는 영향을 관찰했다.

4.2.2. 실험 결과 및 분석

테스트 결과, 비교군 P1-P4에서 각각 배포 성공횟수와 성공률은 그림 64와 같이 나타났다.

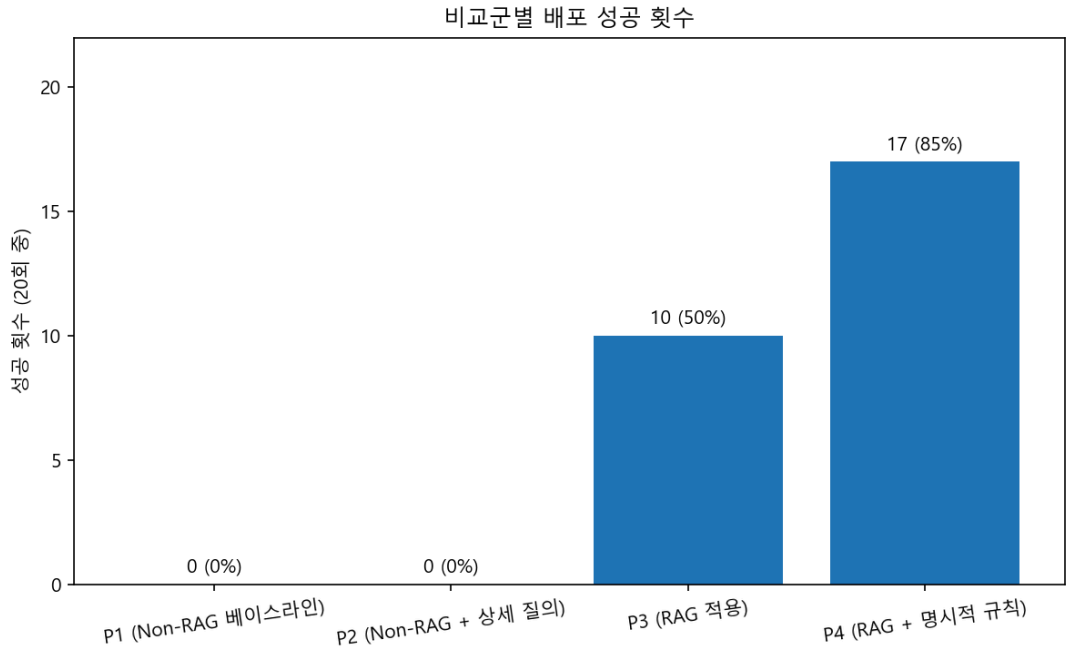


그림 64 비교군별 배포 성공 횟수 막대 그래프

- P1 (T1(Q1), 성공률 0%)

단순 질의(Q1)만 사용한 베이스라인에서는 LLM이 생성 범위를 특정하지 못해 ConfigMap, 일부 마이크로서비스 등 필수 리소스의 구조 자체를 누락하는 문제가 발생했다. 또한 참조 데이터가 없어 모든 값(포트, 이미지 이름 등)이 유효하지 않았다. 20회 시도 모두 배포 가능한 명세의 최소 요건조차 갖추지 못했다.

- P2 (T1(Q2), 성공률 0%)

P1의 리소스 누락 문제를 해결하기 위해 상세 질의(Q2)를 통해 필요한 모든 마이크로서비스를 명시하자, P1에서 발생했던 리소스 구조 누락 문제는 해결되었다. 따라서 상세 질의(Q2)를 통해 모든 리소스의 구조(Deployment/Service 등)를 생성할 수 있음을 확인했다. 하지만, RAG를 사용하지 않았기 때문에 조직 내부 데이터(C)를 참조할 수 없어 포트 번호, 환경 변수, 이미지 경로 등 구체적인 설정 값들을 모두 임의의 값으로 채워졌다. 결과적으로 구조만 있고 내용이 없는 명세가 되어 20회 모두 배포에 실패했다.

- P3 (T2(Q2 + C), 성공률 50%)

P2의 값 설정 오류를 RAG를 적용하여 해결하기 위해 조직 내부 데이터 문맥을 참조하도록 지시하는 템플릿(T2)과 조직 내부 데이터 문맥(C)을 최종 프롬프트에 구성하였다. 그 결과, P2에서 발생했던 설정 값 오류 문제가 대부분 해결되며 50%(10/20)의 배포 성공률을 보였다. RAG의 문맥 교정 절대 규칙(T2)에 따라 LLM은 검색된 조직 내부 데이터 문맥을 기반으로 정확한 포트 번호, 환경 변수 등을 명세에 올바르게 주입했다. 하지만 조직 정책을 준수하지 않거나 (Base64로 인코딩된 값임에도 Secret에서 stringData를 사용) ConfigMap/Secret 리소스에서 metadata.data와 같은 문법 오류가 10번의 배포 실패 오류 중 7번 발생하였다.

- P4 (T3(Q2 + C), 성공률 85%)

P3의 매포 명세 이력 데이터를 통해 발견된 정책·문법 오류를 해결하기 위해 명시적 규칙이 추가된 프롬프트 템플릿(T3)을 추가하자, 배포 성공률이 85%로 상승하며 DORA Elite 등급의 목표치(≥85%)를 달성하였다. 이를 통해 프롬프트 템플릿(T3)에 추가된 YAML 구조 절대 규칙(metadata.data 금지)과 정책 준수(조직에서 Base64로 인코딩된 값을 사용한다면, Secret에서는 data 사용)이 P3에서 발생한 문제를 해결할 수 있음을 확인하였다. 실패한 3번의 오류는 Deployment/Secret 리소스에서 type이 metadata의 하위 필드로 들어가는 오류가 1번, data가 metadata의 하위 필드로 들어가는 오류가 2번으로, 문법 오류가 주를 이루었다. 이는 프롬프트 템플릿(T3)에 명시적인 YAML 구조 규칙(metadata.data 금지)을 부여했음에도, LLM이 학습 과정에서 습득한 잘못된 구조적 패턴(예: metadata와 data 필드의 연관성)을 프롬프트 지침보다 우선시하는 경향이 있음을 시사한다.

표 7은 비교군 별 배포 결과와 주요 오류 유형을 나타낸다.

표 7 비교군 별 배포 결과와 주요 오류 유형

최종 프롬프트	배포 성공률	주요 오류 유형
P1	0% (0/20)	필수 마이크로서비스 리소스(Deployment/Service) 대량 누락
P2	0% (0/20)	리소스 구조는 생성되었으나, 이미지 이름, 환경 변수 등 값 할당 오류
P3	50% (10/20)	조직 정책·문법 위반 - Secret에 stringData 사용(모든 값 Base64 인코딩 후 stringData 필드 사용 규칙 위반), metadata.data의 YAML 문법 오류
P4	85% (17/20)	YAML 문법 오류 3회: (1) metadata.type 1회, (2) metadata.data 2회

표에서 알 수 있듯이, 배포 명세 생성의 정확도를 높이기 위해서는 (1) 사용자 질의는 구체적 질의로 기능·구성요소와 범위를 명확히 하여 리소스/서비스 누락을 줄이고, (2) RAG를 활용하여 조직 내부 데이터로부터 정확한 값을 주입하며, (3) 프롬프트 템플릿에 YAML 구조 불변식(metadata.data 금지) 및 조직 정책 준수(Secret 리소스에서 Base64 인코딩 후 data 사용) 등의 명시적인 규칙을 부여해 문법 오류와 정책 위반을 억제해야 한다는 결론을 도출할 수 있었다.

4.3. 정성평가

본 시스템의 효용성을 정성적으로 평가하기 위해, 도메인 지식을 LLM에 통합하는 대표적인 접근법인 파인튜닝과 정성적으로 비교·평가한다. 평가는 MSA 배포 자동화라는 목적을 고려하여, 다음과 같은 세 가지 기준으로 이루어진다.

- **최신 데이터 반영 수준:** MSA 환경의 구성 정보, 정책, 버전 등은 수시로 변경되므로, 최신 정보를 즉각적으로 반영할 수 있는 능력.
- **데이터 처리 및 운영 효율성:** 새로운 정보가 추가되거나 기존 정보가 변경될 때, 이를 시스템에 반영하는 데 드는 시간, 비용, 노력.
- **근거 추적 가능성:** 생성된 배포 명세의 답변이 어떤 내부 문서를 근거로 하는지 추적하고 제시할 수 있는 능력.

표 8은 각 기준에 대한 두 접근법의 운영상 특징을 요약한 것이다.

표 8 접근법별 비교표

비교 대상 / 평가 기준	제안하는 시스템	파인튜닝
최신 데이터 반영 수준	높음 (실시간에 가까운 반영)	고정적 (전체 재학습 필요)
데이터 처리 및 운영 효율성	높음 (문서 단위 즉시 갱신)	낮음 (대규모 학습 자원 소요)
근거 추적 가능성	가능 (검색된 문서를 근거로 제시)	불가능 (지식 내재화로 근거 추적 불가)

표에서 볼 수 있듯이, 파인튜닝은 학습 시점 이후 데이터가 반영되지 않아 고정적이며 최신 데이터를 반영하려면 재학습에 시간과 비용이 크게 소요되고, 특정 데이터셋·버전에 종속되는 한계가 있다. 또한, 지식이 모델 파라미터에 분산되어 내재화되기 때문에 특정 답변의 근거를 추적하는 것이 원천적으로 불가능하다. 반면 제안하는 시스템은 내부 문서를 벡터 저장소에 임베딩해 두고 질의 시점에 검색·주입하므로 최신 데이터를 즉시 반영할 수 있다. 또한, 문서 추가·갱신 시 재학습 없이 해당 문서의 벡터 임베딩만 다시 수행하면 되므로 운영 효율성이 높다. 무엇보다 LLM이 답변을 생성할 때 참조한 실제

문서를 사용자에게 제시하여 근거 추적이 가능하므로, 결과물의 신뢰도를 확보하고 환각 현상을 억제할 수 있다.

본 연구의 배경(1.1절)에서 서술했듯이, MSA 환경의 내부 데이터는 지속적으로 갱신되며 배포 담당자가 모든 도메인 지식을 보유하기 어려워 배포 명세를 정확하게 작성하는 데 어려움이 있다. 본 연구에서 제안하는 시스템은 지속적으로 갱신되는 데이터를 빠르게 반영하고 근거 추적 기능을 통해 배포 명세의 정확성을 사용자가 직접 검증할 수 있도록 보장한다. 따라서 본 시스템은 MSA 배포·운영의 어려움을 해결하는 효과적인 접근 방식이라고 할 수 있다.

5. 결론 및 향후 연구 방향

5.1. 결론

본 연구에서는 RAG 기술을 활용하여 컨테이너 기반 마이크로서비스의 초기 배포 및 운영 안정성을 향상시키는 컨테이너 기반 마이크로서비스 운영 환경 관리 지원 시스템을 구축하였다. 이 시스템은 복잡하고 지속적으로 변화하는 조직 내부 데이터를 효과적으로 반영하여 Kubernetes 배포 명세를 생성하고, 리소스 메트릭 및 에러 로그를 기반으로 개선 방안을 제시함으로써 기존 MSA 운영 방식의 한계를 극복하고자 하였다.

RAG를 통해 조직 내부 데이터를 반영하여 자동으로 배포 명세를 생성 함으로써, 배포 담당자의 도메인 지식 의존도를 낮출 수 있었고, 수작업으로 인한 오류 가능성 또한 줄일 수 있었다. 정량적 평가에서 사용자 질의, 조직 내부 데이터, 프롬프트 템플릿 세 요소를 적절히 구성했을 때 85%의 배포 성공률을 달성하며 제안 시스템의 유효성을 입증했다.

Filebeat, Logstash, Metricbeat 등의 로그 수집기를 통해 수집된 에러 로그와 리소스 메트릭을 수집하여, 배포 명세의 문제점을 진단하고 구체적인 개선안을 제안하는 기능을 구현했다. 운영 중 발생하는 문제에 동적으로 대응하여 MSA의 안정성과 리소스 사용 효율성을 동시에 향상시키는 데 기여 함으로써 단순히 로그 관찰 및 수집에 머물던 기존의 수동적인 시스템의 한계를 극복하였다.

정성적 평가를 통해 제안 시스템이 파인튜닝 방식보다 최신 데이터 반영 수준, 데이터 처리 및 운영 효율성, 근거 추적 가능성 측면에서 우수함을 확인했다. 특히, 생성된 결과물의 근거를 명확히 제시함으로써, 정확성이 중요한 배포명세 관리의 신뢰성을 확보하였다.

결론적으로, 본 연구에서 제시한 시스템은 RAG 기술을 컨테이너 기반 마이크로서비스 운영 환경에 성공적으로 접목하여, 배포 복잡성을 낮추고 운영 안정성을 강화하는 실질적인 해결책을 제시했다는 점에서 의의가 있다.

5.2. 향후 연구 방향

본 연구에서는 RAG 기반의 컨테이너 기반 마이크로서비스 배포 및 관리 지원 체계를 제안하였으나, 향후에는 CI/CD 파이프라인과의 연동을 통한 자동화 고도화가 필요하다. 구체적으로는 플랫폼 자체의 빌드, 배포, 테스트 과정을 파이프라인에 통합하여, 코드 변경 시 자동으로 RAG 모듈이 최신 데이터와 환경을 반영하도록 하고, 생성된 배포 명세의 정확성을 지속적으로 검증할 수 있는 체계를 마련할 것이다. 이를 통해 연구 성과의 재현성·운영 용이성을 높이고, 실제 산업 현장에서 활용 가능한 완성도 높은 플랫폼으로 발전시키는 것을 향후 연구 방향으로 설정한다.

6. 참고 문헌

[1] J. Lewis and M. Fowler, (2014). Microservices [Online]. Available: https://martinfowler.com/articles/microservices.html (downloaded 2025, Sep. 17)

[2] Microsoft, (2025). *Microservices architectural style* [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices (downloaded 2025, Sep. 19)

[3] Hostinger, (2025). LLM statistics 2025: Comprehensive insights into market trends and integration [Online]. Available: https://www.hostinger.com/tutorials/llm-statistics (downloaded 2025, Sep. 17)

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," Advances in Neural Information Processing Systems, Vol. 33, pp. 9459-9474, 2020.

[5] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv preprint arXiv:2312.10997, 2023.

[6] Atlassian, (2023). DORA Metrics: How to Measure DevOps Performance [Online]. Available: https://www.atlassian.com/devops/frameworks/dora-metrics (downloaded 2025, Sep. 17)

[7] D. Merkel, "Docker: lightweight linux containers for consistent development and deployment," *Linux journal*, Vol. 2014, No. 239, p. 2, 2014.

[8] A. Verma, L. Pedrosa, M. R. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at Google with Borg," Proc. of the Tenth European Conference on Computer Systems (EuroSys '15), 2015.

[9] J. Wang, W. Wang, J. Li, R. Wang, and Z. Lin, "A comprehensive survey on approximate nearest neighbor search," *ACM Computing Surveys (CSUR)*, Vol. 54, No. 6, pp. 1-39, 2021.

[10] M. Yin, D. Wang, C. Li, Z. Wang, Y. Wang, Z. Liu, H. Liu, J. Wang, T. Wang, J. Lou, W. Dou, and Y. Li, "A Survey on LLM-based Autonomous Agents," arXiv preprint arXiv:2308.11432, 2023.

[11] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, O'Reilly Media, Inc., 2015.

[12] The Kubernetes Authors, "Objects In Kubernetes," [Online]. Available: https://kubernetes.io/ko/docs/concepts/overview/working-with-objects/ (downloaded 2025, Sep. 17)

[13] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Computing Surveys*, Vol. 55, No. 9, pp. 1-35, 2023.

[14] D. Kim, D. Yun, J. Park, and K. Yeom, "A Script Generation Method for Microservice Deployment in a Container Orchestration Environment," *Journal of KIISE*, Vol. 46, No. 7, pp. 682-689, 2019.